System Design Interview Prep: Complete Guide

Abhishek Kumar

October 2026

- Complete System Design Interview Guide
- SystemDesign_Prep: Index (start here)
  - Coverage Map: every architecture topic and where it lives
  - Study Method for Depth (what to do with these files)
- System Design Interview Prep: 8-Week Plan
  - 1. How to Use These Files
  - 2. The 8-Week Roadmap
  - 3. Daily Routine (90 minutes)
  - 4. Self-Grading Rubric (use after each mock)
  - 5. Progress Tracker
  - 6. Resources (pick one main per area; don’t collect too many)
  - 7. Interview Day Checklist
- High-Level System Design (HLD) Guide
  - Part A: The Interview Framework (45–60 min)
  - Part B: Building Blocks (know the trade-offs, not just the names)
  - Part C: Estimation Cheat Sheet
  - Part D: Case Studies
  - Part E: Common Mistakes (check yourself)
- Low-Level System Design (LLD) Guide
  - Part A: The LLD Interview Framework (45 min)
  - Part B: Principles and Patterns
  - Part C: LLD Problems
  - Part D: Quick Templates
  - Part E: Practice Order
- Q&A Bank, Behavioral Prep, and Your Resume Stories
  - Part A: Rapid-Fire Questions (answer aloud in 60–90 seconds)
  - Part B: Common Behavioral Prompts (use the STAR format)
  - Part C: Turning Your Resume into System Design Stories
  - Part D: Turning Projects into a System Design Answer (the template)
  - Part E: Questions to Ask the Interviewer
  - Part F: Final Week Checklist
- 04. Deep Foundations: Networking, Time, Storage Engines, Indexes, Transactions
  - 4.1 Latency and Throughput Numbers (memorize the orders of magnitude)
  - 4.2 Networking
  - 4.3 Time, Clocks, and Ordering
  - 4.4 Storage Engines
  - 4.5 Transactions and Isolation
  - 4.6 Data Modeling Essentials
- 05. Deep Distributed Systems: Replication, Partitioning, Consensus, Consistency, Transactions
  - 5.1 Why Distributed Systems Are Hard (the fallacies)
  - 5.2 Replication
  - 5.3 Partitioning (Sharding)
  - 5.4 Consistency Models (know the spectrum)
  - 5.5 CAP and PACELC (precise version)
  - 5.6 Consensus, Leader Election, Coordination
  - 5.7 Failure Detection and Membership
  - 5.8 Idempotency and Exactly-Once
  - 5.9 Distributed Transactions

    - 5.10 Logical Structures and Conflict Resolution
    - 5.11 Distributed Systems Patterns Glossary
    - 5.12 Interview Questions on This Topic (short answers)
- 06. Deep Messaging, Streaming, and Caching
    - 6.1 Messaging Models
    - 6.2 Kafka in Depth
    - 6.3 Other Brokers (know the differences)
    - 6.4 Stream Processing
    - 6.5 Data Lake Table Formats (Iceberg, Delta, Hudi)
    - 6.6 Caching in Depth
    - 6.7 CDN Deep Dive
    - 6.8 Interview Questions (with short model answers)
- 07. Deep APIs, Resilience, Observability, Security, Deployment
    - 7.1 API Design
    - 7.2 Resilience Patterns
    - 7.3 Rate Limiting and Quotas (depth beyond HLD guide)
    - 7.4 Observability
    - 7.5 Security
    - 7.6 Deployment and Release Engineering
    - 7.7 Multi-Region and Disaster Recovery
    - 7.8 Cost Awareness (say this in senior interviews)
    - 7.9 Testing for Distributed Systems
    - 7.10 Interview Questions (short answers)
- 08. Architecture Patterns and Reference Designs
    - 8.1 Architectural Styles
    - 8.2 Reference Architectures for Common Systems
    - 8.3 Pattern Selection Cheat Sheet
    - 8.4 How to Pick a Database (decision guide)
    - 8.5 Interview Technique for Architecture Questions
- 09. Deep Question Bank with Model Answers
    - A. Scalability and Capacity
    - B. Databases and Storage
    - C. Consistency and Transactions
    - D. Distributed Systems
    - E. Messaging and Streams
    - F. Caching
    - G. APIs and Communication
    - H. Reliability and Operations
    - I. Security
    - J. Design Judgment (senior-level)
    - K. Your Resume Topics (be ready for deep follow-ups)
    - L. Rapid Scenario Drills (30 seconds each)
    - M. Behavioral Prompts for Architects
- 11. Full HLD Walkthroughs, Part 1 (Cases 1–5)
    - Case 1: URL Shortener
    - Case 2: Distributed Rate Limiter
    - Case 3: Distributed Cache
    - Case 4: Notification System
    - Case 5: Chat Application (WhatsApp / Slack)
- 12. Full HLD Walkthroughs, Part 2 (Cases 6–10)
    - Case 6: News Feed / Timeline (Twitter, Instagram)
    - Case 7: Typeahead / Autocomplete
    - Case 8: Ride Sharing (Uber / Lyft)
    - Case 9: Payment System
    - Case 10: Web Crawler
- 13. Full HLD Walkthroughs, Part 3 (Cases 11–15)
    - Case 11: Distributed File Storage (Dropbox / Google Drive)

    - Case 12: Metrics / Monitoring System (Prometheus / Datadog)
    - Case 13: Video Streaming (YouTube / Netflix)
    - Case 14: Distributed Job Scheduler / Workflow Engine
    - Case 15: Event Pipeline / Log Analytics
- 14. Resume Alignment (based on your current PDF)
    - 1. Facts to Know Cold
    - 2. Corrections to Make Before Any Interview
    - 3. Story Map (one diagram each, 5 minutes)
    - 4. Gaps an Interviewer May Probe (prepare an honest answer)
    - 5. Resume Keywords vs Deep Files
- 15. Advanced LLD: More Problems and Deeper Techniques
    - Problem A: Chess
    - Problem B: Library Management
    - Problem C: Food Delivery (Swiggy/Zomato-style)
    - Problem D: Meeting Room Scheduler
    - Problem E: Snake and Ladder
    - Problem F: Multi-Level Cache (LLD + Concurrency)
    - Advanced LLD Techniques (concepts to name)
    - Practice Set (after file 02 problems)

Complete System Design Interview Guide

Contents

1. Index and how to use this guide
2. 8-Week study plan
3. HLD guide: framework, building blocks, estimation, case summaries
4. LLD guide: principles, patterns, 10 problems
5. Q&A and resume stories
6. Deep foundations: networking, time, storage, transactions
7. Deep distributed systems
8. Deep messaging, streaming, caching
9. Deep APIs, resilience, observability, security
10. Architecture patterns and reference designs
11. Deep question bank (85 questions)
12. HLD walkthroughs, part 1 (cases 1–5)
13. HLD walkthroughs, part 2 (cases 6–10)
14. HLD walkthroughs, part 3 (cases 11–15)
15. Resume alignment
16. Advanced LLD

SystemDesign_Prep: Index (start here)

Read in this order.

#	File	Covers	Time
00	00_Study_Plan.md	8-week schedule, routine, rubric, tracker	15 min
01	01_HLD_Guide.md	Interview framework, building blocks overview, estimation, 15 case summaries	3 h
02	02_LLD_Guide.md	SOLID, patterns, 10 LLD problems with code	3 h
03	03_QA_and_Your_Stories.md	Rapid-fire Q&A, behavioral prep, resume project stories	1 h
04	04_Deep_Foundations.md	Latency numbers, networking, time, storage engines, indexes, transactions and isolation	3 h
05	05_Deep_Distributed_Systems.md	Replication, partitioning, consistency models, CAP/PACELC, consensus, locks, failure detection, idempotency, distributed transactions, CRDTs	4 h
06	06_Deep_Messaging_Streaming_Caching.md	Kafka internals, stream processing, lakehouse tables, Redis internals, caching patterns, CDN	3 h
07	07_Deep_APIs_Resilience_Observability_Security.md	REST/gRPC/GraphQL, resilience patterns, rate limiting, SLOs, security, deployment, DR	3 h
08	08_Architecture_Patterns_and_Reference_Designs.md	Monolith vs microservices, event-driven, CQRS, event sourcing, cells, 13 reference architectures, DB selection	2.5 h
09	09_Deep_Question_Bank.md	85 questions with model answers, scenario drills, behavioral	2 h
11	11_HLD_Walkthroughs_Part1.md	Full walkthroughs: URL shortener, rate limiter, distributed cache, notifications, chat	4 h
12	12_HLD_Walkthroughs_Part2.md	Full walkthroughs: news feed, typeahead, ride sharing, payments, web crawler	4 h
13	13_HLD_Walkthroughs_Part3.md	Full walkthroughs: file storage, metrics, video streaming, job scheduler, event pipeline	4 h
14	14_Resume_Alignment.md	Your current resume PDF: numbers to know, corrections, story map, gaps	45 min
15	15_LLD_Advanced.md	Chess, library, food delivery, meeting rooms, snake and ladder, multi-level cache with single-flight	4 h

Coverage Map: every architecture topic and where it lives

Foundations: latency numbers (04 §4.1), networking (04 §4.2, 07 §7.1), time (04 §4.3, 05 §5.7) **Storage:** engines (04 §4.4), indexes (04 §4.4.6), data modeling (04 §4.6), columnar and lakehouse (04 §4.4.5, 06 §6.5) **Transactions:** ACID, isolation, anomalies (04 §4.5), distributed transactions and sagas (05 §5.9), outbox and inbox (05 §5.8.3–4) **Distribution:** replication (05 §5.2), partitioning (05 §5.3), consistency models (05 §5.4), CAP and PACELC (05 §5.5), consensus (05 §5.6),

membership and gossip (05 §5.7), CRDTs (05 §5.10) **Messaging and streams:** queues vs logs (06 §6.1), Kafka (06 §6.2), other brokers (06 §6.3), stream processing and windows (06 §6.4), Lambda and Kappa (06 §6.4.4) **Caching and CDN:** Redis internals (06 §6.6.2), eviction (06 §6.6.3), consistency patterns (06 §6.6.4), stampede and penetration (06 §6.6.5–6), CDN (06 §6.7, 04 §4.2.8) **Load balancing and networking:** L4/L7, algorithms (04 §4.2.7), DNS (04 §4.2.2), real-time transport (04 §4.2.6) **APIs:** REST, gRPC, GraphQL, versioning, pagination, webhooks (07 §7.1) **Resilience:** timeouts, retries, circuit breaker, bulkhead, load shedding (07 §7.2), rate limiting (07 §7.3, 03 Case 2) **Observability:** metrics, logs, traces, SLO, error budget, incidents (07 §7.4) **Security:** AuthN/AuthZ, OAuth2/OIDC, JWT, mTLS, encryption, attacks (07 §7.5) **Deployment and DR:** canary, blue-green, migrations, multi-region, cells, RTO/RPO (07 §7.6–7.7, 08 §8.1) **Architecture styles:** monolith, microservices, event-driven, event sourcing, CQRS, hexagonal, serverless, service mesh, BFF (08 §8.1) **Systems:** 13 reference architectures (08 §8.2); 15 full walkthroughs (11, 12, 13); case summaries (01 Part D) **Your resume:** 14_Resume_Alignment.md (use the PDF numbers, not the older .tex) **LLD:** principles, patterns, concurrency, 10 problems (02), 6 advanced problems and techniques (15)

Study Method for Depth (what to do with these files)

1. Read each deep file once, slowly. Write one-line summaries in your own words.
2. For each section, answer aloud: “what, how it works, trade-off, failure mode.”
3. Do the 85 questions in file 09: say your answer, then check. Mark those you missed.
4. Redo two HLD cases per week with the full framework; use the deep files to justify each choice.
5. Before the interview, re-read file 03 and your weakest deep file.

System Design Interview Prep: 8-Week Plan

Target: Backend / SDE-2 level system design (HLD + LLD), with a strong base for senior roles. **Starting point (from your resume):** 2+ years backend, Kafka at 1M+ events/day (per your current PDF resume), Temporal workflows, Druid/Iceberg, GraphQL APIs, a distributed cache built from scratch, Redis-based real-time features. **Weekly time:** about 10 hours. Weekdays 1.5 hrs, weekend 3–4 hrs.

---

1. How to Use These Files

File	What it is	When to use
`00_Study_Plan.md`	This plan, routine, tracker	Every day
`01_HLD_Guide.md`	Interview framework, estimation, building blocks, 15 case studies	Weeks 1–5
`02_LLD_Guide.md`	OOP principles, design patterns, 10 LLD problems with code	Weeks 6–7
`03_QA_and_Your_Stories.md`	Rapid-fire Q&A, how to tell your resume projects as system design	Week 8 and the day before any interview

---

2. The 8-Week Roadmap

Week 1: Foundations (fundamentals you will be tested on)

- Read/skim: Alex Xu, *System Design Interview Vol 1*, Ch. 1–4 (framework, rate limiter, consistent hashing, key-value store).
- Learn: latency numbers, estimation, the interview framework (`01_HLD_Guide.md` Part A and B).
- Practice: 3 estimation drills (see Part C of the HLD guide).
- Goal: you can explain consistent hashing and CAP without notes.

Week 2: Data layer

- Read: *Designing Data-Intensive Applications* (DDIA) Ch. 2, 3, 5, 6 (data models, storage engines, replication, partitioning).
- Learn: B-tree vs LSM, indexes, replication (leader/follower), sharding, SQL vs NoSQL, transactions and isolation levels.
- Practice: design a URL shortener (Case 1) end to end, timed at 45 minutes.

Week 3: Messaging, caching, scale patterns

- Read: DDIA Ch. 8, 9, 11 (distributed issues, consistency, stream processing).
- Learn: Kafka internals (partitions, offsets, consumer groups, delivery semantics), caching strategies, idempotency, outbox, sagas.
- Practice: Cases 2 (rate limiter), 3 (distributed cache), 4 (notification system).

Week 4: Core HLD case studies

- Practice: Cases 5 (chat), 6 (news feed), 7 (typeahead), 8 (ride sharing).
- Do 2 timed mocks with a friend or on your own (record yourself).

Week 5: Advanced HLD

- Practice: Cases 9 (payments), 10 (web crawler), 11 (file storage), 12 (metrics system), 13 (video streaming), 14 (job scheduler), 15 (event pipeline).
- Do 2 more mocks. Target: finish the framework in time with no skipped steps.

Week 6: LLD foundations and patterns

- Read: `02_LLD_Guide.md` Part A and B (SOLID, patterns).
- Practice: Parking lot, LRU cache, vending machine (State pattern), rate limiter LLD.

Week 7: LLD problems and concurrency

- Practice: elevator, logger, Splitwise, movie ticket booking (concurrency), notification (Observer), tic-tac-toe.
- Write the code in Python or Java (whichever you're fastest in), without an IDE's autocomplete.

Week 8: Mocks, weak spots, your stories

- Do 4 full mocks (2 HLD, 2 LLD), each with a 1-page written self-review.
- Prepare your resume project stories using `03_QA_and_Your_Stories.md`.
- Rehearse the 20 rapid-fire questions out loud.

---

3. Daily Routine (90 minutes)

Block	Time	Activity
Warm-up	10 min	Review yesterday's rapid-fire questions (no notes)
Learn	30 min	Read one concept or chapter; write 5–10 bullets in your own words
Practice	40 min	Timed problem (45 min budget) or LLD coding
Review	10 min	Compare with the reference; list 3 gaps

Rules: 1. Draw on paper or a whiteboard, never only in your head. Talk out loud as if an interviewer is listening. 2. Time every practice. Use a timer and stop at the limit. 3. Never read the solution before attempting the problem for at least 20 minutes. 4. Every week, redo one old problem from scratch without notes.

---

4. Self-Grading Rubric (use after each mock)

Score each 1–5:

Dimension	What a 5 looks like
Requirements	Asked 4–6 clarifying questions; stated functional and non-functional requirements; agreed scope out loud
Estimation	Computed QPS (average and peak), storage per year, bandwidth; used the numbers to make a decision
API and data model	Clear endpoints with inputs and outputs; chose storage with a reason
High-level design	Clean diagram; each component has a job; data flow for the main use case is clear
Deep dives	Went deep on 2+ bottlenecks (scaling, hot keys, failures, consistency) with trade-offs
Trade-offs	Named alternatives and why you chose one; said what you gave up
Communication	Structured; checked in with the interviewer; managed time

Passing bar for SDE-2: average 3.5+ with no dimension below 3. **Senior bar:** average 4+ with strong deep dives and proactive failure-mode thinking.

---

5. Progress Tracker

Copy this table into a note and fill it in.

#	Topic	Type	Date	Time taken	Self-score (avg)	Weak spot
1	URL shortener	HLD				
2	Rate limiter	HLD				
3	Distributed cache	HLD				
4	Notification system	HLD				
5	Chat app	HLD				
6	News feed	HLD				
7	Typeahead	HLD				
8	Ride sharing	HLD				

9	Payment system	HLD				
10	Web crawler	HLD				
11	File storage (Dropbox)	HLD				
12	Metrics / monitoring	HLD				
13	Video streaming	HLD				
14	Job scheduler	HLD				
15	Event pipeline	HLD				
16	Parking lot	LLD				
17	LRU cache	LLD				
18	Vending machine	LLD				
19	Rate limiter	LLD				
20	Elevator	LLD				
21	Logger	LLD				
22	Splitwise	LLD				
23	Movie booking	LLD				
24	Notification (Observer)	LLD				
25	Tic-tac-toe	LLD				

---

6. Resources (pick one main per area; don’t collect too many)

Core reading (choose these): - *Designing Data-Intensive Applications* (Martin Kleppmann): the single best book for depth. Read it in weeks 2–3. - *System Design Interview* Vol 1 and 2 (Alex Xu): good for structure and common questions.

Practice: - Hello Interview (hellointerview.com): free, structured HLD and LLD walkthroughs. - ByteByteGo: short visual explainers. - The “system-design-primer” GitHub repo: reference only.

LLD: - Refactoring Guru (design patterns, with Python and Java examples). - *Head First Design Patterns*: optional, for intuition.

Check the current content of these resources yourself before relying on them; they update often.

---

7. Interview Day Checklist

- ☐ Re-read your resume project stories (`03_QA_and_Your_Stories.md`, Part C).
- ☐ Revise the estimation cheat sheet (HLD guide, Part C).
- ☐ Have a 4-step framework in mind: Requirements → Estimates → API/Data → High-level → Deep dives.
- ☐ Ask the interviewer at the start: “Should I focus on the core flow or go broad?”
- ☐ Watch the clock: at 35 min you should be in deep dives.
- ☐ End with a summary: what you built, what you’d improve next, and the main trade-off you made.

High-Level System Design (HLD) Guide

Part A: The Interview Framework (45–60 min)

Step	Time (45 min)	What you do	Output
1. Requirements	5 min	Ask questions; split functional vs non-functional	Scope list
2. Estimation	3–5 min	QPS, storage, bandwidth	3–4 numbers
3. API and data model	5 min	Endpoints, entities, storage choice	API list, schema
4. High-level design	10–12 min	Draw the components and the main flow	Diagram
5. Deep dives	15 min	2–3 bottlenecks or hard parts	Trade-offs
6. Wrap-up	3 min	Failures, monitoring, what's next	Summary

Step 1: Requirements questions to ask

- **Users:** who uses it, how many (DAU/MAU), where (regions)?
- **Core features:** what must work in v1? What is out of scope?
- **Scale:** read vs write ratio, peak traffic, data size growth.
- **Non-functional:** latency target, availability (99.9% vs 99.99%), consistency (strong vs eventual), durability.
- **Constraints:** real-time or batch? Mobile or web? Compliance (GDPR, PCI)?

Write requirements on the board and confirm them out loud: "So I'm designing X for Y users with Z latency. Is that right?"

Step 4: How to draw

- Start with the simplest working design, then add components only when a requirement needs them.
- Standard boxes: Client → DNS/CDN → Load balancer → API servers → Cache → DB; plus queues, workers, and object storage as needed.
- Draw one **main flow** end to end (e.g., "user posts a tweet" or "driver sends location").
- Name the protocol on arrows where it matters (HTTP, gRPC, WebSocket, Kafka topic).

Step 5: Deep dives: pick the right ones

Good deep-dive topics: hot keys, a single DB becoming the bottleneck, fan-out, failure of one component, race conditions, exactly-once vs at-least-once, data partitioning key, cache invalidation, multi-region.

Phrases that score points

- "The bottleneck here will be…, so I'll…"
- "The trade-off is X vs Y. I'm choosing X because our requirement is…"
- "If this component fails, the impact is… and we handle it by…"
- "At 10× scale, the first thing that breaks is…"

Part B: Building Blocks (know the trade-offs, not just the names)

B1. DNS, CDN, Load Balancers

- **CDN:** caches static assets and sometimes API responses near users. Use for images, video, JS/CSS.
- **Load balancer:** L4 (TCP/UDP, fast) vs L7 (HTTP, can route by path/header).
- **Algorithms:** round robin, least connections, consistent hashing (for sticky/cache-affine routing), weighted.
- **Health checks** remove unhealthy nodes. **API gateway** adds auth, rate limiting, routing, and request logging.

B2. Caching

- **Patterns:**
  - *Cache-aside:* app reads cache, falls back to DB, then populates cache. Most common.
  - *Write-through:* write to cache and DB together. Consistent, slower writes.

    - *Write-behind (write-back):* write to cache, flush to DB later. Fast, risk of data loss.
    - *Read-through:* the cache loads from DB on miss, handled by the cache layer.
- **Eviction:** LRU (most common), LFU, TTL.
- **Problems:**
    - *Cache stampede / thundering herd:* many misses hit the DB at once. Fix with locking/single-flight, staggered TTLs, or early refresh.
    - *Hot keys:* one key gets most traffic. Fix with local in-process cache, key replication (`key#1..N`), or splitting the value.
    - *Invalidation:* the hardest part. Use TTLs plus explicit invalidation on writes, or versioned keys.
- **Where:** client, CDN, app-local (e.g., Caffeine), distributed (Redis, Memcached).

B3. Databases

- **SQL (PostgreSQL, MySQL):** ACID, joins, flexible queries. Scale vertically first, then read replicas, then sharding.
- **NoSQL:**
    - Key-value (Redis, DynamoDB): fast point lookups.
    - Wide-column (Cassandra, HBase): high write throughput, time-series-like.
    - Document (MongoDB): flexible schema.
    - Graph (Neo4j): relationship-heavy queries.
- **Storage engines:**
    - *B-tree* (PostgreSQL, MySQL InnoDB): read-optimized, in-place updates.
    - *LSM tree* (Cassandra, RocksDB): write-optimized, compaction costs.
- **Indexes:** speed reads, slow writes. Composite index order matters (leftmost prefix). Covering index avoids table lookups.
- **Replication:**
    - Leader–follower: simple; replication lag causes stale reads.
    - Multi-leader: multi-region writes; conflict resolution needed.
    - Leaderless (Dynamo-style): quorum reads/writes.
- **Sharding:** split data by a key. Choose a key with high cardinality and even access. Problems: hot shards, cross-shard joins and transactions, resharding (use consistent hashing or a directory).

B4. Consistent Hashing

- Hash nodes and keys onto a ring; a key belongs to the next node clockwise.
- Adding or removing a node moves only about `1/N` of the keys.
- **Virtual nodes** (e.g., 100–200 per physical node) even out the load.
- Used by: Dynamo, Cassandra, Memcached clients, Redis Cluster (uses hash slots, a variant).
- *Your resume project* implemented this with 200 virtual nodes. Be ready to explain it.

B5. Replication and Quorum

- Leaderless with N replicas: write quorum W, read quorum R.
- **R + W > N** gives overlap, so reads see the latest write (strong-ish consistency).
- Example: N=3, W=2, R=2.
- Sloppy quorum and hinted handoff help availability during failures.

B6. CAP and PACELC

- **CAP:** during a network partition you choose consistency (C) or availability (A).
- **PACELC:** if Partition, choose A or C; Else (normal operation), choose Latency or Consistency.
- Say in interviews: “This is a payment ledger, so I choose CP. A product-view counter is fine with AP.”

B7. Message Queues and Streams (Kafka)

- **Kafka:** partitioned, ordered per partition, retained log. Consumers track offsets. Consumer groups share partitions.
- **Delivery semantics:**
    - At-most-once: may lose messages.
    - At-least-once: may duplicate. The default in most systems. Requires **idempotent consumers**.
    - Exactly-once: possible within Kafka transactions and idempotent producers; end-to-end still needs idempotent sinks.
- **Partition key:** choose so related events stay in order (e.g., `user_id`) and load is spread.

- **Consumer lag:** a key metric. Rising lag means consumers are too slow.
- **Dead letter queue (DLQ):** holds messages that fail after retries.
- **Ordering:** only within a partition. Global ordering is expensive.
- **Backpressure:** bounded queues, consumer scaling, and rate limiting upstream.

B8. Idempotency, Outbox, Sagas

- **Idempotency key:** client sends a unique key; server stores the result and returns it on retry. Needed for payments and anything with retries.
- **Transactional outbox:** write the event to an outbox table in the same DB transaction as the business change; a relay publishes it to Kafka. Avoids "DB committed but event lost."
- **Saga:** a sequence of local transactions with compensating actions on failure. Choreography (events) or orchestration (a coordinator, e.g., Temporal). Use for multi-service workflows like booking + payment + notification.

B9. Rate Limiting

Algorithm	How	Pros	Cons
Fixed window	Counter per time window	Simple, cheap	Burst at window edges
Sliding window log	Store timestamps	Accurate	Memory heavy
Sliding window counter	Weighted two windows	Good balance	Approximate
Token bucket	Tokens refill at rate r, capacity b	Allows bursts	Tuning
Leaky bucket	Constant outflow queue	Smooth output	Queue latency

- Distributed: store counters in Redis; use a Lua script for atomic check-and-increment.
- Return `429` with `Retry-After`. Rate limit per user, per IP, and per API key.

B10. Unique IDs

- **Auto-increment:** doesn't scale across shards.
- **UUID v4:** unique, random (poor index locality).
- **Snowflake:** 64-bit: timestamp + machine ID + sequence. Time-sortable, no coordination.
- **Ticket server / DB range allocation:** each worker grabs a range of IDs.

B11. Search

- **Inverted index:** term → list of document IDs. Used by Elasticsearch / OpenSearch.
- **Typeahead:** prefix-based trie or precomputed top-k per prefix, cached.

B12. Geospatial

- **Geohash:** encodes lat/long into a string; shared prefix means nearby. Easy to shard.
- **Quadtree:** recursively split space by density. Good for uneven distribution.
- **S2 / H3:** Google/Uber cell systems used in production.

B13. Blob Storage and CDN

- Store large files (images, video, backups) in S3-like object storage. Store only metadata in the DB.
- Use pre-signed URLs for upload/download. Multipart upload for big files.
- Video: transcode to multiple bitrates (adaptive streaming: HLS/DASH), serve via CDN.

B14. Observability

- **Metrics:** RED (Rate, Errors, Duration) for services; USE (Utilization, Saturation, Errors) for resources.
- **Logs and traces:** correlation IDs across services.
- **Alerts:** on SLO burn rate, not raw CPU.

B15. Security basics

- AuthN (who you are) vs AuthZ (what you can do). OAuth2/OIDC for login; JWT for stateless access tokens; RBAC for roles.
- Encrypt in transit (TLS) and at rest. Never store raw passwords (bcrypt/argon2).

B16. Failure handling (say this in every interview)

- **Retries** with exponential backoff and jitter.
- **Timeouts** on every network call.
- **Circuit breaker:** stop calling a failing dependency for a while.
- **Bulkheads:** isolate resource pools per dependency.
- **Graceful degradation:** serve stale or partial data.
- **Replication and multi-AZ** for availability.

Part C: Estimation Cheat Sheet

Powers of 2 / 10

Power	Approx	Unit
2^10	1 thousand	KB
2^20	1 million	MB
2^30	1 billion	GB
2^40	1 trillion	TB

Time

- 1 day ≈ 86,400 s ≈ **10^5 s** (use this for quick math)
- 1 month ≈ 2.5 × 10^6 s
- 1 year ≈ 3 × 10^7 s

Latency (order of magnitude)

- L1 cache: ~1 ns
- Main memory: ~100 ns
- SSD random read: ~100 µs
- Same-DC round trip: ~0.5 ms
- HDD seek: ~10 ms
- Cross-continent round trip: ~150 ms

Formulas

- **Average QPS** = (DAU × actions per user per day) / 86,400
- **Peak QPS** ≈ 2–3 × average
- **Storage per year** = (writes per day × size per record × 365)
- **Bandwidth** = QPS × average payload size
- **Read:write ratio** drives caching and replica decisions.

Worked example: URL shortener

- 100M new URLs/month → 100M / 2.5M s ≈ **40 writes/s** (average), peak ~100.
- Read:write = 100:1 → **4,000 reads/s** average, peak ~10,000.
- Each record ~500 bytes → 100M × 12 × 500 B ≈ **600 GB/year**.
- Conclusion: a single relational DB could handle writes, but read cache and sharding planning are needed later.

Practice: do this estimate for every case study below. Interviewers care more about the reasoning than the exact number.

Part D: Case Studies

Each case gives: requirements, scale, main flow, components, deep dives, and common mistakes. Use the framework from Part A.

Case 1: URL Shortener (TinyURL)

Requirements: shorten a long URL; redirect; optional custom alias; expiry; analytics (basic). **Scale:** 100M URLs/month, 100:1 read:write. **API:** - `POST /urls` {long_url, alias?, ttl?} → {short_url} - `GET /{code}` → 301/302 redirect

Data model: `(short_code PK, long_url, created_at, expires_at, user_id)`. KV store works well. **Short code:** base62 of a unique

ID (Snowflake or range allocator), 7 chars gives 62^7 ≈ 3.5 trillion codes. **Flow:** write → generate ID → base62 → store → return. Read → cache lookup → DB on miss → redirect. **Components:** LB → app servers → Redis (cache-aside, 80/20 rule) → KV DB (sharded by code) → analytics via Kafka. **Deep dives:** - ID generation without collisions across servers (range allocation, Snowflake). - Hot links going viral: cache with TTL; CDN for redirects. - 301 (cacheable, less analytics) vs 302 (analytics-friendly). **Mistake to avoid:** hashing the long URL (MD5) and taking the first 7 chars, without handling collisions.

Case 2: Distributed Rate Limiter

Requirements: limit requests per user/API key/IP; return 429; low overhead (<1 ms); works across many app servers. **Scale:** 1M users, 100K req/s overall. **Where:** at the API gateway or as middleware. Store counters in Redis. **Algorithm:** token bucket with a Lua script in Redis (atomic). Or sliding window counter for simpler limits. **Deep dives:** - Race conditions: use Lua script or `INCR` with `EXPIRE` in a transaction. - Redis failure: fail open (allow traffic) vs fail closed. Choose per use case; usually fail open for public APIs. - Multi-region: per-region limits, or a global limit with async sync (accept some overage). - Hot users: separate limits per tier; local in-memory pre-check to reduce Redis load. **Mistake to avoid:** using a database for counters.

Case 3: Distributed Cache (like Redis or Memcached)

Requirements: get/set/delete with TTL; horizontal scaling; high availability; sub-ms latency. **Scale:** 1 TB data, 500K ops/s. **Components:** - Client-side or proxy-based routing using consistent hashing. - Partitions with replicas (master–slave). Failover via a coordinator (ZooKeeper, etcd) or gossip. **Deep dives:** - Consistent hashing with virtual nodes; rebalancing on node add/remove (only ~1/N keys move). - Replication: async master → replica (fast, may lose recent writes on failover). Sync is slower. - Eviction: LRU with approximate tracking (sample k keys). - Hot keys and cache stampede. - Persistence: snapshot (RDB) and append log (AOF). **Link to your project:** explain how your Node.js cache handled node churn, why you chose 200 vnodes, and what the failover looked like.

Case 4: Notification System

Requirements: push, SMS, email, in-app; templates; user preferences; rate limits per user; retries; tracking delivery status. **Scale:** 10M notifications/day, bursts 10× during events. **Flow:** service → API → validate and dedupe → queue (Kafka, one topic per channel) → workers per channel → third-party provider (APNs, FCM, Twilio, SES) → status events. **Deep dives:** - Priority: separate queues for OTP/transactional vs marketing, so promotions don't delay OTPs. - Idempotency: notification ID as key; providers may duplicate. - Retries with backoff; DLQ after N attempts. - User preference and quiet hours: check before sending. - Fan-out to millions (broadcast): batch, rate-limit to provider limits. - Template rendering: cache compiled templates.

Case 5: Chat Application (WhatsApp / Slack)

Requirements: 1:1 and group chat; delivery receipts; online presence; message history; multi-device; end-to-end encryption (optional). **Scale:** 50M DAU, 10 messages per user per day → 500M msgs/day ≈ 6K msgs/s average. **Components:** - WebSocket gateway servers (stateful connection); a connection registry maps user → gateway server (Redis). - Message service writes to a store (wide-column or sharded DB keyed by `chat_id`, ordered by time/Snowflake ID). - Pub/sub (Kafka or Redis pub/sub) routes messages to the gateway holding the recipient's connection. - Offline users: store and push notification. **Deep dives:** - Ordering: per-chat sequence numbers. - Delivery guarantees: client ACKs; resend on no ACK; idempotent message IDs. - Presence: heartbeats with TTL in Redis; avoid broadcasting presence to everyone (fan-out limits). - Group fan-out: small groups fan out on write; large channels fan out on read. - Sync across devices: per-device cursor. **Mistake to avoid:** polling for messages (ok for small scale; WebSocket for real-time).

Case 6: News Feed / Twitter Timeline

Requirements: post, follow, home timeline; timeline latency <200 ms; celebrities with millions of followers. **Scale:** 300M DAU, 2 posts/day/user for a small fraction; 10:1 read:write. **Core decision: fan-out on write vs fan-out on read.** - *Write (push):* on post, insert into each follower's precomputed timeline (Redis list). Fast reads, expensive for celebrities. - *Read (pull):* merge posts from followed users at read time. Cheap writes, slow reads. - *Hybrid:* push for normal users; pull for celebrities at read time and merge. **Components:** post service → Kafka → fan-out workers → timeline cache (Redis) → feed service. Media on object storage + CDN. Ranking service for ML-based ordering (optional). **Deep dives:** celebrity problem, hot tweets, cache rebuild after eviction, pagination with cursor (not offset).

Case 7: Typeahead / Autocomplete

Requirements: top 5 suggestions per prefix; <100 ms; updated daily or hourly; typos optional. **Scale:** 1B searches/day →

~12K/s average, ~40K/s peak. **Components:** - Data collection: query logs → Kafka → aggregation (Spark/Flink) → counts per phrase. - Build a trie or precomputed top-k per prefix; store in a KV or cached per prefix. - Serve from an in-memory trie / cache close to users. **Deep dives:** - Trie memory: compress (radix tree); top-k stored at each node. - Update strategy: periodic rebuild (blue/green) rather than live mutation. - Personalization and filtering of unsafe terms. - Client-side debounce to cut requests.

Case 8: Ride Sharing (Uber)

Requirements: rider requests ride; match with nearby driver; live location; ETA; pricing; trip history. **Scale:** 5M drivers sending location every 4 s → ~1.25M location updates/s. **Components:** - Location service: stores driver location in an in-memory geospatial index (geohash/H3 cells), sharded by cell. - Matching service: query nearby cells, rank drivers by ETA, send offers with timeout. - Trip service: state machine (requested → matched → picked up → completed); persist in DB. - Real-time updates to rider via WebSocket. **Deep dives:** - High write rate for location: don't store every update durably; keep latest position in memory; sample for history. - Matching race: a driver accepts two rides; use a lock or atomic "offer state" transition. - Surge pricing: compute per cell from demand/supply in a stream job. - Region failover and dispatch correctness.

Case 9: Payment System

Requirements: pay for an order; refunds; no double charge; PCI compliance; reconciliation with the bank. **Scale:** 1M transactions/day (~12/s average, peak ~100/s). **Components:** payment API → idempotency store → ledger service (double-entry, append-only) → payment processor adapter → webhook handler → reconciliation job. **Deep dives:** - **Idempotency:** client sends `Idempotency-Key`; store key + response; return the same result on retry. - **State machine:** `created → processing → succeeded / failed → refunded`. Transitions are atomic (DB constraints or versioning). - **Exactly-once effect:** at-least-once messages + idempotent consumers + ledger uniqueness. - **Money:** store amounts as integers in minor units; never floats. - **Reconciliation:** nightly job matches internal ledger with the processor's settlement file. - **Strong consistency:** choose a relational DB with transactions.

Case 10: Web Crawler

Requirements: crawl billions of pages; respect robots.txt; politeness per domain; avoid duplicates; refresh periodically. **Scale:** 1B pages/month → ~400 pages/s. **Components:** - URL frontier: priority queue + per-domain queues for politeness. - Fetcher workers (distributed); DNS cache. - Content parser → dedupe with content hash (SimHash/MinHash for near-duplicates). - URL seen-set: Bloom filter (memory efficient, false positives OK) backed by a store. - Storage: object store for raw HTML; index pipeline separate. **Deep dives:** politeness and robots.txt caching; traps (infinite URLs); prioritizing fresh pages; fault tolerance (checkpoint frontier).

Case 11: Distributed File Storage (Dropbox / Google Drive)

Requirements: upload/download files; sync across devices; versioning; sharing; large files. **Scale:** 100M users, 500 MB average storage each → 50 PB (say so; then scope). **Components:** - Metadata service (file tree, versions, permissions) on a relational DB, sharded by user. - Block storage: split files into chunks (4 MB), content-addressed by hash → dedupe. - Object store for chunks; CDN for downloads. - Sync service: notifications via long-poll/WebSocket; change log per user to compute deltas. **Deep dives:** - Chunking: only changed chunks upload (delta sync). - Conflict resolution on concurrent edits (last writer wins vs create conflict copy). - Upload resumability with chunk offsets. - Consistency between metadata and blobs: write blob first, then metadata; GC orphaned blobs.

Case 12: Metrics / Monitoring System (like Datadog or Prometheus)

Requirements: ingest metrics from 100K hosts; query dashboards; alerting; retention 1 year with downsampling. **Scale:** 100K hosts × 500 metrics × every 10 s → 5M points/s. **Components:** - Agents → ingestion gateway → Kafka (partition by metric name/host). - Time-series DB (Druid, Prometheus TSDB, InfluxDB, or Cassandra-style store) with rollups. - Query service with caching for dashboards. - Alert evaluator: streaming or periodic queries; notifications with dedupe and grouping. **Deep dives:** - Compression of time-series (delta-of-delta, Gorilla). - Cardinality explosion (too many label combinations) and limits. - Downsampling: 10 s raw for 7 days, 1 min for 90 days, 1 h for 1 year. - Late and out-of-order data: watermarks; windowing. **Link to your project:** Druid ingestion from Kafka, segment compaction, rollup, and query latency. Use this as a strong story.

Case 13: Video Streaming (YouTube / Netflix)

Requirements: upload; transcoding; streaming with adaptive bitrate; search; view counts. **Scale:** 500 hours uploaded/min;

1B views/day. **Components:** - Upload to blob storage (resumable) → event → transcoding workers (multiple resolutions, codecs) → segments (HLS/DASH) → object store. - CDN for segments; origin shield. - Metadata DB; search index; view-count pipeline (approximate counts with batching). **Deep dives:** - Adaptive bitrate: client picks quality based on bandwidth. - Transcoding is expensive: priority queues, parallel chunked encoding. - Hot videos: CDN caching; popularity-based replication. - View counts: write-behind aggregation, not one DB write per view.

Case 14: Distributed Job Scheduler / Workflow Engine

Requirements: schedule one-time and recurring jobs; retries; exactly-once-ish execution; at least 1M jobs/day; visibility. **Components:** - API → job store (DB with `next_run_at` index). - Scheduler polls due jobs (or uses time-wheel / delay queue); claims jobs with lease (`locked_until`). - Workers execute; heartbeat extends lease; on expiry, another worker retries. - Workflow orchestration for multi-step flows (Temporal-style durable execution). **Deep dives:** - Duplicate execution: leases + idempotent job handlers. - Clock skew and time-wheel precision. - Priority and fairness across tenants. - Partitioning the scheduler (shard by job ID hash). **Link to your project:** your Temporal workflows: retry policies, timeouts, task queues, and how you handled long-running ETL.

Case 15: Event Pipeline / Log Analytics

Requirements: collect events from apps; process in near real time; store for analytics; dashboards; replay. **Scale:** 1B events/day → ~12K/s average, peak 50K/s. **Components:** SDK → ingestion API (validate, enrich) → Kafka (topics by domain, partitioned by key) → stream processing (Flink/Spark Structured Streaming, or consumers) → OLAP store (Druid, ClickHouse) and data lake (Iceberg/Parquet on S3). **Deep dives:** - Partition key choice (even load, per-entity ordering). - Exactly-once in aggregations: idempotent writes using event IDs; dedupe window. - Late data: watermarks and allowed lateness. - Schema evolution: schema registry (Avro/Protobuf); backward-compatible changes. - Compaction of small files in the lake (Iceberg); partition pruning for queries. - Replay: keep Kafka retention or read from the lake for backfills. **Link to your project:** the Kafka → Druid and Kafka → Iceberg migration. Explain why you moved batch-heavy ingestion, what you gained (40% less supervisor load), and the trade-offs (freshness vs cost).

Part E: Common Mistakes (check yourself)

1. Jumping to a design before asking requirements.
2. Skipping estimates, then choosing a technology with no reason.
3. Drawing many boxes with no data flow.
4. Ignoring failure: “what if this node dies?”
5. Choosing Kafka or Redis as a buzzword without explaining the guarantee you need.
6. Going too deep on one component too early, then running out of time.
7. Forgetting monitoring, security, and cost.

Low-Level System Design (LLD) Guide

LLD = object-oriented design of a single system: classes, relationships, interfaces, patterns, and working code. Interviewers check whether you can model a problem cleanly and extend it without rewriting.

Part A: The LLD Interview Framework (45 min)

Step	Time	What to do
1. Clarify requirements	5 min	Ask: actors, core actions, constraints, scale of the single machine, concurrency
2. Identify entities	3 min	Nouns become classes; verbs become methods
3. Define relationships	3 min	Composition vs inheritance; ownership; cardinality
4. Class diagram + APIs	7 min	Public methods, key fields, enums
5. Core code	20 min	Write the main flow; keep it compiling in your head
6. Extensibility + edge cases	7 min	Show how a new rule plugs in; concurrency; invalid input

Clarifying questions to always ask: - What is in scope? (e.g., “just booking, or also payments?”) - Single-threaded or concurrent access? - What must be extensible? (new vehicle types, new pricing rules) - What should happen on invalid input? (exceptions vs return codes) - Does it need persistence, or in-memory only?

Say out loud: “I’ll start with the core entities, then the main flow, then handle extensibility and concurrency.”

Part B: Principles and Patterns

B1. SOLID (one line each, with a quick example)

- **S: Single Responsibility.** A class has one reason to change. Example: separate `Parker` (who parks) from `FeeCalculator` (how much to charge).
- **O: Open/Closed.** Open for extension, closed for modification. Example: add a new discount by adding a class that implements `DiscountStrategy`, not by editing an `if` chain.
- **L: Liskov Substitution.** Subtypes must work wherever the base type works. Example: a `Square` that extends `Rectangle` and breaks `setWidth` violates this.
- **I: Interface Segregation.** Small, focused interfaces. Example: `Printable` and `Scannable` instead of one fat `Machine` interface.
- **D: Dependency Inversion.** Depend on abstractions. Example: `OrderService` takes a `PaymentGateway` interface, not `StripeClient`.

B2. Composition over inheritance

Prefer “has-a” over “is-a” unless the is-a relationship is stable. Composition makes behavior swappable at runtime.

B3. Design patterns you must know (with the signal that tells you to use them)

Pattern	Use when	Example
Strategy	Multiple interchangeable algorithms	Pricing rules, rate-limit algorithms, parking fee
State	Object behavior changes with its state	Vending machine, elevator, order lifecycle
Observer	One event notifies many listeners	Notification service, stock price updates
Factory / Factory Method	Creation depends on type or config	Creating vehicles, notification channels
Builder	Many optional parameters	Building a complex request or a meal
Singleton	Exactly one instance (use sparingly)	Config, logger; prefer dependency injection
Decorator	Add behavior dynamically	Coffee with add-ons, logging/compression wrappers
Command	Encapsulate a request, enable undo	Text editor undo, job queue

Chain of Responsibility	Pass a request along handlers	Logger levels, middleware, approval chains
Template Method	Fixed algorithm with customizable steps	Report generation, game turn
Iterator	Traverse a collection without exposing its internals	Playlist, tree traversal
Facade	Simple interface over a complex subsystem	`OrderFacade` over inventory, payment, shipping

B4. Concurrency in LLD (a common follow-up)

- **Shared state:** protect with a lock (`threading.Lock` in Python, `synchronized` or `ReentrantLock` in Java).
- **Check-then-act race:** `if seat.is_free(): seat.book()` is a race. Make the check and the act atomic under one lock.
- **Lock granularity:** lock per resource (per seat, per account), not globally, unless simplicity matters more.
- **Deadlock:** acquire multiple locks in a consistent global order (e.g., by ID).
- **Thread-safe collections:** `ConcurrentHashMap` (Java), `queue.Queue` (Python).

B5. Common modeling mistakes

- God class that does everything.
- Using strings where enums belong (`status = "ACTVE"`).
- Public mutable fields that let anyone change state.
- Business logic in the controller or in the data class.
- `if type == ...` chains that should be polymorphism or Strategy.

Part C: LLD Problems

Each problem gives: requirements, entities, key patterns, core code, and extension points. Code is in Python for readability; translate to Java or Go if your interview requires it.

Problem 1: Parking Lot

Requirements: multi-floor parking lot; vehicle types (bike, car, truck) with matching spot sizes; issue ticket on entry; calculate fee on exit (hourly, different rates by type); multiple entry/exit gates; count of free spots by type.

Entities: `ParkingLot`, `Floor`, `ParkingSpot` (with type), `Vehicle` (with type), `Ticket`, `FeeStrategy`, `Gate`.

Patterns: Strategy (fee calculation), Factory (spot allocation by vehicle size).

```python
from dataclasses import dataclass, field
from datetime import datetime
from enum import Enum
from math import ceil
import threading

class VehicleType(Enum):
    BIKE = 1
    CAR = 2
    TRUCK = 3

class SpotType(Enum):
    SMALL = 1
    MEDIUM = 2
    LARGE = 3

# Vehicle type -> compatible spot type
COMPATIBLE = {
    VehicleType.BIKE: [SpotType.SMALL, SpotType.MEDIUM, SpotType.LARGE],
    VehicleType.CAR: [SpotType.MEDIUM, SpotType.LARGE],
    VehicleType.TRUCK: [SpotType.LARGE],
}

@dataclass
class Vehicle:
    plate: str
    vtype: VehicleType

@dataclass
class ParkingSpot:
    spot_id: str
    spot_type: SpotType
    vehicle: "Vehicle | None" = None

    @property
    def free(self) -> bool:
        return self.vehicle is None

@dataclass
class Ticket:
    ticket_id: str
    vehicle: Vehicle
    spot: ParkingSpot
    entry_time: datetime

class FeeStrategy:
    RATE_PER_HOUR = {VehicleType.BIKE: 10, VehicleType.CAR: 20, VehicleType.TRUCK: 40}
    def fee(self, ticket: Ticket, exit_time: datetime) -> int:
        hours = max(1, ceil((exit_time - ticket.entry_time).total_seconds() / 3600))
        return hours * self.RATE_PER_HOUR[ticket.vehicle.vtype]

class ParkingLot:
    def __init__(self, spots: list[ParkingSpot], fee_strategy: FeeStrategy):
        self._spots = spots
        self._active: dict[str, Ticket] = {}   # ticket_id -> Ticket
        self._fee = fee_strategy
        self._lock = threading.Lock()
        self._counter = 0

    def park(self, vehicle: Vehicle, now: datetime) -> Ticket | None:
        with self._lock:                       # check-and-allocate must be atomic
            for spot in self._spots:
                if spot.free and spot.spot_type in COMPATIBLE[vehicle.vtype]:
                    spot.vehicle = vehicle
                    self._counter += 1
                    t = Ticket(f"T{self._counter}", vehicle, spot, now)
                    self._active[t.ticket_id] = t
                    return t
            return None                        # lot full for this type

    def unpark(self, ticket_id: str, now: datetime) -> int:
        with self._lock:
            t = self._active.pop(ticket_id)    # raises KeyError on invalid ticket
            t.spot.vehicle = None
            return self._fee.fee(t, now)

    def free_count(self, spot_type: SpotType) -> int:
        return sum(1 for s in self._spots if s.free and s.spot_type == spot_type)
```

Extension points: - New pricing (weekend, EV charging): add a `FeeStrategy` subclass. - Spot allocation policy (nearest to entrance): replace the loop with a `SpotAllocator` strategy. - Multiple floors: group spots by floor, keep a per-type free-list for O(1) allocation.

Problem 2: LRU Cache

Requirements: `get(key)` and `put(key, value)` in O(1); evict least recently used on capacity overflow.

Key idea: hash map for O(1) lookup + doubly linked list for O(1) reordering.

```
        class _Node:
            __slots__ = ("key", "val", "prev", "next")
            def __init__(self, key=None, val=None):
                self.key, self.val = key, val
                self.prev = self.next = None

        class LRUCache:
            def __init__(self, capacity: int):
                self.cap = capacity
                self.map: dict[int, _Node] = {}
                # sentinels: head.next is most recent, tail.prev is least recent
                self.head, self.tail = _Node(), _Node()
                self.head.next, self.tail.prev = self.tail, self.head

            def _remove(self, node: _Node) -> None:
                node.prev.next, node.next.prev = node.next, node.prev

            def _add_front(self, node: _Node) -> None:
                node.next, node.prev = self.head.next, self.head
                self.head.next.prev = node
                self.head.next = node

            def get(self, key: int) -> int:
                if key not in self.map:
                    return -1
                node = self.map[key]
                self._remove(node)
                self._add_front(node)
                return node.val

            def put(self, key: int, value: int) -> None:
                if key in self.map:
                    node = self.map[key]
                    node.val = value
                    self._remove(node)
                    self._add_front(node)
                    return
                if len(self.map) == self.cap:
                    lru = self.tail.prev              # evict least recent
                    self._remove(lru)
                    del self.map[lru.key]
                node = _Node(key, value)
                self.map[key] = node
                self._add_front(node)
```

Follow-ups: thread safety (wrap with lock), TTL (store expiry time in node), generic types, LFU variant (frequency buckets).

Problem 3: Vending Machine (State Pattern)

Requirements: select product; insert coins; dispense; return change; cancel. Behavior depends on state (no coin, has coin, dispensing, out of stock).

Pattern: State. Each state class handles the same actions differently. No big `if state == ...` chain.

```python
from abc import ABC, abstractmethod

class VendingMachine:
    def __init__(self, inventory: dict[str, tuple[int, int]]):  # code -> (price, qty)
        self.inventory = inventory
        self.balance = 0
        self.selected: str | None = None
        self.state: "State" = IdleState()

    def set_state(self, s: "State") -> None:
        self.state = s

    def insert_coin(self, amount: int): self.state.insert_coin(self, amount)
    def select(self, code: str): self.state.select(self, code)
    def dispense(self): self.state.dispense(self)
    def cancel(self): self.state.cancel(self)

class State(ABC):
    @abstractmethod
    def insert_coin(self, m, amount): ...
    @abstractmethod
    def select(self, m, code): ...
    @abstractmethod
    def dispense(self, m): ...
    @abstractmethod
    def cancel(self, m): ...

class IdleState(State):
    def insert_coin(self, m, amount):
        m.balance += amount
        m.set_state(HasMoneyState())
    def select(self, m, code): print("Insert money first")
    def dispense(self, m): print("Select a product first")
    def cancel(self, m): print("Nothing to cancel")

class HasMoneyState(State):
    def insert_coin(self, m, amount): m.balance += amount
    def select(self, m, code):
        price, qty = m.inventory.get(code, (None, 0))
        if price is None or qty == 0:
            print("Unavailable"); return
        if m.balance < price:
            print("Insufficient balance"); return
        m.selected = code
        m.set_state(DispenseState())
    def dispense(self, m): print("Select a product first")
    def cancel(self, m):
        print(f"Refund {m.balance}"); m.balance = 0; m.set_state(IdleState())

class DispenseState(State):
    def insert_coin(self, m, amount): print("Please wait")
    def select(self, m, code): print("Already dispensing")
    def dispense(self, m):
        price, qty = m.inventory[m.selected]
        m.inventory[m.selected] = (price, qty - 1)
        m.balance -= price
        print(f"Dispensed {m.selected}, change {m.balance}")
        m.balance, m.selected = 0, None
        m.set_state(IdleState())
    def cancel(self, m): print("Cannot cancel while dispensing")
```

Follow-ups: out-of-stock state, coin validation, persistence of state, admin refill flow.

Problem 4: Rate Limiter (LLD version with Strategy)

Requirements: pluggable algorithm per API; per-key limits; thread-safe.

```
import time, threading
from abc import ABC, abstractmethod

class RateLimitStrategy(ABC):
    @abstractmethod
    def allow(self, key: str) -> bool: ...

class TokenBucket(RateLimitStrategy):
    def __init__(self, capacity: int, refill_per_sec: float):
        self.cap, self.rate = capacity, refill_per_sec
        self.state: dict[str, tuple[float, float]] = {}   # key -> (tokens, last_ts)
        self.lock = threading.Lock()

    def allow(self, key: str) -> bool:
        now = time.monotonic()
        with self.lock:
            tokens, last = self.state.get(key, (self.cap, now))
            tokens = min(self.cap, tokens + (now - last) * self.rate)
            if tokens >= 1:
                self.state[key] = (tokens - 1, now)
                return True
            self.state[key] = (tokens, now)
            return False

class FixedWindow(RateLimitStrategy):
    def __init__(self, limit: int, window_sec: int):
        self.limit, self.window = limit, window_sec
        self.counts: dict[tuple[str, int], int] = {}
        self.lock = threading.Lock()

    def allow(self, key: str) -> bool:
        bucket = (key, int(time.time() // self.window))
        with self.lock:
            self.counts[bucket] = self.counts.get(bucket, 0) + 1
            return self.counts[bucket] <= self.limit
```

Follow-ups: cleanup of stale keys (LRU or TTL), distributed version with Redis Lua, per-tier limits via a config map.

Problem 5: Elevator System

Requirements: multiple elevators; floor requests from inside and outside; move toward requested floors efficiently; door open/close.

Entities: `Elevator` (direction, current floor, stop set), `Controller` (assigns requests), `Request`. **Patterns:** State (IDLE, MOVING_UP, MOVING_DOWN, DOORS_OPEN), Strategy (scheduling: nearest elevator, SCAN/LOOK algorithm).

Core ideas: - Each elevator keeps two sorted sets of stops: `up_stops` and `down_stops`. - While moving up, serve stops above the current floor; when none remain, reverse (LOOK algorithm). - Controller assigns an external request to the elevator with the lowest cost: distance plus direction mismatch penalty. - Use a lock per elevator; the controller holds a lock only while assigning.

Skeleton:

```
class Elevator:
    def __init__(self, eid: int):
        self.id, self.floor, self.direction = eid, 0, 0    # 0 idle, 1 up, -1 down
        self.stops: set[int] = set()

    def add_stop(self, floor: int) -> None:
        self.stops.add(floor)

    def cost(self, floor: int, direction: int) -> int:
        """Lower is better: distance plus penalty if moving the wrong way."""
        dist = abs(self.floor - floor)
        if self.direction == 0:
            return dist                       # idle: just distance
        ahead = (self.direction == 1 and floor >= self.floor) or \
                (self.direction == -1 and floor <= self.floor)
        if ahead and direction == self.direction:
            return dist                       # already heading that way
        return dist + 10                      # wrong way: penalty (tunable)

    def step(self) -> None:
        """Advance one floor toward the next stop (called each tick)."""
        if not self.stops:
            self.direction = 0
            return
        if self.direction == 0:
            target = min(self.stops, key=lambda f: abs(f - self.floor))
            self.direction = 1 if target > self.floor else -1 if target < self.floor else 0
        if self.direction == 1 and any(f > self.floor for f in self.stops):
            self.floor += 1
        elif self.direction == -1 and any(f < self.floor for f in self.stops):
            self.floor -= 1
        else:
            self.direction = -self.direction  # reverse when nothing ahead (LOOK)
        self.stops.discard(self.floor)        # open doors, serve stop
```

Follow-ups: weight/capacity limits, emergency mode, priority requests, what happens if a door is blocked.

Problem 6: Logger (Chain of Responsibility)

Requirements: log levels (DEBUG, INFO, WARN, ERROR); multiple sinks (console, file, remote); each sink handles levels at or above its threshold; easy to add sinks.

```
from abc import ABC, abstractmethod
from enum import IntEnum

class Level(IntEnum):
    DEBUG = 1
    INFO = 2
    WARN = 3
    ERROR = 4

class Sink(ABC):
    def __init__(self, min_level: Level, next_sink: "Sink | None" = None):
        self.min_level, self.next = min_level, next_sink

    def log(self, level: Level, msg: str) -> None:
        if level >= self.min_level:
            self.write(msg)
        if self.next:
            self.next.log(level, msg)

    @abstractmethod
    def write(self, msg: str) -> None: ...

class ConsoleSink(Sink):
    def write(self, msg: str) -> None: print("[console]", msg)

class FileSink(Sink):
    def write(self, msg: str) -> None:
        with open("app.log", "a") as f: f.write(msg + "\n")
```

Note: this is Chain of Responsibility where every handler may act and the chain continues. A “first handler wins” version is also common; say which you chose and why.

Follow-ups: async logging with a queue and background thread, formatters (Decorator), log rotation.

Problem 7: Splitwise (Expense Sharing)

Requirements: add users and group expenses; split equally, by exact amount, or by percentage; show balances; simplify debts.

Entities: `User`, `Expense` (payer, amount, splits), `SplitStrategy` (Strategy), `BalanceSheet` (net balances per user pair).

Key algorithm (debt simplification): 1. Compute net balance per user: `paid - owed`. 2. Split into creditors (positive) and debtors (negative). 3. Greedily match largest debtor with largest creditor; transfer `min(|debt|, credit)`. 4. Repeat until all zero. This minimizes the number of transactions in practice (exact minimum is NP-hard in general).

```
        import heapq
        from collections import defaultdict

        def simplify(balances: dict[str, int]) -> list[tuple[str, str, int]]:
            """balances: user -> net (positive = is owed money). Amounts in paise (ints)."""
            creditors = [(-v, u) for u, v in balances.items() if v > 0]   # max-heap via negation
            debtors = [(v, u) for u, v in balances.items() if v < 0]      # v is negative
            heapq.heapify(creditors); heapq.heapify(debtors)
            txns = []
            while creditors and debtors:
                c_neg, c = heapq.heappop(creditors)
                d_val, d = heapq.heappop(debtors)
                c_amt, d_amt = -c_neg, -d_val
                pay = min(c_amt, d_amt)
                txns.append((d, c, pay))
                if c_amt > pay: heapq.heappush(creditors, (-(c_amt - pay), c))
                if d_amt > pay: heapq.heappush(debtors, (-(d_amt - pay), d))
            return txns
```

Follow-ups: use integers for money (avoid float rounding), rounding remainders for equal splits, validation (splits sum to total).

Problem 8: Movie Ticket Booking (Concurrency Focus)

Requirements: shows, seats; user holds seats for a limited time; confirm or release; no double booking under concurrent requests.

Key ideas: - Seat states: `AVAILABLE → HELD (with expiry) → BOOKED`. A background job releases expired holds. - Atomic transition: `hold(seat, user)` must check and set state under a lock or with a DB conditional update (`UPDATE seats SET status='HELD' WHERE id=? AND status='AVAILABLE'`). - Lock per show, or per seat for finer granularity. Acquire multiple seat locks in sorted order to avoid deadlock.

```python
import threading, time

class Seat:
    def __init__(self, sid: str):
        self.sid, self.status, self.holder, self.expires = sid, "AVAILABLE", None, 0.0

class Show:
    def __init__(self, seats: list[str], hold_sec: int = 300):
        self.seats = {s: Seat(s) for s in seats}
        self.lock = threading.Lock()
        self.hold_sec = hold_sec

    def _expire(self, now: float) -> None:
        for s in self.seats.values():
            if s.status == "HELD" and s.expires < now:
                s.status, s.holder = "AVAILABLE", None

    def hold(self, user: str, seat_ids: list[str]) -> bool:
        now = time.monotonic()
        with self.lock:                          # all-or-nothing for the group
            self._expire(now)
            if any(self.seats[s].status != "AVAILABLE" for s in seat_ids):
                return False
            for s in seat_ids:
                self.seats[s].status, self.seats[s].holder = "HELD", user
                self.seats[s].expires = now + self.hold_sec
            return True

    def confirm(self, user: str, seat_ids: list[str]) -> bool:
        now = time.monotonic()
        with self.lock:
            ok = all(self.seats[s].status == "HELD" and
                     self.seats[s].holder == user and
                     self.seats[s].expires >= now for s in seat_ids)
            if ok:
                for s in seat_ids:
                    self.seats[s].status = "BOOKED"
            return ok
```

Follow-ups: payment timeout, distributed version with Redis locks (with TTL and fencing tokens), DB row locking vs optimistic versioning.

Problem 9: Observer-Based Notification Service

Requirements: publishers emit events (order shipped, price drop); subscribers choose channels (email, SMS, push); add channels without changing publishers.

```python
from abc import ABC, abstractmethod

class Subscriber(ABC):
    @abstractmethod
    def notify(self, event: dict) -> None: ...

class EventBus:
    def __init__(self):
        self._subs: dict[str, list[Subscriber]] = {}

    def subscribe(self, topic: str, sub: Subscriber) -> None:
        self._subs.setdefault(topic, []).append(sub)

    def publish(self, topic: str, event: dict) -> None:
        for sub in self._subs.get(topic, []):
            try:
                sub.notify(event)
            except Exception as e:      # one bad subscriber must not block the others
                print(f"subscriber failed: {e}")

class EmailSender(Subscriber):
    def notify(self, event: dict) -> None: print("email:", event)

class SmsSender(Subscriber):
    def notify(self, event: dict) -> None: print("sms:", event)

bus = EventBus()
bus.subscribe("order.shipped", EmailSender())
bus.subscribe("order.shipped", SmsSender())
bus.publish("order.shipped", {"order_id": 42})
```

Follow-ups: async delivery with a queue per subscriber, retries, unsubscribe, ordering guarantees.

Problem 10: Tic-Tac-Toe (Extensible Board Game)

Requirements: NxN board; two players; win detection; draw; easy to add players or AI.

Design: `Board` (grid + win check), `Player` (symbol + strategy), `Game` (turn loop). AI is a `MoveStrategy` (Strategy pattern).

```
        class Board:
            def __init__(self, n: int = 3):
                self.n = n
                self.grid = [[None] * n for _ in range(n)]
                self.moves = 0

            def place(self, r: int, c: int, sym: str) -> bool:
                if not (0 <= r < self.n and 0 <= c < self.n) or self.grid[r][c] is not None:
                    return False
                self.grid[r][c] = sym
                self.moves += 1
                return True

            def winner_at(self, r: int, c: int) -> bool:
                """Only check lines through the last move: O(n), not O(n^2)."""
                s = self.grid[r][c]
                n = self.n
                row = all(self.grid[r][j] == s for j in range(n))
                col = all(self.grid[i][c] == s for i in range(n))
                diag = r == c and all(self.grid[i][i] == s for i in range(n))
                anti = r + c == n - 1 and all(self.grid[i][n - 1 - i] == s for i in range(n))
                return row or col or diag or anti

            def is_full(self) -> bool:
                return self.moves == self.n * self.n
```

Follow-ups: undo (Command pattern), AI with minimax, saving game state, a Connect Four variant with a different win check.

Part D: Quick Templates

Class diagram shorthand you can write on a whiteboard:

```
ParkingLot 1 ── * ParkingSpot
ParkingSpot  ── 0..1 Vehicle
ParkingLot ──> FeeStrategy (interface)
  ├─ HourlyFee
  └─ WeekendFee
```

Final checklist before you say “done”: - [ ] Each class has one responsibility. - [ ] No `if type ==` chains for behavior that could be a strategy. - [ ] Shared state is locked; check-and-act is atomic. - [ ] Invalid inputs are handled (raise or return an error, and say which). - [ ] You showed where a new feature plugs in without editing existing code.

Part E: Practice Order

1. Parking lot (warm-up for entities and Strategy).
2. LRU cache (data structures + coding).
3. Vending machine (State).
4. Rate limiter (Strategy + thread safety).
5. Logger (Chain of Responsibility).
6. Notification (Observer).
7. Splitwise (algorithms in design).
8. Elevator (State + scheduling).
9. Movie booking (concurrency).
10. Tic-tac-toe (clean extensible code, under time pressure).

Redo the code from scratch after 3 days, without looking. That’s where the learning sticks.

Q&A Bank, Behavioral Prep, and Your Resume Stories

Part A: Rapid-Fire Questions (answer aloud in 60–90 seconds)

Fundamentals 1. What is the difference between horizontal and vertical scaling? When would you pick each? 2. Explain CAP theorem with a real example. 3. What is PACELC and how is it different from CAP? 4. SQL vs NoSQL: how do you choose? 5. What is a B-tree and why do databases use it? 6. What is an LSM tree? Where is it used? 7. Explain database indexes and the trade-off they introduce. 8. What is replication lag and how do you handle read-your-own-writes? 9. Explain sharding. What makes a good shard key? 10. What is consistent hashing and why use virtual nodes?

Caching and messaging 11. Cache-aside vs write-through vs write-back: when do you use each? 12. What is a cache stampede and how do you prevent it? 13. How do you handle hot keys? 14. Kafka: what guarantees does it give for ordering? Within what scope? 15. At-least-once vs exactly-once: what do you actually implement in practice? 16. How do you make a consumer idempotent? 17. What is consumer lag and what do you do when it grows? 18. What is a dead letter queue and when do messages go there?

Reliability and distributed systems 19. What is idempotency and where do you need it? 20. Explain the transactional outbox pattern. 21. Saga vs two-phase commit: what are the trade-offs? 22. How do you design retries? (backoff, jitter, retry budgets) 23. What is a circuit breaker and when does it open or close? 24. What is quorum? Explain R + W > N. 25. How do you detect and handle a failed node? 26. What is a leader election, and what tools provide it? (ZooKeeper, etcd, Raft) 27. Explain eventual consistency with an example users would notice.

APIs, security, and ops 28. REST vs gRPC vs GraphQL: when do you use each? 29. Pagination: offset vs cursor. Which and why? 30. How do you version an API? 31. JWT vs session tokens: pros and cons. 32. OAuth2 vs OIDC. 33. How do you prevent a double-charge on a retry? 34. What metrics would you monitor for this service? (RED, USE, SLOs) 35. Canary deployment vs blue-green deployment.

Estimation and numbers 36. How many seconds are in a day? (~86,400, use 10^5) 37. Estimate QPS for 10M DAU with 10 actions each. (~1,157 avg; peak ~3K) 38. Roughly how long does a cross-continent round trip take? (~150 ms) 39. How much storage for 1B records of 1 KB each? (1 TB) 40. When would you add a CDN? (static content, geographically spread users)

Keep answers short. If you can’t answer in 90 seconds, you probably need one more pass on that topic.

Part B: Common Behavioral Prompts (use the STAR format)

STAR = **Situation, Task, Action, Result**. Keep each part to 2–3 sentences. End with a number.

1. Tell me about a system you built that had to scale. What was the hardest part?
2. Describe a production incident you handled. What did you learn?
3. Tell me about a time you disagreed with a technical decision.
4. Describe a design you would change if you did it again.
5. How do you decide between build and buy?
6. Tell me about a time you had to learn something quickly.
7. How do you prioritize when you have too many tasks?
8. Why are you leaving your current role? (keep it positive and forward-looking)

Prepare two stories from your work: - A **scaling story** (Kafka/Druid/Iceberg). - An **incident or failure story** (for example: a retry storm, a bad migration, a Temporal workflow stuck in a retry loop, a data mismatch found by reconciliation). Use a real one.

Part C: Turning Your Resume into System Design Stories

Your resume has strong material. Interviewers often ask “tell me more about this project” and then go into system design. Prepare each project as a 5-minute story with a diagram.

Story 1: Telecom NPM Platform (Whiteklay Technology)

Pitch (30 seconds): “We ingested telecom performance data from five network vendors. I built the parsing and orchestration for 1M+ XML files per day, and Kafka streams that bring 1M+ events/day into Druid events per day into Apache Druid for near-real-time KPI dashboards.”

Architecture to draw:

```
Vendor files (XML) → Ingestion (MinIO/S3)
    → Temporal workflows (retries, timeouts, task queues) → ETL parsers (config-driven, normalize counters)
    → Kafka (partitioned) → Druid (real-time ingestion, segments, rollup)
    → Query API (GraphQL, Node.js) → Dashboards
Batch path: Kafka → Iceberg (Parquet on S3) → compaction/dedup jobs → Druid for direct partition queries
```

Talking points by topic:

Topic	What you did	What to say about it
Orchestration	Temporal workflows with retry policies, timeouts, dedicated task queues	Durable execution: workflow state survives crashes; activities are retried; idempotent activities are essential
Kafka at 1M+/day	Streaming ingestion into Druid	Partition key choice; consumer scaling; lag monitoring; at-least-once with idempotent writes/dedupe
Kafka → Iceberg migration	Moved batch-heavy ingestion, cut Druid supervisor load by 40%	Trade-off: Druid is great for fresh, low-latency queries; Iceberg is cheaper for large batches and supports partition pruning and time travel
SQL query-generation service	Generates Druid SQL for analytics; cut report generation time by 95%; standardized queries across teams	Compiling inputs into a safe query; validation; avoiding injection; caching; consistency across teams
Compaction and rollup	Python jobs for compaction, dedup, rollup; up to 10x faster queries	Small-file problem; rollup trades granularity for speed; compaction schedule vs freshness
GraphQL API	Apollo Server, Prisma, PostgreSQL, Redis, Keycloak RBAC	N+1 problem and DataLoader; caching at the resolver; field-level authorization

Likely follow-ups (prepare answers): - “What happens if Kafka consumers fall behind?” → lag alerts, scale consumers up to partition count, check for hot partitions, backpressure. - “How do you avoid duplicate records in Druid?” → idempotent event IDs, dedup job on the lake, and the reconciliation check you built. - “How do you handle a vendor sending a malformed file?” → validation, DLQ or quarantine, alerting, and replay after a fix. - “Why Temporal and not plain cron plus a queue?” → retries with state, long-running workflows, visibility, and cancellation.

Verify before the interview: the figures on your resume (1M+/day, 1M+ files/day, 40%, 95%, 10x, 70–80%). You must be able to explain how each was measured. If a number is an estimate, say so (“roughly”, “peak”) and give the calculation.

Story 2: Distributed Cache (Your Project)

Pitch: “I built a sharded cache cluster in Node.js with consistent hashing using 200 virtual nodes per server, and master–slave replication with a load balancer. Adding or removing nodes causes minimal key movement.”

Draw it:

```
Client → Load balancer/router (consistent hash ring, 200 vnodes per node)
    → Shard A (master) ↔ replica
    → Shard B (master) ↔ replica
    → Shard C (master) ↔ replica
```

Be ready to answer: - Why 200 virtual nodes? → Smooths distribution; trade-off is ring memory and lookup cost (log n). - What happens when a node is removed? → Only keys on its arc move (about 1/N). Explain how you avoided “zero invalidation” in your project (did keys move? were they re-fetched from the source?). **Be precise about what your implementation really does.** - Replication: async or sync? → What did you choose and why? What is the failure window? - What happens on master failure? → Promotion, client redirect, and data loss window. - How is it different from Redis Cluster? → Hash slots (16384) vs a ring; gossip-based membership; built-in failover. - How would you make it production-ready? → Persistence, monitoring, connection pooling, eviction policy, security, tests under failure.

Story 3: CamelThar Travel Platform (Personal Project)

Pitch: “I built and launched a Next.js travel platform on Vercel with 13+ REST endpoints, JWT auth, Redis rate limiting, and a Redis-first real-time layer for chat delivery and presence without WebSockets.”

Good discussion points: - **Serverless constraints:** no long-lived WebSocket connections on Vercel functions. Explain why you used Redis and polling-style or SSE-style patterns, and what the trade-off is (latency vs cost and complexity). - **Presence with sorted sets:** store user ID with a timestamp score; expire stale entries; count online users with a range query. - **Typing indicators with TTL:** keys expire automatically, so no cleanup job is needed. - **Atomic booking:** how you prevented race conditions (a Redis lock, or a conditional DB update). Explain the difference. - **Rate limiting:** which

algorithm, where it runs, and what happens if Redis is down (fail open or closed?).

Interview tip: this project shows that you can design end to end as a single engineer. Use it when they ask about a full product or a smaller-scale system.

Part D: Turning Projects into a System Design Answer (the template)

Use this when asked “design X” and it matches one of your projects: 1. “I’ve worked on something similar: [one-line description with a number].” 2. Requirements: “Let me confirm…” (use your project’s real constraints). 3. Estimation: use the real scale, then extend it (“at 10× we would…”). 4. Design: draw what you built. 5. Deep dives: pick the parts where you learned the most, including what you would change. 6. Reflection: “If I rebuilt this today, I’d change X because Y.”

Interviewers love the reflection step. It shows judgment, not just memory.

Part E: Questions to Ask the Interviewer

- What does the team’s current architecture look like at a high level?
- What’s the biggest scaling challenge the team faces this year?
- How do you handle on-call and incident reviews?
- What does success look like for this role in the first 6 months?
- How do engineers here decide between building and buying a system?

Avoid asking about salary or benefits in the technical round.

Part F: Final Week Checklist

- [ ] Rapid-fire questions: answered out loud, all 40, within 90 seconds each.
- [ ] Two behavioral stories (scaling and failure) written as STAR, under 2 minutes each.
- [ ] Three project stories rehearsed with a diagram each (Part C).
- [ ] Every number on your resume: you can explain how it was measured.
- [ ] Four full mock interviews done (two HLD, two LLD), each self-graded with the rubric in `00_Study_Plan.md`.
- [ ] A one-page cheat sheet made from the framework, estimation numbers, and your three stories.

04. Deep Foundations: Networking, Time, Storage Engines, Indexes, Transactions

Self-contained. Each section: what it is, how it works inside, trade-offs, failure modes, and what to say in an interview.

4.1 Latency and Throughput Numbers (memorize the orders of magnitude)

Operation	Approx. time
L1 cache reference	1 ns
Branch mispredict	3 ns
L2 cache	4 ns
Mutex lock/unlock	17 ns
Main memory (RAM) reference	100 ns
Compress 1 KB (Snappy/Zippy)	2 µs
Send 1 KB over 1 Gbps	10 µs
SSD random 4 KB read	16–100 µs
Sequential read 1 MB from RAM	250 µs
Datacenter round trip	0.5 ms
Sequential read 1 MB from SSD	1 ms
HDD seek	10 ms
Sequential read 1 MB from HDD	20 ms
Packet CA → Netherlands → CA	150 ms

Lessons: memory is ~100x faster than SSD random reads; disk seeks dominate; network round trips across regions are the floor for any synchronous cross-region call. Never make a cross-region synchronous call on a hot path.

Throughput math: a single Redis instance does on the order of 100K+ simple ops/s; a single Postgres node commonly handles low thousands to tens of thousands of simple writes/s depending on hardware and durability settings; Kafka partitions can each do tens of MB/s. Always state these as rough, hardware-dependent figures.

4.2 Networking

4.2.1 The request path

```
Client → DNS → (CDN) → L4/L7 load balancer → API gateway → service → cache → DB
```

4.2.2 DNS

- Hierarchical resolution: recursive resolver → root → TLD → authoritative.
- Records: A/AAAA (IP), CNAME (alias), MX (mail), TXT, SRV, NS.
- **TTL** controls caching. Low TTL = fast failover, more DNS load. Common in failover designs.
- **GeoDNS / latency-based routing:** return different IPs by client location.
- Limitation: clients and resolvers cache beyond TTL sometimes; DNS failover is not instant.

4.2.3 TCP vs UDP

- **TCP:** reliable, ordered, congestion control, 3-way handshake (1 RTT) + TLS handshake. Head-of-line blocking on packet loss.
- **UDP:** unreliable, unordered, no handshake. Used for DNS, video, gaming, QUIC.
- **Connection costs:** pooling and keep-alive matter because handshakes add RTTs.

4.2.4 HTTP versions

- **HTTP/1.1:** one request per connection at a time (pipelining rarely used). Browsers open ~6 connections per host.
- **HTTP/2:** multiplexed streams over one TCP connection, binary framing, header compression (HPACK). Still suffers TCP head-of-line blocking.

- **HTTP/3:** runs over **QUIC** (UDP). Stream-level independence, 0-RTT or 1-RTT setup, connection migration across networks.

4.2.5 TLS

- TLS 1.3: 1-RTT handshake (0-RTT with resumption, with replay risk for non-idempotent requests).
- Terminate TLS at the load balancer or at the edge; use **mTLS** service-to-service inside a mesh.

4.2.6 Real-time communication options (know all four)

Mechanism	Direction	How	Use when
Short polling	Client pulls	Repeated HTTP requests	Simple, low frequency
Long polling	Client pulls	Server holds request until data or timeout	Notifications, moderate scale
Server-Sent Events (SSE)	Server → client	One long HTTP response streaming events	One-way feeds, dashboards, LLM token streams
WebSocket	Bidirectional	Upgraded TCP connection	Chat, multiplayer, collaboration

Stateful gateways: WebSocket servers hold connections; you need a registry (user → gateway node) and a way to route messages to the right node (pub/sub). Scaling connections means memory and file-descriptor limits (~tens of thousands to hundreds of thousands per box with tuning).

4.2.7 Load balancers in depth

- **L4 (transport):** routes by IP/port; very fast; no content awareness. Example: AWS NLB.
- **L7 (application):** routes by path, host, header, cookie; can terminate TLS, rewrite, retry. Example: AWS ALB, Envoy, NGINX.
- **Algorithms:** round robin; least connections; weighted; **power of two choices** (sample 2 random backends, pick the less loaded: near-optimal, cheap); consistent hashing (affinity/cache locality).
- **Health checks:** active (probe endpoint) and passive (observed errors). Use separate liveness vs readiness.
- **Connection draining:** on deploy, stop new requests and let in-flight ones finish.
- **Global LB:** Anycast IP or GeoDNS to pick a region; regional LB inside.

4.2.8 CDN

- Edge POPs cache static content and cacheable API responses.
- **Cache keys**, `Cache-Control` (max-age, s-maxage, stale-while-revalidate), `ETag`/`If-None-Match` for conditional requests.
- **Origin shielding:** a mid-tier cache so many edges don't hit origin on a miss.
- **Purge/invalidation:** slow and costly; prefer versioned URLs (`app.v3.js`).

4.3 Time, Clocks, and Ordering

- **Wall clocks** drift; NTP keeps them within ms on good networks, worse otherwise. Never assume two machines' clocks agree.
- **Monotonic clocks** measure elapsed time locally; use them for timeouts.
- **Lamport clock:** a counter; on send increment, on receive set to `max(local, received) + 1`. Gives a happens-before-consistent order, not real-time order.
- **Vector clock:** a vector of counters, one per node. Detects concurrent (conflicting) updates. Used by Dynamo-style systems for conflict detection. Size grows with nodes (pruning needed).
- **Hybrid Logical Clock (HLC):** combines wall time with a logical counter. Used by CockroachDB-style systems. Close to real time and still causal.
- **TrueTime (Spanner):** GPS and atomic clocks give an uncertainty interval `[earliest, latest]`; commit waits out the uncertainty to get external consistency.
- **Interview line:** "I'd use sequence numbers or a DB-assigned version for ordering, not wall-clock timestamps, because clocks skew."

4.4 Storage Engines

4.4.1 Write-ahead log (WAL)

- Every change is appended to a log before the data page is updated. On crash, replay the log.
- Basis of durability and replication in almost every database (Postgres WAL, MySQL binlog/redo log, Kafka log).

4.4.2 B-tree (PostgreSQL, MySQL InnoDB, most RDBMS indexes)

- Balanced tree of pages (typically 4–16 KB). Lookup O(log n) page reads, shallow (3–4 levels for billions of rows).
- Updates in place; page splits on overflow.
- **Read-optimized, good range scans.** Write amplification from page updates and WAL.
- **Fragmentation** and **page splits** under random inserts (e.g., random UUIDs as PKs).

4.4.3 LSM tree (Cassandra, RocksDB, LevelDB, HBase, ScyllaDB)

- Writes go to an in-memory **memtable** (plus WAL), flushed as immutable sorted files (**SSTables**).
- Background **compaction** merges SSTables (size-tiered or leveled).
- **Reads** check memtable, then SSTables newest first. Speed-ups: **Bloom filters** (per SSTable, no false negatives, tunable false positives), block cache, index blocks.
- **Trade-off:** excellent write throughput (sequential IO), read and space amplification, compaction CPU/IO spikes, tombstones for deletes (can slow reads until compacted).

4.4.4 Bloom filter

- Bit array + k hash functions. “Definitely not in set” or “possibly in set.” No false negatives.
- Use cases: avoid disk reads for missing keys (LSM), duplicate checks in crawlers, “has this user seen this item” at scale.
- Size: about 10 bits per element gives ~1% false positive rate.

4.4.5 Columnar storage (OLAP)

- Store each column together. Good compression (similar values), reading only needed columns, vectorized execution.
- Formats: **Parquet**, **ORC**. Engines: ClickHouse, Druid (segments), BigQuery, Redshift, Snowflake.
- Row store (OLTP) vs column store (OLAP): transactions and point lookups vs scans and aggregations.

4.4.6 Indexes

- **Primary (clustered) index:** table stored in PK order (InnoDB). **Secondary index** points to PK.
- **Composite index:** column order matters. `(a, b, c)` serves `a`, `a,b`, `a,b,c` but not `b` alone.
- **Covering index:** includes all selected columns; no table lookup.
- **Partial index:** indexes a subset (e.g., `WHERE status='active'`).
- **Hash index:** O(1) equality, no range scans.
- **Inverted index:** term → postings list (search engines, GIN in Postgres).
- **Geospatial:** R-tree, geohash, quadtree (see HLD guide).
- **Write cost:** each index adds write cost and storage. Index what the queries need.

4.4.7 Query planning

- Cost-based optimizer uses statistics (row counts, histograms). Stale statistics produce bad plans: run `ANALYZE`.
- `EXPLAIN` / `EXPLAIN ANALYZE` to read plans. Look for sequential scans on large tables, sort spills, nested-loop joins over large inputs.
- **N+1 queries:** an app pattern (1 query for a list, then N for each item). Fix with joins, batching (DataLoader), or prefetch.

4.5 Transactions and Isolation

4.5.1 ACID

- **Atomicity:** all or nothing (WAL + rollback).
- **Consistency:** invariants hold (application + DB constraints). Not the same as CAP consistency.
- **Isolation:** concurrent transactions behave as if serialized, to a degree set by the level.
- **Durability:** committed data survives crashes (fsync/WAL, replication).

4.5.2 Anomalies

What happens

Anomaly	
Dirty read	Read uncommitted data that later rolls back
Non-repeatable read	Same row read twice, different values
Phantom read	Same query returns different rows set (new rows inserted)
Lost update	Two read-modify-write cycles overwrite each other
Write skew	Two txns read overlapping data, write disjoint rows, violate an invariant (e.g., two doctors both go off-call)

4.5.3 Isolation levels

Level	Prevents
Read uncommitted	Nothing (rare)
Read committed	Dirty reads (default in Postgres, Oracle)
Repeatable read	Non-repeatable reads (MySQL InnoDB default; uses next-key locks)
Snapshot isolation	Reads from a consistent snapshot; prevents dirty/non-repeatable/phantom reads in most engines, **but allows write skew**
Serializable	All anomalies (2PL, or serializable snapshot isolation (SSI) as in Postgres)

4.5.4 Concurrency control

- **Pessimistic:** lock rows (`SELECT ... FOR UPDATE`). Simple, blocks others, risk of deadlock.
- **Optimistic:** version column; `UPDATE ... WHERE version = ?`; retry on conflict. Good when conflicts are rare.
- **MVCC:** keep multiple row versions; readers don't block writers. Postgres, MySQL InnoDB, Spanner.
- **Deadlocks:** detected by the DB (wait-for graph); mitigate with consistent lock ordering and short transactions.

4.5.5 Interview lines

- "The default isolation is read committed, so for a balance check-and-update I'd use `SELECT FOR UPDATE` or a conditional update to avoid lost updates."
- "Snapshot isolation allows write skew; for a constraint across rows I'd use serializable or an explicit lock."

4.6 Data Modeling Essentials

- **Normalization** reduces duplication and update anomalies; **denormalization** speeds reads at the cost of update complexity.
- **Access-pattern-driven modeling** (NoSQL): design tables around queries, not entities.
- **Keys:** natural vs surrogate; prefer stable surrogate IDs (Snowflake-style, time-sortable).
- **Soft delete vs hard delete;** tombstones in distributed stores.
- **Schema evolution:** add optional fields; never reuse field numbers in protobuf; expand then contract migrations (add column, backfill, switch reads, drop old).
- **Money:** integers in minor units plus currency code. Never floats.
- **Time:** store UTC; include time zone only for display.

05. Deep Distributed Systems: Replication, Partitioning, Consensus, Consistency, Transactions

---

5.1 Why Distributed Systems Are Hard (the fallacies)

Assume none of these are true: 1. The network is reliable. 2. Latency is zero. 3. Bandwidth is infinite. 4. The network is secure. 5. Topology doesn't change. 6. There is one administrator. 7. Transport cost is zero. 8. The network is homogeneous.

Partial failure is the core problem: a node can be slow, partitioned, or dead, and from another node these look the same. Every design decision is about how to make progress and stay correct despite that.

Timeouts are guesses. A timeout does not tell you whether the remote operation happened. This is why idempotency matters (Section 5.8).

5.2 Replication

5.2.1 Single-leader (primary–replica)

- All writes go to the leader; it streams the WAL/binlog to followers.
- **Sync replication:** leader waits for follower ack. No data loss on failover, slower writes, one slow follower stalls writes.
- **Async replication:** leader acks immediately. Fast, but failover can lose the last few writes.
- **Semi-sync:** wait for one follower.
- **Replication lag:** followers may serve stale reads. Solutions:
  - **Read-your-writes:** route the user's reads to the leader for a short window after writes, or track the write's LSN/timestamp and read from a replica that has caught up.
  - **Monotonic reads:** pin a user to one replica.
  - **Consistent prefix:** causally related writes stay in order.
- **Failover:**
  1. Detect leader failure (heartbeat timeout).
  2. Elect a new leader (most up-to-date follower).
  3. Reconfigure clients and old leader (fencing).
  - **Split brain:** two nodes think they are leader. Prevent with **fencing tokens** (monotonic token; storage rejects older tokens) and STONITH (power off the old leader).
  - **Lost writes:** async failover can drop committed-on-old-leader writes. Mitigate with sync replication or accepting the risk explicitly.

5.2.2 Multi-leader

- Each region has a leader; writes local; async replication between leaders.
- **Use for:** multi-region writes with low local latency; offline-capable clients (each device is a leader, e.g., calendars, notes).
- **Conflict resolution:**
  - Last write wins (LWW) by timestamp: simple, **silently loses data**.
  - Merge via application logic (e.g., union of sets).
  - CRDTs (Section 5.9).
  - Ask the user (Git-style merges).
- **Write conflicts** detected via version vectors.

5.2.3 Leaderless (Dynamo-style: Cassandra, Riak, Voldemort)

- Client (or coordinator) writes to N replicas; succeeds when W ack; reads from R replicas.
- **Quorum:** `R + W > N` ensures read and write sets overlap. Typical N=3, W=2, R=2.
- **Sloppy quorum + hinted handoff:** if a home replica is down, write to another node with a hint to forward later. Higher availability, weaker guarantees.
- **Read repair:** on read, detect stale replicas and update them.
- **Anti-entropy:** background process compares replicas using **Merkle trees** and syncs differences.

- **Limit:** quorum does not give linearizability on its own (races, sloppy quorums, concurrent writes).

5.2.4 Consensus-based replication (Raft/Paxos)

- Strongly consistent replicated state machine: etcd, ZooKeeper, Consul, CockroachDB ranges, Spanner (Paxos groups), Kafka KRaft.
- Majority commit; survives minority failures. A 5-node group tolerates 2 failures.

5.3 Partitioning (Sharding)

5.3.1 Strategies

Strategy	How	Pros	Cons
Range	Keys sorted into contiguous ranges	Range scans, ordered data	Hot spots on sequential keys (timestamps)
Hash (modulo)	`hash(key) % N`	Even spread	Resharding moves almost everything
Consistent hash	Ring with vnodes	Minimal movement on change	Less range locality
Directory / lookup	Table maps key → shard	Flexible, move any key	Directory is a bottleneck and must be HA
Geo / tenant	By region or tenant	Data locality, compliance	Uneven load, cross-tenant queries hard

5.3.2 Secondary indexes on partitioned data

- **Document-partitioned (local index):** each shard indexes its own docs. Writes are local; reads for a secondary attribute must **scatter-gather** to all shards.
- **Term-partitioned (global index):** index partitioned by term. Reads go to one shard; writes update index shards across the cluster (often async, so index may lag).

5.3.3 Rebalancing

- Fixed number of partitions (e.g., 1024) assigned to nodes; move whole partitions when nodes join. Used by Kafka, Riak, Elasticsearch.
- Dynamic splitting (range-based, e.g., HBase, Spanner) when a partition gets too big or hot.
- Avoid automatic rebalancing storms: throttle moves, and rebalance only on true membership changes.

5.3.4 Hot partitions and hot keys

- Detect with per-partition metrics.
- **Salting:** `key + random suffix` for write-hot keys; reads fan out to all suffixes and merge.
- **Split the entity:** e.g., per-user counters aggregated periodically.
- **Local aggregation / batching** before writing.
- **Celebrity problem:** a key with millions of followers or a viral item.

5.3.5 Cross-partition operations

- Joins across shards are expensive: denormalize, or do them in a stream/batch job.
- Transactions across shards need 2PC, sagas, or a single shard design (co-locate related data by a tenant or user key).
- Choose the shard key so the common query hits **one** shard.

5.4 Consistency Models (know the spectrum)

From strongest to weakest: 1. **Strict serializability** (linearizable transactions) — Spanner, CockroachDB (with caveats). 2. **Linearizability** — every operation appears to take effect atomically at a point between its invocation and response. Like a single copy. Needed for leader election, locks, unique constraints. 3. **Sequential consistency** — all nodes see the same order, not necessarily real-time order. 4. **Causal consistency** — causally related writes are seen in order; concurrent writes may differ. Achievable with available systems. 5. **Session guarantees:** read-your-writes, monotonic reads, monotonic writes, writes-follow-reads. 6. **Eventual consistency** — replicas converge if writes stop. No bound on staleness.

Practical mapping: - Money balance, inventory decrement, unique usernames → linearizable on the key (or a serializable

transaction). - Like counts, feed ordering, view counters → eventual is fine. - User profile after update → read-your-writes.

5.4.1 Linearizability costs

- Requires coordination (quorum or leader round trip) on every op. Latency floor = cross-region RTT when global.
- Not a free lunch: stale reads from followers break it. Reads must go to the leader or use read-index / lease reads.

5.5 CAP and PACELC (precise version)

- **CAP:** when a network **partition** happens, a system must pick **consistency** (refuse some requests to stay correct) or **availability** (answer, possibly stale). Only during partitions.
- **PACELC:** if **P**artition → A or C; **E**lse (normal) → **L**atency or **C**onsistency.
  - Dynamo/Cassandra: PA/EL (available, low latency; eventual).
  - Spanner: PC/EC (consistent; higher latency).
  - MongoDB default configs: depends on write concern and read preference.
- **Mistakes to avoid in interviews:** saying “pick 2 of 3” as a permanent choice; calling a system “CA” for a distributed deployment (CA only makes sense for a single node).
- **Good sentence:** “Partitions are rare but latency is always there, so I optimize the normal path for latency and accept eventual consistency for non-critical reads.”

5.6 Consensus, Leader Election, Coordination

5.6.1 The problem

Get a group of nodes to agree on a value (or a sequence of values) despite crashes and message delays. Used for: leader election, config, membership, atomic commit, lock services.

5.6.2 Paxos

- Proposers, acceptors, learners. Two phases: prepare/promise, accept/accepted. Requires a majority.
- Correct but hard to implement and reason about (Multi-Paxos for logs).

5.6.3 Raft (understandable consensus)

- **Leader election:** nodes have terms; a follower times out, becomes candidate, requests votes; majority wins. Randomized timeouts avoid split votes.
- **Log replication:** leader appends entries and sends to followers; entry is **committed** when a majority stores it; applied to the state machine in order.
- **Safety:** a candidate only wins if its log is at least as up to date as a majority.
- **Membership changes** done with joint consensus or single-server changes.
- Used by: etcd, Consul, CockroachDB (per range), TiKV, Kafka KRaft (KRaft is Raft-based).

5.6.4 Coordination services

- **ZooKeeper:** ephemeral znodes (session-bound), watches, sequential nodes. Used for leader election, locks, config.
- **etcd:** key-value with watch and leases; backing store of Kubernetes.
- **Typical uses:** leader election, service discovery, distributed config, distributed locks (with fencing).

5.6.5 Distributed locks

- A lease-based lock: holder must renew; if it pauses (GC, network), the lease expires and another holder takes over.
- **Danger:** the old holder doesn’t know it lost the lock and may write. **Fencing tokens** fix this: each lock grant has an increasing token; the resource (storage) rejects writes with lower tokens.
- **Redlock critique (Kleppmann):** using multiple Redis instances with timing assumptions is unsafe under clock jumps and pauses. Use a consensus-backed lock (etcd, ZooKeeper) with fencing for correctness-critical locks; Redis locks are fine for efficiency (avoid duplicate work), not correctness.

5.7 Failure Detection and Membership

- **Heartbeats** with timeouts: simple; false positives under load or GC pauses.

- **Phi accrual failure detector:** outputs a suspicion level based on heartbeat inter-arrival statistics (Cassandra, Akka).
- **Gossip protocols:** each node periodically exchanges state with random peers. Scales O(log N) rounds to spread information (Cassandra, Consul, Dynamo membership).
- **SWIM:** gossip-based membership with indirect pings.
- **Fencing:** a node that's been declared dead must be prevented from acting (fencing tokens, epoch numbers).
- **Timeouts design:** set per call, derived from p99 latency, with a retry budget.

5.8 Idempotency and Exactly-Once

5.8.1 Delivery semantics recap

- **At-most-once:** send and don't retry. May lose.
- **At-least-once:** retry until acked. May duplicate.
- **Exactly-once (effect):** at-least-once delivery + idempotent processing, or transactional atomic "process + record offset/ID".

5.8.2 Idempotency patterns

- **Idempotency key:** client-generated key per logical operation; server stores key → result (with TTL) and returns the stored result on retry. Must check-and-store atomically (unique constraint).
- **Natural idempotency:** `PUT` with full resource state, `SET x=5` not `x=x+1`.
- **Dedup table:** `processed_messages(message_id PK)` inserted in the same DB transaction as the business change.
- **Conditional writes:** `UPDATE ... WHERE version=?`.

5.8.3 Outbox pattern (publish events reliably)

1. In one local DB transaction: update business table and insert event into `outbox` table.
2. A relay (poller or CDC from the WAL via Debezium) reads outbox and publishes to Kafka, marks as sent.
3. Consumers are idempotent.

- Avoids the dual-write problem (DB commit succeeds, Kafka publish fails, or vice versa).

5.8.4 Inbox pattern

- Consumer records processed message IDs in the same transaction as its state change. Duplicates skipped.

5.9 Distributed Transactions

5.9.1 Two-phase commit (2PC)

- Coordinator asks participants to **prepare** (lock resources, write intent to log); if all vote yes, coordinator sends **commit**; else **abort**.
- **Pros:** atomic across resources.
- **Cons:** blocking. If coordinator crashes after prepare, participants are stuck holding locks until it recovers. Latency = 2 round trips + logging.
- Used inside some databases (XA transactions), Spanner (with Paxos groups), not common across microservices.

5.9.2 Saga

- Sequence of local transactions T1..Tn, each with compensation C1..Cn for rollback.
- **Choreography:** services react to each other's events. Decentralized; harder to follow.
- **Orchestration:** a coordinator (e.g., Temporal workflow) drives steps and compensations. Easier to reason about and monitor.
- **Trade-offs:** no isolation (intermediate states visible), compensations must be idempotent and sometimes semantic (refund, not "un-charge").
- **Design:** define the state machine, timeouts per step, retry policy, and what to do if a compensation fails (manual queue).

5.9.3 TCC (Try-Confirm-Cancel)

- Reserve resources in "try", finalize in "confirm" or release in "cancel". Used for payments and bookings. Requires each service to support reservation.

5.9.4 Choosing

- Same DB: local transaction.
- Few resources, one DB cluster: 2PC inside DB or a distributed SQL DB.
- Microservices: saga + outbox + idempotency. Accept eventual consistency and design user-visible “pending” states.

5.10 Logical Structures and Conflict Resolution

5.10.1 CRDTs (Conflict-free Replicated Data Types)

- Data types whose concurrent updates merge deterministically without coordination.
- **G-Counter:** grow-only counter; each node has its own slot; value = sum of max per node.
- **PN-Counter:** two G-Counters (increments and decrements).
- **G-Set / OR-Set:** add/remove sets (OR-Set uses unique tags).
- **LWW-Register:** last write wins by timestamp (simple, can lose updates).
- **Sequence CRDTs (RGA, Yjs, Automerge):** collaborative text editing.
- Use for: collaborative editing, offline-first apps, counters in multi-region systems.

5.10.2 Version vectors and sibling resolution

- Detect concurrency; keep siblings (Dynamo/Riak) and let the app merge, or apply LWW.

5.11 Distributed Systems Patterns Glossary

- **Leader and followers:** one coordinator, others replicate.
- **Heartbeat:** periodic liveness signal.
- **Quorum:** majority or W/R set.
- **Lease:** time-bound exclusive right; must renew.
- **Generation / epoch / term:** monotonic number to reject stale leaders.
- **Write-ahead log:** durability first.
- **Two-phase commit, saga:** atomic work across nodes.
- **Gossip:** epidemic information spread.
- **Consistent hashing:** minimal data moves on membership change.
- **Bulkhead, circuit breaker:** isolation and fail-fast (see Section 07 file).

5.12 Interview Questions on This Topic (short answers)

1. **How do you ensure only one leader?** Consensus (Raft/Paxos) with a lease, plus fencing tokens on the protected resource.
2. **Why not just use Redis locks?** Redis lock timing assumptions can allow two holders under pauses; fine for efficiency locks, not correctness. Use a consensus store with fencing for correctness.
3. **How do you handle replica lag for a user who just wrote?** Read from leader for a window, or use a version token and wait-for-version on read.
4. **What is the danger of sloppy quorums?** Reads can miss the latest write; it’s availability over consistency.
5. **How do you migrate a shard key?** Dual-write or dual-read with backfill, verify with reconciliation, then cut over; keep a rollback path.

06. Deep Messaging, Streaming, and Caching

6.1 Messaging Models

Model	Example	Semantics
Point-to-point queue	SQS, RabbitMQ queue	Each message consumed by one worker; removed on ack
Pub/sub (fan-out)	SNS, Redis pub/sub, RabbitMQ fanout exchange	Each subscriber gets a copy
Log (replayable stream)	Kafka, Kinesis, Pulsar	Messages retained by time/size; consumers keep offsets; replay possible

Queue vs log, the key distinction: queues delete on ack and don't support replay or multiple independent readers well; logs retain data and let each consumer group move at its own pace.

6.1.1 Choosing

- Task queue with retries and DLQ, no replay needed → SQS or RabbitMQ.
- Event stream, many consumers, replay, high throughput, ordering per key → Kafka.
- Simple in-process or low-stakes notifications → Redis pub/sub (no persistence: a disconnected subscriber misses messages).
- Serverless and managed, moderate throughput → SQS/SNS, EventBridge, Pub/Sub (GCP).

6.2 Kafka in Depth

6.2.1 Structure

- **Topic** → split into **partitions**. Each partition is an ordered, append-only log of records with offsets.
- **Segments:** a partition is stored as segment files (log + index). Old segments deleted by retention (time/size) or compacted.
- **Broker:** a server hosting partition replicas.
- **Replication factor (RF):** typically 3. One replica is the **leader** for a partition; others are followers.
- **ISR (In-Sync Replicas):** the set of replicas caught up with the leader.
- **Controller:** elected broker managing partition leadership. Modern Kafka uses **KRaft** (Raft-based metadata quorum). ZooKeeper support was removed in Kafka 4.0.

6.2.2 Producer guarantees

- `acks=0`: fire and forget.
- `acks=1`: leader wrote it. Lost if leader dies before replicating.
- `acks=all` (`-1`): all ISR have it. Combined with `min.insync.replicas=2` and RF=3, tolerates one broker failure without loss.
- **Idempotent producer** (`enable.idempotence=true`): producer ID + per-partition sequence numbers; broker deduplicates retries. No duplicates from producer retries within a session.
- **Transactions:** atomic writes across multiple partitions/topics plus consumer offset commits (`read_committed` consumers see only committed data). Gives exactly-once for Kafka-to-Kafka pipelines (e.g., Kafka Streams).
- Batching: `linger.ms` and `batch.size` trade latency for throughput. Compression (lz4, zstd) reduces network and disk.

6.2.3 Consumer model

- **Consumer group:** each partition is assigned to exactly one consumer in a group. Max useful consumers = partitions count.
- **Offsets:** committed per group per partition, stored in `__consumer_offsets`.
- **Commit strategy:**
  - Auto-commit: may commit before processing finishes → message loss on crash.
  - Manual commit after processing → at-least-once (duplicates on crash).
- **Rebalancing:** when consumers join/leave, partitions are reassigned. Cooperative/incremental rebalancing reduces stop-the-world pauses.
- **Consumer lag:** end offset minus committed offset. Alert on growth rate, not just absolute value.

- **Poison message:** a message that always fails. Retry with backoff, then dead letter topic. Don't block the partition forever.

6.2.4 Ordering

- Ordering is guaranteed **within a partition** only. Choose the key so events needing order share a key (e.g., `order_id`).
- Changing partition count reshuffles key→partition mapping, breaking ordering for in-flight keys. Plan partition count up front.

6.2.5 Retention and compaction

- **Time/size retention:** delete old segments.
- **Log compaction:** keep the latest value per key (changelog/state topics, CDC snapshots).
- Tombstone (null value) deletes a key during compaction.

6.2.6 Performance facts

- Sequential disk writes + page cache + zero-copy (`sendfile`) make Kafka fast.
- Throughput bound by partitions, network, and batching. Hot partitions limit throughput: skewed keys are the common cause.

6.2.7 Kafka failure scenarios

- **Leader broker dies:** a new leader is elected from ISR. With `acks=all` and `min.insync.replicas`, no acked data lost.
- **Unclean leader election** (allowing non-ISR leader) risks data loss; disable it for correctness-critical topics.
- **Consumer crash:** partitions reassigned; messages after last commit reprocessed (idempotency needed).
- **Producer crash mid-batch:** idempotent producer prevents duplicate append on retry within session.

6.3 Other Brokers (know the differences)

- **RabbitMQ:** exchanges (direct, topic, fanout, headers) route to queues; acknowledgments; per-message TTL; priorities; good for complex routing and task queues; not built for massive replay.
- **Amazon SQS:** managed queue; standard (at-least-once, best-effort ordering) vs FIFO (exactly-once processing within dedup window, ordered per group); visibility timeout; DLQ.
- **Amazon Kinesis / Google Pub/Sub / Pulsar:** streaming with partitions/shards; Pulsar separates compute and storage (BookKeeper).
- **Redis Streams:** log-like structure with consumer groups and pending entries (ack). Good for moderate scale inside an app ecosystem.

6.4 Stream Processing

6.4.1 Concepts

- **Event time vs processing time:** when it happened vs when it was processed. Always use event time for correct analytics.
- **Windows:**
  - Tumbling: fixed, non-overlapping (every 1 min).
  - Sliding (hopping): fixed size, overlapping (5 min window every 1 min).
  - Session: gap-based (close after 30 min inactivity).
- **Watermark:** a heuristic "no events older than T are expected." Triggers window results.
- **Late data:** events after the watermark. Options: drop, allowed lateness window with updates, or side output for corrections.
- **Stateful processing:** keep per-key state (counts, joins) in local state store (RocksDB) backed by changelog topic.
- **Checkpointing:** periodically snapshot state + source offsets (Flink Chandy–Lamport-style barriers). On failure, restore and replay from offsets → exactly-once state effects.
- **Exactly-once sinks:** transactional or idempotent writes (upsert by key or event ID).

6.4.2 Joins in streams

- Stream–stream join: both inputs buffered within a time window.
- Stream–table join: enrich events with latest state from a changelog-backed table.

- Repartitioning is needed so both sides share the join key on the same partitions (co-partitioning).

6.4.3 Frameworks

- Kafka Streams (library, no separate cluster), Apache Flink (cluster, rich event-time semantics), Spark Structured Streaming (micro-batch), Beam (portable model).

6.4.4 Lambda vs Kappa architecture

- **Lambda:** batch layer (complete, slow, recomputes truth) + speed layer (fresh, approximate) + serving layer merging both. Complex: two code paths.
- **Kappa:** everything is a stream; reprocess by replaying the log with a new job version. Simpler, needs a replayable log with enough retention.

6.5 Data Lake Table Formats (Iceberg, Delta, Hudi)

- Files on object storage (Parquet) plus **metadata** tracking which files belong to the table snapshot.
- **Iceberg:** snapshot isolation for tables, hidden partitioning, partition evolution, time travel, schema evolution, metadata tree (manifest lists → manifests → data files).
- **Benefits over raw Parquet folders:** atomic commits, no partial reads, safe concurrent writers, fast planning via manifests, file-level pruning by column stats.
- **Small file problem:** many small files from streaming writes slow queries. Fix with compaction (rewrite into larger files) and sorting.
- **Maintenance:** expire snapshots, delete orphan files, rewrite manifests.
- **Query engines:** Spark, Trino/Presto, Flink, DuckDB, Druid ingestion (Iceberg input).

6.6 Caching in Depth

6.6.1 Where caches live

- Browser cache, CDN edge, reverse proxy cache, application in-process cache (Caffeine/Guava, LRU maps), distributed cache (Redis/Memcached), database buffer pool (internal).
- Principle: cache closest to the consumer that can tolerate its staleness.

6.6.2 Redis internals (what to say)

- Single-threaded command execution (I/O threads in newer versions), so commands are atomic. Pipelining and Lua scripts give multi-command atomicity.
- Data types: strings, hashes, lists, sets, sorted sets (score-based ranking, presence, leaderboards), streams, HyperLogLog (approximate cardinality), bitmaps.
- **Persistence:** RDB snapshots (point-in-time, fork-based), AOF (append log, fsync policies).
- **Replication:** async primary → replicas. Failover via Sentinel (managed failover for non-cluster) or Redis Cluster (built-in).
- **Redis Cluster:** 16,384 hash slots, distributed across primaries; key hashing with `{hashtag}` for co-location. Multi-key operations need same slot.
- **Memory:** `maxmemory` with eviction policies: `allkeys-lru`, `volatile-lru`, `allkeys-lfu`, `volatile-ttl`, `noeviction`.
- Memcached: multi-threaded, simpler, no persistence or rich types.

6.6.3 Eviction policies

- **LRU:** evict least recently used. Good general default.
- **LFU:** evict least frequently used. Better for skewed popularity, handles scan pollution.
- **TTL/random:** simple, cheap.
- **Admission policies (TinyLFU, used by Caffeine):** only admit new items likely to be reused, protects hot set.

6.6.4 Consistency patterns

- **Cache-aside (lazy loading):** read: check cache → miss → DB → populate. Write: update DB → **delete** cache key (preferred over set, avoids race with concurrent readers repopulating stale value).
- **Race in cache-aside:** reader reads old DB value, writer updates DB and deletes cache, reader then writes old value into cache → stale until TTL. Mitigate with short TTL, versioned values, or CDC-driven invalidation.

- **Write-through:** write to cache and DB in one path; still need transaction semantics for failure.
- **Write-behind:** async flush from cache to DB; risk of data loss on cache crash; batching gains.
- **Refresh-ahead:** refresh before expiry for predictable hot keys.
- **CDC-based invalidation (Debezium):** DB change log drives cache deletes; robust against app bugs.

6.6.5 Stampede and hot-key protection

- **Single-flight / request coalescing:** one request recomputes, others wait for the result.
- **Probabilistic early expiration:** recompute slightly before TTL, randomly by request, so one hot expiry doesn't cascade.
- **Stale-while-revalidate:** serve stale value while refreshing in background.
- **Jitter TTL:** `ttl = base + random(0, base*0.1)` to avoid synchronized expiry.
- **Hot key replicas:** `key#0..key#N` with random selection on read.
- **Local (L1) cache** per app server with short TTL absorbs hot-key reads.

6.6.6 Cache penetration and avalanche

- **Penetration:** queries for non-existent keys always miss and hit DB. Fix: cache negative results briefly; Bloom filter of existing keys.
- **Avalanche:** many keys expire together or cache node dies → DB overload. Fix: TTL jitter, multi-node replication, circuit breakers, load shedding on DB.

6.6.7 Cache sizing

- Hit ratio drives value: `avg latency = h * t_cache + (1-h) * t_db`. Model hit ratio from Zipf-like access distribution (small set gets most traffic).
- Working set: keys accessed within a window; size cache to hold it.

6.7 CDN Deep Dive

- Anycast routing directs clients to nearest POP.
- Edge caching with `Cache-Control` (`public, max-age=…, s-maxage=…`, `stale-if-error`, `stale-while-revalidate`).
- Vary headers and cache keys (query string, cookies) can destroy hit rates if too broad.
- Signed URLs/cookies for private content; token auth at edge.
- Video: segmented streaming (HLS/DASH) cached per segment; prefetch next segments.

6.8 Interview Questions (with short model answers)

1. **Kafka lost messages after broker crash — why?** Likely `acks=1` or unclean leader election, or RF=1. Fix: `acks=all`, `min.insync.replicas=2`, RF=3, disable unclean election.
2. **How do you get exactly-once in a pipeline?** Idempotent producer + transactions for Kafka-to-Kafka; idempotent upserts or dedup keys for external sinks.
3. **Consumer lag keeps growing. What now?** Check consumer throughput per partition, hot partition skew, downstream DB latency; scale consumers to partition count; batch writes; if skew, rekey.
4. **How do you order events per user?** Key by user ID; single partition per key; consumer processes partitions sequentially.
5. **Cache and DB disagree. Why?** Race in cache-aside or failed invalidation; use delete-after-write, TTL bound, CDC invalidation, and reconciliation jobs.
6. **Redis is the bottleneck. What do you do?** Check hot keys (`--hotkeys`, `MONITOR` sampling), add read replicas, shard via Cluster, use local L1 cache, reduce payload size, pipeline commands.
7. **Why do you need watermarks?** To know when a window can close despite out-of-order events; trade-off between completeness and latency.

07. Deep APIs, Resilience, Observability, Security, Deployment

---

7.1 API Design

7.1.1 REST

- Resources as nouns (`/orders/{id}/items`), HTTP verbs with semantics:
  - `GET` safe and idempotent; `PUT` idempotent full replace; `PATCH` partial; `DELETE` idempotent; `POST` not idempotent unless keyed.
- Status codes: 200/201/204; 400 bad input; 401 unauthenticated; 403 forbidden; 404; 409 conflict (version mismatch); 412 precondition failed (ETag); 422 semantic; 429 rate limited (+`Retry-After`); 500/503/504.
- **Pagination:**
  - Offset/limit: simple; slow on deep pages; unstable under inserts (duplicates/skips).
  - Cursor/keyset: `WHERE (created_at, id) < (?, ?) ORDER BY … LIMIT n`; stable and fast with an index.
- **Filtering, sorting:** whitelist fields; avoid arbitrary SQL.
- **Partial responses / field selection** for mobile.
- **Versioning:** URL (`/v1`), header, or content negotiation. Prefer additive changes; never remove or rename fields without a version bump.
- **Conditional requests:** `ETag`/`If-Match` for optimistic concurrency (lost update prevention).
- **Idempotency-Key header** on POST for payments and creates.
- **Long-running operations:** return `202 Accepted` with a status resource or webhook.

7.1.2 gRPC

- Protobuf schemas, HTTP/2, streaming (unary, server, client, bidirectional), strong typing, codegen.
- Use for internal service-to-service with low latency and high throughput. Harder to use from browsers (needs gRPC-Web).
- Schema evolution: never reuse field numbers; add optional fields; deprecate, don't remove.
- Deadlines propagate across calls (deadline budget).

7.1.3 GraphQL

- Client specifies fields; single endpoint; typed schema.
- Good for BFF and many client shapes; reduces over-fetching.
- **Risks:** N+1 queries (fix with DataLoader batching), expensive nested queries (limit depth, complexity analysis, persisted queries), caching is harder (no HTTP GET caching by default), authorization must be per-field.

7.1.4 Webhooks and events

- Push notifications to customer URLs: sign payloads (HMAC), include event ID and timestamp, retry with exponential backoff, expose delivery logs, allow replay.
- Consumers must be idempotent (events can repeat).

7.1.5 API gateway responsibilities

- AuthN (token validation), rate limiting, routing, TLS termination, request/response transforms, caching, metrics and logs, WAF, quota management, versioned routing.

7.1.6 Backward compatibility rules

- Safe changes: adding optional fields, adding endpoints, adding enum values (if clients tolerate unknown values).
- Breaking: removing/renaming fields, changing types, changing semantics, making optional required.
- Use consumer-driven contract tests (Pact) to catch breaks.

7.2 Resilience Patterns

7.2.1 Timeouts

- Every remote call has a timeout. Without it, a slow dependency consumes all threads.
- Derive from p99 of the dependency plus margin; propagate remaining deadline to downstream calls.

7.2.2 Retries

- Retry only idempotent operations, or ones with idempotency keys.
- **Exponential backoff with full jitter:** `sleep = random(0, min(cap, base * 2^attempt))`.
- **Retry budget:** cap retries to a fraction of total traffic (e.g., 10%) to avoid retry storms.
- Retry on transient errors (timeouts, 503, 429 with Retry-After), not on 400/401/403/404.
- Retries multiply load: 3 layers × 3 retries = 27× traffic. Retry at one layer, usually the edge or the caller.

7.2.3 Circuit breaker

- States: **Closed** (calls pass; count failures) → **Open** (fail fast for a cool-down) → **Half-open** (allow a few trial calls; success closes, failure reopens).
- Thresholds: failure rate over a sliding window (e.g., >50% over 20 calls).
- Pair with fallbacks: cached response, default value, degraded feature.

7.2.4 Bulkhead

- Isolate resources per dependency: separate thread pools, connection pools, or semaphores. One slow dependency can't starve others.

7.2.5 Load shedding and backpressure

- Reject early when overloaded (503 or 429), prioritize critical traffic.
- Queue with bounded size; drop oldest or reject newest; use adaptive concurrency limits (e.g., gradient-based limiter based on latency).
- Backpressure from consumers to producers: bounded buffers, pull-based consumption, rate-limited producers.

7.2.6 Graceful degradation

- Show cached data, hide non-critical widgets, reduce quality (lower image resolution), fall back to a simpler algorithm.
- Define which features are critical per product (e.g., checkout > recommendations).

7.2.7 Health checks, graceful shutdown, and startup

- Liveness (process is alive), readiness (ready for traffic), startup probe for slow boot.
- On SIGTERM: fail readiness, drain connections, finish in-flight work, then exit.

7.2.8 Chaos and game days

- Inject failures (kill nodes, add latency, partition networks) in staging and controlled production. Verify timeouts, retries, failover, and alerts behave as designed.

7.3 Rate Limiting and Quotas (depth beyond HLD guide)

- Limit dimensions: per user, per API key, per IP, per endpoint, per tenant (fairness).
- **Global vs local:** local per-node limits are cheap but sum to more than the global limit; global needs shared state (Redis) with latency cost. Hybrid: local token buckets with periodic sync.
- **Multi-tenant fairness:** weighted fair queuing; per-tenant queues.
- **Quota vs rate:** quotas are long-window totals (monthly); rate limits are short-window (per second).
- Communicate limits: `X-RateLimit-Limit`, `X-RateLimit-Remaining`, `X-RateLimit-Reset`, `Retry-After`.

7.4 Observability

7.4.1 Three pillars plus

- **Metrics:** numeric time series (counters, gauges, histograms). Cheap; aggregatable; use for alerts and dashboards.
- **Logs:** discrete events with context. Structured (JSON) with request ID, user/tenant, latency, error. Expensive at scale; sample high-volume logs.
- **Traces:** a request's path across services as spans with timing and parent-child links (OpenTelemetry standard).

Essential for microservice latency debugging.
- **Profiles:** CPU/memory profiles in production (continuous profiling).

7.4.2 Metric types and what to measure

- **RED** for request-driven services: Rate, Errors, Duration (p50, p95, p99).
- **USE** for resources: Utilization, Saturation (queue depth), Errors.
- **Golden signals:** latency, traffic, errors, saturation.
- **Histograms over averages:** averages hide tails. Use percentiles from histograms; be careful averaging percentiles across hosts (aggregate histogram buckets instead).
- **Cardinality:** labels like `user_id` explode storage. Use bounded labels (route template, not raw path).

7.4.3 SLI, SLO, SLA, error budgets

- **SLI:** measured indicator (fraction of requests < 300 ms and successful).
- **SLO:** target (99.9% over 30 days).
- **SLA:** contract with consequences.
- **Error budget:** 1 − SLO. With 99.9%, about 43 minutes of full downtime per 30 days. Spend on releases and experiments; freeze when exhausted.
- **Burn-rate alerts:** alert when budget is being consumed faster than sustainable (multi-window, e.g., 14× over 1 h and 6× over 6 h).

7.4.4 Alerting philosophy

- Alert on symptoms users feel (SLO burn, error rate, latency), not every cause (CPU 80%).
- Every page should be actionable and have a runbook.
- Reduce noise: deduplicate, group, route by ownership.

7.4.5 Distributed tracing details

- Propagate `traceparent` (W3C Trace Context) through HTTP headers and message headers (Kafka record headers) to link async hops.
- Sampling: head-based (decide at start) vs tail-based (decide after seeing errors/latency).

7.4.6 Incident management

- Severity levels, incident commander, communication cadence, status page.
- Postmortem: blameless, timeline, root cause, contributing factors, action items with owners.

7.5 Security

7.5.1 Authentication and authorization

- **AuthN:** password (bcrypt/argon2/scrypt with per-user salt, never plaintext or fast hashes), MFA, passkeys/WebAuthn, SSO via SAML/OIDC.
- **OAuth 2.0:** delegated authorization. Flows: authorization code with PKCE (for web/mobile), client credentials (service-to-service). Implicit flow deprecated.
- **OpenID Connect (OIDC):** identity layer on OAuth2; ID token (JWT) says who the user is.
- **Access tokens:** short-lived (minutes), refresh tokens long-lived and rotated, revocable.
- **JWT:** signed (JWS) claims (`sub`, `exp`, `aud`, `iss`, roles). Validate signature, `exp`, `aud`, `iss`; prefer asymmetric keys (RS256/ES256); don't put secrets in payload (only base64 encoded). Revocation is hard for stateless tokens; mitigate with short TTL or a denylist.
- **Session cookies:** HttpOnly, Secure, SameSite; server-side session store allows instant revocation.
- **AuthZ models:** RBAC (roles), ABAC (attributes), ReBAC (relationships, Zanzibar/OpenFGA style). Enforce at service boundary, not only at gateway.
- **Keycloak** (your resume): OIDC/OAuth2 provider with realms, roles, and client scopes.

7.5.2 Service-to-service security

- **mTLS:** both sides present certificates; identity by SPIFFE IDs (service mesh like Istio/Linkerd).
- **Zero trust:** verify every request; no implicit trust from the internal network.
- **Secrets management:** Vault, cloud KMS/Secrets Manager; rotate; never in code or images.

7.5.3 Data protection

- **In transit:** TLS 1.2+ (prefer 1.3).
- **At rest:** AES-256 with envelope encryption (data keys encrypted by a KMS master key).
- **PII:** minimize collection, tokenize, restrict access, audit.
- **Compliance:** GDPR (right to erasure → data deletion pipelines and backups policy), PCI DSS (don't store card data; use tokenization via processor), HIPAA.

7.5.4 Common attacks and defenses

Attack	Defense
SQL injection	Parameterized queries, ORM, least-privilege DB user
XSS	Output encoding, CSP header, HttpOnly cookies
CSRF	SameSite cookies, CSRF tokens, custom headers
SSRF	Allowlist outbound hosts, block metadata IPs
Broken access control (IDOR)	Check ownership on every object access; don't trust IDs
DDoS	Edge scrubbing, CDN, rate limits, SYN cookies, autoscaling buffers
Credential stuffing	Rate limits, MFA, breached-password checks
Replay attacks	Nonces, timestamps, idempotency keys
Supply chain	Dependency scanning, signed images, SBOM

7.5.5 Payload signing and webhooks

- HMAC-SHA256 over body with shared secret and timestamp; reject old timestamps (replay window).

7.6 Deployment and Release Engineering

- **Immutable artifacts:** build once, promote through environments.
- **Blue-green:** two full environments; switch traffic. Instant rollback; double cost during switch.
- **Canary:** route 1% → 10% → 50% → 100% with automated metric checks (error rate, latency) and auto rollback.
- **Rolling update:** replace instances in batches; needs readiness and backward-compatible versions running side by side.
- **Feature flags:** decouple deploy from release; kill switches; percentage rollouts; cleanup flags after launch.
- **Database migrations:** expand → migrate → contract. Never deploy code that needs a schema change without the schema being ready first. Avoid long locks (online DDL, batched backfills).
- **Config:** externalized, versioned, hot-reloadable where safe.
- **Infrastructure as code:** Terraform, Helm, GitOps (Argo CD).

7.7 Multi-Region and Disaster Recovery

- **RTO:** how long to recover. **RPO:** how much data loss is acceptable.
- **Backup and restore:** cheapest, highest RTO/RPO. Test restores.
- **Pilot light / warm standby / hot active-active:** increasing cost and decreasing RTO.
- **Active-passive:** one region serves; failover on incident. Simpler, DNS switch or LB switch.
- **Active-active:** all regions serve; requires conflict handling (partitioned ownership by user/tenant, CRDTs, or global DB like Spanner/CockroachDB). Use **geo-partitioning**: each user's home region owns their data.
- **Cell-based architecture:** split the system into independent cells, each serving a subset of users. Blast radius limited to one cell.
- **Data residency:** keep EU data in EU; route accordingly.
- **Global data:** strongly consistent global writes are expensive (cross-region quorum latency). Keep most writes regional.

7.8 Cost Awareness (say this in senior interviews)

- Biggest levers: storage tiering (hot/warm/cold; S3 Glacier), caching (reduce DB/compute), right-sizing, autoscaling, reserved/spot capacity, compression, data retention policies, egress costs (CDN and cross-region transfer are often

the hidden cost).
- Trade-off framing: “This design costs ~X per month; I’d reduce it by Y by moving Z to cold storage.”

7.9 Testing for Distributed Systems

- Unit tests for logic; integration tests with real dependencies (Testcontainers).
- Contract tests between services (Pact).
- Load testing (k6, Gatling) against realistic traffic shapes; find the knee of the latency curve.
- Fault injection (Toxiproxy, Chaos Mesh, Gremlin).
- Property-based and model-based testing for state machines.
- Jepsen-style testing for consistency claims (understand the concept; it checks linearizability under partitions).

7.10 Interview Questions (short answers)

1. **How do you prevent a retry storm?** Retry budget, exponential backoff with jitter, circuit breakers, retry only at one layer, respect Retry-After.
2. **Access token is stolen. What now?** Short TTL limits window; revoke refresh token; use token binding or DPoP; detect anomalies; force re-auth for sensitive actions.
3. **Design an SLO for checkout.** SLI: fraction of checkout requests succeeding under 1 s. SLO: 99.9% over 28 days. Error budget ~40 min. Alert on burn rates.
4. **Canary shows higher p99 but same error rate. Action?** Investigate latency regression via traces; compare by version; roll back if p99 breaches SLO.
5. **How do you migrate a column type on a live table?** Expand (add new column), dual write, backfill in batches, switch reads, stop writes to old, contract (drop old). Keep rollback.
6. **Active-active across regions for a payment system?** Usually not for the ledger. Use regional ownership per account, async replication for read models, and a strongly consistent store for balances.

08. Architecture Patterns and Reference Designs

8.1 Architectural Styles

Monolith

- One deployable unit. Simple to develop, test, deploy, and debug. Transactions are local.
- Scales vertically and by horizontal replicas of the whole app. Breaks down when teams and deploy frequency collide.
- **Modular monolith:** strict module boundaries, separate schemas, internal APIs. Good default for small teams. Can be split later.

Microservices

- Independently deployable services, each owning its data.
- **Benefits:** team autonomy, independent scaling, fault isolation, technology choice.
- **Costs:** network calls (latency, failures), distributed transactions, observability, versioning, operational overhead, data duplication, harder testing.
- **Decompose by:** business capability or bounded context (DDD), not technical layer. Signs of a bad split: every feature touches 5 services; chatty calls; shared database.
- **Database per service:** each service owns its data; others access via API or events.
- **Start monolith, split when pain is real.** Strangler fig migration: route pieces of traffic to new services gradually behind a facade.

Event-driven architecture

- Services publish facts (“OrderPlaced”) to a broker; interested services react.
- **Choreography** (decentralized) vs **orchestration** (central workflow).
- Decouples producers and consumers in time and space; enables new consumers without changing producers.
- **Risks:** eventual consistency, duplicate events, out-of-order events, schema drift, hard-to-trace flows (needs tracing and event catalogs).
- **Event notification** (small “something happened”, fetch details) vs **event-carried state transfer** (include the data) vs **event sourcing** (events are the source of truth).

Event sourcing

- Store the sequence of events as the source of truth; current state = fold over events. Snapshots every N events for speed.
- **Benefits:** full audit trail, temporal queries, replay for new projections, debugging.
- **Costs:** schema evolution of events (upcasting), eventual consistency for read models, complexity, GDPR deletion (crypto-shredding: encrypt per-user data with a key you delete).

CQRS (Command Query Responsibility Segregation)

- Separate write model (commands, normalized, consistent) and read models (queries, denormalized, optimized per view).
- Read models are updated from events (eventual consistency). Pair with event sourcing often, but not required.
- Use when read and write shapes or scaling needs differ greatly (e.g., feed rendering vs order placement).

Layered / Hexagonal / Clean architecture

- **Hexagonal (ports and adapters):** domain core has no framework dependency; adapters implement ports for DB, HTTP, queues. Makes testing and swapping infrastructure easier.
- Dependency rule: dependencies point inward toward the domain.

Serverless

- Functions triggered by events; scale to zero; pay per invocation.
- **Limits:** cold starts, execution time limits, no long-lived connections (WebSockets need a managed gateway), vendor lock-in, harder local testing, connection pool exhaustion to databases (use proxies like RDS Proxy).
- Good for spiky, event-triggered, glue workloads.

Cell-based architecture

- Partition the whole stack into self-contained cells, each serving a slice of users, with its own data and dependencies.
- Limits blast radius (a bad deploy or hot tenant affects only one cell). A thin routing layer maps users to cells.

Data architectures

- **Data warehouse:** structured, curated, schema-on-write, SQL analytics (Snowflake, BigQuery, Redshift).
- **Data lake:** raw files on object storage, schema-on-read; with table formats (Iceberg) becomes a "lakehouse."
- **Lambda:** batch + speed layers (two code paths).
- **Kappa:** stream-only, replay for reprocessing.
- **Medallion (bronze/silver/gold):** raw → cleaned → business-level aggregates.

Service mesh and sidecars

- Sidecar proxy (Envoy) per pod handles mTLS, retries, timeouts, load balancing, metrics, tracing, without app code changes.
- Control plane (Istio, Linkerd) pushes config. Adds latency and resource overhead; adopt when fleet size justifies it.

BFF (Backend for Frontend)

- A thin backend per client type (web, mobile, TV) aggregating downstream APIs into the shape each client needs.

Gateway and aggregator

- API gateway for cross-cutting concerns; aggregation layer (GraphQL or BFF) for composing responses.

8.2 Reference Architectures for Common Systems

Use these as starting skeletons. In an interview, adapt to requirements and say what you'd change.

8.2.1 E-commerce

```
Client → CDN (static, product images) → API gateway (auth, rate limit)
  → Catalog service (search index: Elasticsearch; product DB; cache)
  → Cart service (Redis, TTL per user)
  → Order service (saga/orchestrator; outbox → Kafka)
      → Inventory (reservation with TTL; atomic decrement)
      → Payment (idempotent; PSP adapter; webhooks)
      → Shipping / Notification (async consumers)
  → Analytics: Kafka → lake (Iceberg) → warehouse
```

Key details: inventory reservation with expiry (avoid overselling via conditional decrement); checkout saga with compensations (release inventory on payment failure); search index updated via CDC; price consistency at checkout.

8.2.2 Social feed (Twitter/Instagram-like)

```
Post service → DB (sharded by author) + object store (media) → Kafka "post.created"
  → Fan-out workers → per-user timeline cache (Redis lists/sorted sets)
  → Feed service: merge precomputed timeline + pulled celebrity posts; ranking
  → Edge cache for hot feeds; CDN for media
Graph service: follow relations (sharded adjacency lists, cached counts)
```

Key details: hybrid fan-out; cursor pagination; ranking with a separate scoring service; counts are approximate (async counters).

8.2.3 Messaging (WhatsApp/Slack-like)

```
Clients ⇄ WebSocket gateways (stateful; connection registry in Redis: user → gateway id)
  → Message service: assign per-chat sequence IDs, persist (wide-column or sharded DB by chat_id)
  → Pub/sub routing (Kafka partitioned by chat or user; or Redis pub/sub channel per gateway)
  → Delivery to online recipients; push notifications (APNs/FCM) for offline
  → Media: upload to object store with pre-signed URLs; thumbnails async
Presence: heartbeats with TTL in Redis; coarse updates
```

Key details: ordering per chat; at-least-once delivery with client ACKs and dedupe by message ID; end-to-end encryption shifts key management to clients; multi-device sync via per-device cursor.

8.2.4 Video streaming (YouTube/Netflix-like)

```
Upload (resumable, pre-signed multipart) → raw storage → event
  → Transcoding workers (DAG of jobs: per resolution/codec; chunked parallel encoding)
  → Packaging (HLS/DASH segments + manifests) → origin object store
  → CDN with multi-tier caching (edge + regional + origin shield)
  → Playback: client adaptive bitrate (ABR) with buffer-based or throughput-based selection
Metadata DB, search index, recommendations, view counts (batched aggregation)
```

Key details: transcoding cost control (encode only needed renditions; popularity-based extra encodes); DRM; thumbnail generation; start-up latency.

8.2.5 Ride sharing (Uber-like)

```
Driver app → location updates (every 3–5 s) → Location ingestion (in-memory geo index per cell: H3/S2)
Rider app → ride request → Matching service (query nearby cells; ETA via routing service; offer with timeout)
Trip service: state machine in DB (requested → matched → en route → started → completed)
Pricing: surge computed per cell from supply/demand stream
Realtime: WebSocket/push to rider and driver; maps/ETA service with precomputed road graph
```

Key details: location is ephemeral (keep latest only; sample for history); offer locking so a driver gets one offer; ETA uses road-network routing (contraction hierarchies), not straight-line.

8.2.6 Payments

```
Merchant → Payment API (Idempotency-Key) → Payment orchestrator (state machine)
  → Ledger (double-entry, append-only, strongly consistent DB)
  → PSP adapter (card networks / bank rails) → webhooks → reconciliation
  → Risk/fraud service (sync scoring in path, timeouts with fallback rules)
  → Audit log, PCI scope limited to tokenization service
```

Key details: exactly-once effect via idempotency and ledger constraints; money as integer minor units; async webhooks with signature verification; daily reconciliation against settlement files; refunds and chargebacks as separate ledger events.

8.2.7 Search (Google/Elasticsearch-like)

```
Crawler/ingest → normalization → indexing pipeline (Kafka → indexers)
  → Inverted index shards (term → postings; doc values for sorting) with replicas
  → Query: parse → fan-out to shards → merge top-k → rank (BM25 + ML re-rank)
  → Cache for popular queries; autocomplete via separate prefix index
```

Key details: near-real-time indexing (refresh interval); segment merges; relevance tuning; typo tolerance (edit distance); sharding by document, querying all shards.

8.2.8 Distributed key-value store (Dynamo/Cassandra-like)

```
Client → coordinator node (any node) → consistent-hash ring with vnodes
  → N replicas (preference list), quorum W/R, vector clocks / LWW for conflicts
  → Storage engine: commit log + memtable + SSTables (LSM) + Bloom filters
  → Gossip membership, failure detection (phi accrual), hinted handoff, read repair, Merkle-tree anti-entropy
```

Key details: tunable consistency per request; no single point of failure; compaction strategy; tombstones and TTL.

8.2.9 Metrics/monitoring (Prometheus/Datadog-like)

```
Agents/SDKs → ingestion gateway (auth, rate limits, cardinality limits)
  → Kafka (partition by series hash) → TSDB writers (time-bucketed blocks, compression: delta-of-delta + XOR)
  → Query layer (downsampling, caching, fan-out) → dashboards
  → Rule evaluation / alerting service (dedupe, grouping, routing)
Long-term: rollups in columnar store (Druid / ClickHouse)
```

8.2.10 Log analytics and event pipeline

```
SDK / agents → collectors (batch, compress) → Kafka (topics by domain, schema registry)
  → Stream processing (enrich, dedupe, aggregate; watermarks)
  → Hot store (Druid/ClickHouse: sub-second queries on recent data)
  → Cold store (Iceberg/Parquet on S3: cheap, full history, compaction)
  → Query engine (Trino/Spark) for ad hoc; retention policies per tier
```

8.2.11 Job scheduler / workflow engine

```
API → workflow/job definition store
  → Durable execution engine (Temporal-style: workflow history event-sourced; activities retried)
  → Task queues per worker pool; leases and heartbeats; timers for delays
  → Visibility store (search by status), audit history
```

8.2.12 File sync (Dropbox-like)

```
Client: chunking (content-defined chunking, e.g., Rabin fingerprint), hashing, local index
  → Metadata service (sharded DB: file tree, versions, ACLs)
  → Block service (content-addressed chunks in object store, dedupe across users)
  → Notification service (long-poll / WebSocket: "changes available for namespace X")
  → Sync engine: delta computation from change log cursor
```

8.2.13 Web crawler

```
Seeds → URL frontier (priority + per-host politeness queues)
  → DNS cache → fetchers (robots.txt cache; rate per host)
  → Parser → content dedupe (SimHash) → URL normalization → seen-set (Bloom filter + store)
  → Storage (raw object store, index pipeline)
```

8.3 Pattern Selection Cheat Sheet

Need	Pattern
Atomic multi-service action	Saga (orchestration) + outbox + idempotency
Reliable event publishing	Transactional outbox / CDC
Fast, varied reads from writes	CQRS with read models
Full audit history	Event sourcing
Many consumers of a stream	Kafka consumer groups
Protect dependency from overload	Timeout + bulkhead + circuit breaker + load shedding
Massive hot reads	CDN + multi-tier cache + read replicas
Hot write key	Sharding by key + salting + batching
Isolate failures	Cells, bulkheads, per-tenant queues
Multi-region writes	Regional ownership (geo-partitioning) or CRDTs
Zero-downtime release	Canary + feature flags + expand/contract migrations
Cross-service auth	OIDC at edge + JWT propagation + mTLS internally + authz at service

---

8.4 How to Pick a Database (decision guide)

- Relational with joins and transactions, moderate scale → PostgreSQL / MySQL (+ replicas, partitioning, later sharding like Citus, Vitess).
- Global strong consistency, horizontal SQL → Spanner, CockroachDB, YugabyteDB.
- Wide-column, massive writes, known query patterns → Cassandra, ScyllaDB, HBase, Bigtable.
- Key-value, low latency, managed scale → DynamoDB (single-table design), Redis (in-memory, rich types).
- Document, flexible schema → MongoDB, Couchbase.
- Search and text → Elasticsearch / OpenSearch.
- Time-series → Prometheus TSDB, InfluxDB, TimescaleDB, Druid.
- OLAP analytics → ClickHouse, Druid, Pinot, BigQuery, Snowflake.
- Graph relationships → Neo4j, or a sharded adjacency store.
- Object/blob → S3, GCS, Azure Blob.

Justify with access patterns, consistency needs, scale, and team familiarity. Never pick on popularity alone.

8.5 Interview Technique for Architecture Questions

1. Start from the user flow and draw the simplest version that works.
2. Add components only when a requirement forces it, and say why (“we need async because the email provider is slow and we don’t want checkout to wait”).

3. Make every arrow a decision: sync or async, which protocol, what happens on failure.
4. Name the bottleneck before the audience does.
5. For each component: its data, its failure mode, its scaling plan.
6. End with what you'd monitor and what you'd build next.

09. Deep Question Bank with Model Answers

Use: cover the answer, say yours out loud, then compare. Each answer names the trade-off, which is what interviewers score.

A. Scalability and Capacity

1. **How do you scale a stateless API?** Horizontal replicas behind an LB, autoscaling on CPU/latency/queue depth, externalize session state (Redis or JWT), idempotent endpoints, graceful shutdown.
2. **How do you scale a stateful service?** Partition by key (consistent hashing or ranges), replicate each partition (leader + followers), route via a directory or ring, rebalance by moving partitions.
3. **Vertical vs horizontal?** Vertical is simple, has hardware ceilings and single-point risk. Horizontal needs partitioning and coordination. Start vertical for simple DBs, then replicas, then shard.
4. **How do you handle a 10x traffic spike?** Confirm it's real (not a bot), autoscale, cache aggressively, shed non-critical load, queue writes, pre-warm caches, request quota increases, use CDN for static.
5. **What breaks first at scale?** Usually the database (connections, hot keys, lock contention), then fan-out, then the network. Name the first one for your design and how you'd detect it.
6. **How do you estimate a database size for 5 years?** Records/day × size × 365 × 5 × replication factor × index overhead (~1.3–2x) + growth buffer (~30%).
7. **How do you handle hot partitions?** Detect via per-shard metrics; split the key (salting), add a cache layer, separate a hot tenant onto dedicated shard, rate-limit that key.

B. Databases and Storage

8. **Why would you choose Cassandra over Postgres?** Massive write throughput, multi-datacenter, known query patterns by partition key, tolerance for eventual consistency. Cost: no joins, limited ad hoc queries, modeling by query.
9. **When not to use NoSQL?** When you need multi-row transactions, complex joins, ad hoc analytics, or strong constraints; or when the team lacks experience and scale doesn't demand it.
10. **How does a B-tree index differ from an LSM for reads?** B-tree: predictable point and range reads, in-place updates. LSM: writes fast, reads check multiple files (mitigated by Bloom filters), background compaction.
11. **Why does a random UUID primary key hurt B-tree performance?** Random inserts scatter across pages, causing splits and cache misses. Use time-ordered IDs (UUIDv7, Snowflake, ULID).
12. **What does an index cost?** Storage, write amplification, slower inserts/updates, optimizer choices. Index for actual queries; remove unused indexes.
13. **How do you paginate a 100M-row table?** Keyset pagination on an indexed column; avoid OFFSET; return an opaque cursor.
14. **What is a covering index?** An index containing every column a query needs, so the table is never read.
15. **How do you migrate a 2 TB table without downtime?** Create the new table/column, dual write, backfill in throttled batches, verify with checksums, switch reads behind a flag, stop old writes, drop old later.
16. **How do you choose between row and column storage?** OLTP point access and updates → row. Analytics over few columns across many rows → columnar.

C. Consistency and Transactions

17. **Strong vs eventual consistency for a shopping cart?** Eventual is acceptable with merge rules (union of items); final checkout must validate inventory strongly.
18. **What is a race condition in a booking system and how to prevent it?** Two users see the last seat free and both book. Prevent with a conditional update or unique constraint on (show, seat), or a row lock within a transaction; return conflict to the loser.
19. **Explain write skew with an example.** Two on-call doctors each check "at least one other doctor is on call," both see yes, both go off call. Snapshot isolation allows it. Fix: serializable isolation or explicit locking on a shared row.
20. **How do you make an API call exactly once?** You can't, over an unreliable network. Use an idempotency key and dedupe server-side so the effect happens once.
21. **Two-phase commit vs saga?** 2PC: atomic but blocking and tightly coupled, used within a DB cluster. Saga: eventual, compensating, more resilient, needs business-level compensation design.
22. **How do you handle a failure midway in a 5-step saga?** Compensate completed steps in reverse order, each compensation idempotent; retry transient failures; park irrecoverable cases in a manual queue with alerts.
23. **What is read-your-writes and how do you provide it in a replicated system?** Read from leader for a window,

or track the write's log position and read from a replica that has reached it, or send the client a version token.

D. Distributed Systems

24. **How does Raft elect a leader?** Terms; followers timeout (randomized) and become candidates; they request votes; majority wins; the new leader sends heartbeats. Safety requires voters to only vote for candidates with up-to-date logs.
25. **How does Kafka choose a new leader for a partition?** The controller picks a new leader from the ISR. Unclean election (out-of-sync replica) is optional and risks loss.
26. **What is a fencing token and why is it needed?** A monotonic number issued with each lock; the protected resource rejects requests with older tokens. Needed because a paused client can wake up thinking it still holds the lock.
27. **How do you detect a dead node without false positives?** Use multiple signals (heartbeats plus indirect probes), adaptive detectors (phi accrual), confirm with quorum before removing a node from membership, and fence it.
28. **What is a split brain and how do you prevent it?** Two nodes both act as leader after a partition. Prevent with quorum (majority required to act), leases, fencing tokens, STONITH.
29. **What is a Merkle tree used for?** Efficiently find differing data between replicas: compare root hashes, descend into mismatched subtrees.
30. **Why does gossip scale?** Each node talks to a few random peers per round; information spreads in O(log N) rounds with no central coordinator and constant per-node load.

E. Messaging and Streams

31. **How do you choose a Kafka partition count?** Target throughput per partition (tens of MB/s), consumer parallelism needed, room for growth (partitions can increase but reshuffle keys), broker limits. Common: a multiple of expected consumer count.
32. **How do you handle a poison message?** Bounded retries with backoff, then DLQ with metadata; alert; replay after fix; never block the partition forever.
33. **Kafka consumer processes then crashes before commit. What happens?** Message is redelivered after rebalance; processing must be idempotent.
34. **How do you process events in order per entity across retries?** Key by entity; single consumer per partition; on retry, don't skip ahead; or use per-entity sequence numbers and buffer out-of-order.
35. **What is backpressure in a streaming system?** Consumers signal slowness through bounded buffers or pull-based consumption so producers slow down instead of overwhelming downstream.
36. **Event time vs processing time: why does it matter?** Mobile clients send delayed events; processing-time windows give wrong aggregates. Event-time windows with watermarks give correct results with bounded lateness.
37. **How do you de-duplicate a stream with at-least-once delivery?** Unique event IDs, dedupe state with TTL (window), or idempotent upserts in the sink.

F. Caching

38. **Why delete instead of update the cache on write?** Concurrent updates can write stale values back; deletion forces the next read to load fresh data. Still possible race; mitigate with TTL and versioning.
39. **Cache hit ratio dropped from 95% to 60%. Why?** Possible causes: key change (format or version bump), eviction from memory pressure, TTL too short, traffic pattern shift, cache node restart (cold cache), or cache key too granular.
40. **How does a CDN serve personalized content?** Cache the static shell at edge; fetch personalized parts via API; or cache per-user keys only where hit rate justifies it; use `Vary` carefully.
41. **Redis vs Memcached?** Redis: rich types, persistence, replication, Lua scripting, cluster. Memcached: simpler, multi-threaded, pure cache. Pick Redis for most modern needs.

G. APIs and Communication

42. **REST vs gRPC for internal service calls?** gRPC for performance, streaming, strong contracts, and polyglot codegen; REST for simplicity, browser access, and public APIs.
43. **How do you version an event schema?** Schema registry with compatibility modes (backward, forward, full); add optional fields with defaults; never change meaning; use new topic for breaking change.
44. **How do you design an idempotent POST for creating an order?** Client sends Idempotency-Key; server stores (key, request hash, response, status) atomically; repeat with same key returns stored response; different body with same key returns 422.
45. **Polling vs WebSocket vs SSE for a live score feed?** SSE for one-way server push with auto-reconnect and HTTP

compatibility; WebSocket if the client must send often; polling at low frequency if simplicity matters more than latency.

H. Reliability and Operations

46. **How do you set timeouts across a call chain?** Each hop's timeout is smaller than its caller's remaining budget; propagate deadlines; avoid timeouts larger than the user's patience.
47. **How do you avoid cascading failure?** Timeouts, bulkheads, circuit breakers, load shedding, bounded queues, retry budgets, and graceful degradation of non-critical features.
48. **What is an SLO and how do you pick one?** Start from user expectations and what the business needs; measure current performance; set target slightly below current reliability you can sustain; define error budget and policy.
49. **How do you do a safe deployment of a breaking change?** Expand-contract: deploy readers that support both, then writers, then remove old. Feature flags, canary, rollback plan.
50. **How do you debug high p99 latency?** Traces to find the slow span; check queue depth, GC pauses, lock contention, DB slow queries, connection pool wait time, retries and timeouts, noisy neighbors; compare per-version and per-region.
51. **What is a runbook?** Documented steps for a known alert: symptoms, checks, mitigation, escalation, rollback commands.

I. Security

52. **How do you store passwords?** Slow salted hash (argon2id/bcrypt/scrypt), unique salt per user, pepper optional in a KMS, rate-limit logins, support MFA.
53. **Access token vs refresh token?** Access token short-lived, sent with each request; refresh token long-lived, exchanged for new access tokens, rotated and revocable.
54. **How do you prevent IDOR?** Check object ownership or permission on every request server-side; use non-guessable IDs as defense-in-depth, not as the control.
55. **How do you secure service-to-service calls?** mTLS with workload identity, short-lived certificates, authorization policies per service, secrets in a vault, network policies.

J. Design Judgment (senior-level)

56. **When would you not use microservices?** Small team, early product, unclear domain boundaries, low scale, no independent deploy need. Start with a modular monolith.
57. **How do you decide build vs buy for a component?** Core differentiator and unique requirements → build; commodity (auth, payments, email, search at small scale) → buy; consider lock-in, cost at scale, team capacity.
58. **How would you migrate a monolith to services?** Identify bounded contexts, put a facade in front, extract one low-risk capability first, dual-run with comparison, move data ownership last, keep rollback path.
59. **How do you handle technical debt?** Track it, tie it to business cost, pay down in the same area as feature work, set guardrails.
60. **What would you cut to ship in 2 weeks?** Non-critical features, exotic consistency, multi-region, perfect analytics; keep correctness of money/auth and observability.

K. Your Resume Topics (be ready for deep follow-ups)

61. **Kafka at 1M+ events/day: how did you partition?** Explain key choice, number of partitions, consumer group sizing, and what hot-partition issues you saw and how you handled them.
62. **Why migrate from Kafka to Iceberg for batch-heavy ingestion?** Druid supervisors were loaded with large batch ingestion; Iceberg gives cheap, partitioned Parquet storage and direct partition queries; trade-off is freshness, so real-time queries stayed in Druid.
63. **How did Temporal help with 1M+ files/day?** Durable workflows with per-activity retries and timeouts, task queues to isolate workloads, activities designed idempotent; visibility for stuck workflows.
64. **How did you get 95% faster report generation?** Turned formulas into optimized Druid SQL, pre-aggregated via rollup and compaction, cut repeated computation. Quantify the baseline and measurement.
65. **Consistent hashing with 200 vnodes: why?** Even distribution (law of large numbers across vnodes), smaller data moves on node churn, tunable weight per machine. Cost: ring size and lookup cost (negligible with binary search).
66. **Zero cache invalidation on node add/remove: is that accurate?** Be precise: keys that move to the new node must be re-fetched or migrated; say what your implementation actually did. Credibility depends on precision.
67. **Distributed cache replication: sync or async?** State what you did and the failure window; explain how you would choose the other option under stricter requirements.

68. **Redis-based real-time presence on serverless without WebSockets?** Presence in sorted set scored by last-seen timestamp; TTL cleanup; clients poll or use SSE; trade-off is latency and polling load vs cost and complexity of managing WebSocket servers.
69. **Keycloak with RBAC on GraphQL: how did you enforce authz?** Token claims for roles at the gateway; field-level or resolver-level checks for object ownership; never rely only on the gateway.
70. **How would you scale your ETL platform 10x?** Partition work by vendor and time, scale Temporal workers and task queues separately, Kafka partitions to match, Iceberg compaction schedules, storage tiering, and cost controls.

L. Rapid Scenario Drills (30 seconds each)

71. Database primary crashes. First three actions? Confirm failure (not partition), trigger failover to most up-to-date replica with fencing, check for lost writes and reconcile, communicate status.
72. Latency doubled after deploy. Steps? Compare canary metrics by version, check traces, roll back if SLO burn, then diagnose.
73. Queue depth climbing. Steps? Check consumer health and throughput, downstream latency, partition skew, scale consumers, shed or pause producers if necessary.
74. Cache hit ratio falls to zero. Steps? Check cache cluster health, key format change, eviction, deployment with new key prefix; protect DB with rate limits; warm cache.
75. Duplicate charges reported. Steps? Check idempotency key handling, retry policy at all layers, webhook replays, reconciliation; fix root cause; refund and notify.
76. One region goes down. Steps? Confirm scope, shift traffic per failover runbook (DNS/global LB), check data replication lag, handle writes conflicts, communicate.
77. A hot user causes shard overload. Steps? Identify key via metrics, throttle that key, move tenant to dedicated shard, add caching, plan long-term fix.
78. Disk is full on a log broker. Steps? Free space by reducing retention or moving logs, avoid data loss by checking ISR and replication health, add capacity, prevent recurrence with alerts at 70/85%.
79. Cert expired, mTLS failing. Steps? Identify affected services, rotate certificates, verify trust chain, add expiry alerts and automated renewal.
80. Schema migration locks a table. Steps? Abort if safe, use online DDL or batched approach, have a rollback, test on staging with production-size data.

M. Behavioral Prompts for Architects

81. Describe a design you regret. What would you do differently?
82. Describe a time you pushed back on a design or a requirement.
83. Describe a time you led a migration.
84. How do you communicate a complex design to non-engineers?
85. How do you mentor engineers on system design?

Model answer structure for 81–85: situation (1 sentence), decision and reasoning, what happened (with metric), what you learned, what you do now.

11. Full HLD Walkthroughs, Part 1 (Cases 1–5)

Each walkthrough follows the interview clock: requirements → estimates → API → data model → architecture → flows → deep dives → failures → scale-up → probes. Say the reasoning out loud; the numbers are there so you can reuse the method, not memorize the answers.

Case 1: URL Shortener

1. Requirements (5 min)

- **Functional:** create a short code for a long URL; redirect a code to its URL; optional custom alias; optional expiry; click count.
- **Non-functional:** redirect latency p99 < 50 ms; 99.99% availability for redirects (reads matter more); codes must be unique; short codes are not guessable enough to enumerate private links (they are public by design, so state this).
- **Out of scope:** user accounts beyond API keys, link previews, malware scanning (mention as follow-up).

2. Estimates

- 100M new links/month → 100M / (30 × 86,400 ≈ 2.6M s) ≈ **40 writes/s**, peak ~120/s.
- Read:write 100:1 → **4,000 reads/s** average, peak ~12,000/s.
- Record ≈ 500 B (code 7 B, URL up to 2 KB, metadata). 100M × 12 × 500 B ≈ **600 GB/year**; with 5 years and replication ×3 ≈ 9 TB raw.
- Code space: base62, 7 characters → 62^7 ≈ **3.5×10^{12}** codes. At 1.2B/year, 7 chars last ~2,900 years. Length 7 is enough.
- Bandwidth: 12,000 redirects/s × ~300 B response ≈ 3.6 MB/s — trivial.

3. API

```
POST /v1/urls        {long_url, custom_alias?, expires_at?}  -> 201 {short_url, code}
GET  /{code}         -> 301 or 302 with Location header; 404 if unknown; 410 if expired
DELETE /v1/urls/{code} -> 204 (owner only)
GET  /v1/urls/{code}/stats -> {clicks, last_click}
```

301 vs 302: 301 lets browsers and CDNs cache the redirect permanently, so later clicks never reach you (no analytics, and you cannot change the target). 302 (or 307) keeps every click flowing through you. Pick 302 if analytics or edits matter; say so.

4. Data model

- **links** (KV store, partitioned by `code`): `code PK, long_url, owner_id, created_at, expires_at, is_custom`.
- **clicks**: append-only events (`code, ts, country, referrer`) → Kafka → aggregated counters.
- Why KV: access is always by code; no joins; scales by hash-partitioning on `code`.
- Why not hash the URL: MD5 of the URL then truncate gives collisions and the same URL always gets the same code (privacy leak). Use a counter-based ID instead.

5. Code generation

Option A (chosen): **range allocation.** A ticket service hands each app server a block of 10,000 IDs (`UPDATE counter SET next = next + 10000 RETURNING next`). Server uses IDs locally, base62-encodes them. No coordination per request; a crash wastes at most one block. Option B: Snowflake (64-bit timestamp + machine + sequence) then base62. Longer codes (~11 chars) unless truncated. Custom alias: insert with unique constraint on `code`; conflict → 409.

6. Architecture

```
Client ──► CDN/edge (optional, caches 302 only if you accept no analytics)
       ──► L7 LB ──► Redirect service (stateless, read-heavy)
                       ├─► Redis cache (code → long_url, TTL 24h, cache-aside)
                       └─► KV store (sharded by code, 3 replicas)
       ──► Write service ──► ID block allocator ──► KV store
                          └─► Kafka "clicks" ──► aggregator ──► analytics store
```

Split read and write services so the 100:1 read path scales independently.

7. Flows

- **Redirect:** LB → redirect svc → Redis GET → hit: return 302 (≈1 ms). Miss: KV read → SET in Redis → 302. Expired/unknown: 404/410. Click event sent async to Kafka (fire and forget with bounded buffer; losing a few clicks is acceptable, say so).
- **Create:** validate URL (scheme allowlist, length, resolvability optional) → get next ID from local block → base62 → insert (conditional on not exists for aliases) → return.

8. Deep dives

a) Cache sizing. Power-law popularity: top 20% of links drive ~80% of clicks. Working set ≈ 20% of links accessed daily. If 1 day of new links is 3.3M × 500 B ≈ 1.6 GB, a 16 GB Redis cluster holds the hot set comfortably. Expect hit ratio > 95%. **b) Hot link (viral).** One code gets 50,000 req/s. Redis single key can handle this on one shard (~100K ops/s), but add an in-process L1 cache with a 5 s TTL on redirect servers so the hot key never hits Redis in aggregate. **c) Expiry.** Do not run a sweeper that scans the whole table. Store `expires_at`; check at read time (lazy deletion); a background job deletes in batches using an index on `expires_at`. Cache TTL = min(24 h, time to expiry). **d) Analytics without slowing redirects.** Click increments are async. Aggregate in Kafka consumers into 1-minute buckets, write to a time-series or OLAP store. Redirects never wait on analytics.

9. Failure scenarios

Failure	Behavior
Redis down	Fall through to KV; latency rises, availability holds; protect KV with a bulkhead/limit
One KV shard down	Replicas serve reads (quorum read); writes to that range fail until failover
ID allocator down	Servers keep using their current block; when exhausted, writes fail, reads unaffected
Kafka down	Click events buffered locally then dropped after limit; redirects unaffected

10. Scale-up (10x traffic)

Add Redis shards and redirect replicas; split ID blocks larger; move click analytics to a sampled model if needed; regional deployment with each region serving reads from a local replica and writing to a home region.

11. Probes and answers

- *Two users create the same custom alias at once?* Unique constraint on `code`; one succeeds, one gets 409. No check-then-insert race.
- *How do you prevent abuse (spam links)?* Per-key rate limit on create, URL denylist, Safe Browsing check async, reporting endpoint.
- *What if the original URL changes content?* Out of our control; we store only the mapping, which is the point.

Case 2: Distributed Rate Limiter

1. Requirements

- Limit per API key and per IP; configurable per route and per plan (e.g., 100 req/min free, 10k/min paid).
- Return 429 with `Retry-After`; overhead < 1 ms p99; works across 200 gateway instances; accuracy within ~5%.
- Fail-open if the limiter store is down (say this explicitly and justify).

2. Estimates

- 500K API keys active, peak 200K req/s total.
- Each request = 1 counter operation → 200K ops/s on the limiter store. A Redis cluster of 3–6 primaries handles this (each ~50–100K ops/s).
- Memory: 500K keys × ~100 B state ≈ 50 MB. Trivial.

3. Where to enforce

- Gateway (edge of your system): protects all services, one place to change policy. Chosen.
- Per-service middleware: needed for internal fairness; add later.
- Client SDK: advisory only; never the enforcement point.

4. Algorithm choice (go deeper than “token bucket”)

- **Fixed window:** `INCR key:minute` with `EXPIRE`. Boundary burst: 100 requests at 0:59 and 100 at 1:00 = 200 in 2 seconds. Cheap but wrong at edges.
- **Sliding window log:** ZSET of timestamps; exact, but O(requests) memory per key. Too heavy at 100K req/min per key.
- **Sliding window counter:** `estimate = prev_count × (1 − elapsed_fraction) + curr_count`. Two counters per key. Error bounded and small (Cloudflare-style). **Chosen for paid plans with strict-ish limits.**
- **Token bucket:** tokens refill at `r` per second up to capacity `b`. Allows bursts up to `b`. **Chosen for smoothness and burst tolerance.** Store `(tokens, last_refill_ts)`.

5. Atomic implementation (Redis Lua)

```lua
-- KEYS[1]=bucket key; ARGV: capacity, refill_per_ms, now_ms, cost
local data = redis.call('HMGET', KEYS[1], 'tokens', 'ts')
local tokens = tonumber(data[1]) or tonumber(ARGV[1])
local ts = tonumber(data[2]) or tonumber(ARGV[3])
local now = tonumber(ARGV[3])
tokens = math.min(tonumber(ARGV[1]), tokens + (now - ts) * tonumber(ARGV[2]))
local allowed = 0
if tokens >= tonumber(ARGV[4]) then
  tokens = tokens - tonumber(ARGV[4]); allowed = 1
end
redis.call('HMSET', KEYS[1], 'tokens', tokens, 'ts', now)
redis.call('PEXPIRE', KEYS[1], 120000)
return {allowed, math.floor(tokens)}
```

Why Lua: read-modify-write must be atomic; a Lua script runs without interleaving on one Redis shard. Clock: use Redis TIME inside the script rather than app clocks, so skew between gateways cannot grant extra tokens.

6. Architecture

```
Request → Gateway (local L1: small in-memory bucket, sync every 100 ms)
        → Redis Cluster (hash tag {api_key} keeps a key on one shard)
        → allow → upstream service
        → deny  → 429 + X-RateLimit-* headers
Config service: limits per plan/route, pushed to gateways, cached locally
```

7. Deep dives

a) Hot key. One API key at 50K req/s on a single Redis shard is a hotspot. Options: local token pre-allocation (gateway leases 100 tokens from Redis, serves locally, renews) — trades strict accuracy for throughput. State the accuracy bound: at most (instances × lease size) overshoot. **b) Redis failure.** Circuit breaker around the limiter; on open, fall back to local per-instance limits (divided by instance count), not unlimited. Fail-open only for public read endpoints; fail-closed for expensive endpoints. **c) Clock.** Use server time inside Lua (see above). Never use gateway wall clocks directly. **d) Multi-region.** Per-region budget (global limit ÷ regions) with periodic rebalancing; accept small overage. Strict global limit requires cross-region coordination, which costs latency on every request.

8. Failure table

Failure	Result
Redis shard down	Keys on that shard fall back to local limits
Gateway crash	Its local leases expire; no double counting
Config push delayed	Gateways use last known limits

9. Probes

- *Why not a database counter?* ~10–50x lower throughput with row locks; contention on hot keys.
- *How do you return remaining quota?* Return tokens remaining from the Lua result in headers.
- *Who is limited behind NAT?* Per-IP limits punish shared IPs; prioritize API keys, use IP limits only for anonymous traffic.

Case 3: Distributed Cache

1. Requirements

- get/set/delete with TTL; 1 TB working data across cluster; 500K ops/s; p99 < 2 ms in-region; survives a node failure

with at most a brief unavailability of its keys.
- Consistency: clients accept eventual consistency across replicas; a delete must eventually remove the key everywhere.

2. Estimates

- 1 TB / (50 GB usable per node) → **20 primaries**, plus replicas → 40 nodes.
- 500K ops/s / 20 primaries ≈ 25K ops/s each — comfortable for an in-memory store.
- Network: 500K × avg 1 KB ≈ 500 MB/s cluster-wide → 10 GbE NICs per node are fine.

3. Partitioning

- Consistent hashing ring with **200 virtual nodes** per physical node → each node owns ~1/20 of keys with low variance (standard deviation of load shrinks as vnode count grows).
- Adding one node moves ~1/21 of keys (only keys whose ring position falls in new node's arcs). Removing one moves ~1/20 to neighbors.
- Hash function: MurmurHash3 or xxHash for speed and uniformity. Use ring lookup via sorted array + binary search: O(log(N×V)) = log(4,000) ≈ 12 comparisons.

4. Replication

- Each primary has 1–2 async replicas. Reads may go to replicas (eventually consistent) or primary (strong for that key).
- **Sync vs async:** async gives ~0.2–1 ms lower write latency and keeps the cache available; cost is losing the last few writes on failover. For a cache, losing a write means a miss, which is acceptable, so async is the right default. (Contrast with a primary database.)

5. Membership and failover

- Membership via gossip (or a coordinator like etcd storing the ring). A node missing heartbeats for T seconds is marked suspect, then dead after quorum agrees.
- Failover: promote the replica; clients refresh their ring from the coordinator. Stale client rings route to the old primary for a few seconds — acceptable for a cache; for writes that must not be lost, route through a proxy.

6. Eviction and memory

- Per-node `maxmemory`; LRU approximated by sampling 5–10 random keys and evicting the oldest (Redis-style). Cheap and close to true LRU.
- Track memory per key (overhead ~50–90 B per key in typical stores); small values waste memory — batch or use hashes.

7. Deep dives

a) Hot key. One key at 100K req/s overwhelms its primary. Fix: client-side local cache with 1–5 s TTL; or replicate hot key to k replicas and read round-robin. **b) Node join with data movement.** Moving 1/21 of data while serving traffic: double-read during migration (read new owner, fall back to old), write to new owner, background copy, then drop old copy. Mention cost and that some misses are inevitable. **c) Cache stampede on restart.** A restarted node is cold; all its keys miss to the DB. Mitigation: warm from replica snapshot, request coalescing, staggered rejoin, DB rate limit. **d) Persistence.** Usually none (it's a cache). If needed: periodic snapshot (fork-based) plus append log; recovery time grows with data size; mention trade-off.

8. Failure table

Failure	Impact
Primary down	Replica promoted; keys briefly missing → DB load spike
Network partition	Minority side unavailable for its keys; cache semantics tolerate this
Gossip storm after node flap	Dampening: require multiple missed heartbeats; back-off

9. Your project mapping

You can present: consistent hashing with 200 vnodes and minimal movement on churn; master–slave replication; a load balancer routing by ring. Prepare to explain precisely how keys were handled on node removal (re-fetch vs migrate) and the failure window of your async replication. Precision is worth more than the headline number.

10. Probes

- *Why not Redis Cluster?* You can, and it's the production answer; building yours shows understanding. Compare: Redis Cluster uses 16,384 fixed slots and gossip; you used a ring with vnodes.
- *Strong consistency needed for a feature flag cache?* Serve from cache with version checks; the source of truth stays in the DB.

Case 4: Notification System

1. Requirements

- Channels: push (APNs/FCM), SMS, email, in-app, webhook.
- Types: transactional (OTP, payment receipts: must arrive within seconds), marketing (bulk, can be delayed).
- Respect user preferences, quiet hours, and per-channel frequency caps.
- Deliver at least once with dedupe; track status (queued, sent, delivered, failed, opened).
- Scale: 10M/day average, bursts of 5M within 10 minutes (campaign).

2. Estimates

- 10M/day ÷ 86,400 ≈ **115/s average**. Campaign burst: 5M / 600 s ≈ **8,300/s**. Design for 10K/s with queues absorbing bursts.
- Storage: 10M × 1 KB log × 90 days retention ≈ 900 GB (compressed less).
- Provider limits matter more than your throughput: e.g., SMS providers throttle per account; you must pace outbound to them.

3. API

```
POST /v1/notifications  {user_id | recipients[], template_id, params, channels?, priority, idempotency_key}
  -> 202 {notification_id}
GET  /v1/notifications/{id} -> status per channel
POST /v1/providers/webhooks/{provider} -> delivery receipts
```

Always 202: the request is accepted, delivery is async.

4. Data model

- **notifications** (`id PK, user_id, template_id, params_hash, priority, created_at`), partition by user_id.
- **deliveries** (`notification_id, channel, status, attempts, provider_msg_id, updated_at`) — one row per channel.
- **preferences** (`user_id, channel, topic, enabled, quiet_hours`) — cached in Redis.
- **templates** — versioned; render at send time with version pinned.

5. Architecture

```
Producer → Notification API (validate, dedupe via idempotency_key in Redis SET NX with TTL)
  → Preference filter (cache) → Router (per channel)
  → Kafka topics: notif.transactional.{channel}  (high partitions, dedicated consumers)
                  notif.marketing.{channel}      (throttled consumers)
  → Channel workers (rate-limited per provider) → Provider (APNs, FCM, Twilio, SES)
  → Provider webhooks → status updater → deliveries table
  → DLQ after N attempts → alerting + replay tool
```

Isolation: separate topics and worker pools so a marketing blast never delays OTP.

6. Flows

1. API receives request; idempotency check; writes notification row in one transaction with outbox event.
2. Outbox relay publishes to Kafka.
3. Filter worker checks preferences, quiet hours (in user's timezone), caps (Redis counter per user per day).
4. Channel worker renders template, calls provider with its own idempotency key (if provider supports it), records `provider_msg_id`.
5. Provider receipt updates status; no receipt in T minutes → retry or mark unknown.

7. Deep dives

a) Dedupe. Producer retries cause duplicates: idempotency key in API. Provider retries cause duplicates: provider idempotency keys where supported; otherwise accept rare duplicates and keep them rare with timeouts that are long enough. **b) Rate limiting outbound.** Token bucket per provider account (e.g., 500 SMS/s). Workers pause and leave

messages in Kafka when out of tokens — backpressure instead of dropping. **c) Fan-out to millions.** Do not expand recipients in the API request. Store a campaign with a segment query; a fan-out job enumerates user IDs in batches of 10K into Kafka. The API stays fast. **d) Retries and provider outages.** Exponential backoff with jitter per channel; circuit breaker per provider; on sustained failure, route to fallback provider (e.g., second SMS vendor) if contracted. **e) Time zones and quiet hours.** Store user timezone; schedule via delayed queue (Kafka with timestamp-based delay or a scheduler table) rather than sleeping workers.

8. Failure table

Failure	Handling
Provider down	Circuit opens; queue accumulates; alert; fallback provider
Kafka lag on OTP topic	Dedicated consumers; lag alert pages on-call
Duplicate OTP sent	Acceptable for OTP if the code remains valid; prevent via dedupe window
Preferences service slow	Cache with TTL; default to *not* sending marketing when unknown

9. Probes

- *Exactly-once notification?* Not achievable end to end; effectively-once for the user: dedupe + provider keys + idempotent user-visible effect.
- *How do you measure delivery?* Provider receipts, opens/clicks via tracking pixels/redirects, with the caveat that opens are unreliable (privacy features).

Case 5: Chat Application (WhatsApp / Slack)

1. Requirements

- 1:1 and group chats (groups up to 1,000 members, large channels up to 100K).
- Message delivery: sent / delivered / read receipts; ordered within a conversation.
- Presence (online / last seen), typing indicators (ephemeral).
- Multi-device sync; history retention forever; offline delivery with push.
- Latency: online delivery p99 < 300 ms between two users in the same region.

2. Estimates

- 50M DAU × 40 messages/day = 2B messages/day ≈ **23K msgs/s average**, peak ~70K/s.
- Each message ~200 B + metadata ≈ 300 B → 600 GB/day raw → ~220 TB/year before replication and compression. Message history is the storage problem; keep recent hot data on SSD, older on cheaper tiers.
- Connections: 50M DAU, perhaps 20M concurrent sockets at peak. At ~100K connections per gateway box → 200 gateway servers.

3. API and protocol

- Clients open a **WebSocket** to a gateway (persistent, bidirectional).
- Frames: `SEND {client_msg_id, conv_id, body}`, `ACK {server_msg_id, seq}`, `DELIVER {conv_id, seq, body}`, `READ {conv_id, up_to_seq}`, `TYPING`, `PRESENCE`.
- REST for history pagination and media upload (pre-signed URLs).
- Long-lived connection: reconnect with exponential backoff; resume using last seen `seq` per conversation.

4. Data model

- **messages** partitioned by `conv_id`, clustered by `seq` (wide-column like Cassandra/HBase or sharded SQL). Row: `conv_id, seq, msg_id, sender_id, body, created_at`.
- **sequence numbers:** per conversation, monotonic. Assign with a per-conversation counter on the conversation's owner partition (single writer per conversation ⇒ no conflicts). Gaps detected client-side trigger a sync.
- **user_inbox / cursors:** `user_id, conv_id, last_delivered_seq, last_read_seq`.
- **connection registry:** Redis `user:{id}` → `{gateway_id, device_ids}` with TTL refreshed by heartbeat.

5. Architecture

```
Clients ⇄ WebSocket Gateways (stateful; ~100K conn each)
    │  registry: Redis user→gateway
    ▼
Chat Service (stateless) ── assigns seq via conversation owner (partition by conv_id)
    ├─► Message store (partitioned by conv_id)
    ├─► Kafka "messages" (partition by conv_id; ordering preserved)
    │       └─► Delivery router: look up recipient gateways → push via gateway RPC/pub-sub
    │       └─► Push service: APNs/FCM for offline devices
    └─► Presence service (Redis with TTL heartbeats)
Media service: upload to object store; thumbnails async
```

6. Send flow (step by step)

1. Client A sends `SEND` with `client_msg_id` (UUID generated on device) over its socket.
2. Gateway forwards to Chat Service (RPC).
3. Chat Service checks membership (cached), assigns `seq` from conversation counter, writes to message store. Idempotent on `(conv_id, client_msg_id)`.
4. Chat Service returns `ACK` to A with `seq` (A shows “sent”).
5. Event published to Kafka keyed by `conv_id`.
6. Delivery router looks up each recipient’s gateway in Redis; sends `DELIVER`. Recipient’s client sends `ACK`; server marks “delivered” and notifies sender.
7. Offline recipient: no registry entry → push notification; on next connect, client syncs from its cursor.

7. Deep dives

a) Ordering. Single writer per conversation (partition by `conv_id`) gives total order per conversation without global locks. Cross-conversation order is not guaranteed and not needed. **b) Group fan-out.** Group of 1,000: fan-out on write to 1,000 inboxes is fine. Channel of 100K members: don’t write 100K copies; readers pull from the conversation log by cursor, and the router pushes only “new message available” to online members. **c) Presence at scale.** Broadcasting every online/offline change to every contact is quadratic. Solutions: only subscribe clients to presence of people currently visible on screen; coarse “last seen” updated every few minutes; heartbeat TTL in Redis. **d) Gateway failure.** Socket drops; clients reconnect to another gateway; registry updated; client resumes with last `seq`. Messages are persisted before ACK, so nothing is lost even if the push was in flight. **e) Multi-device.** Each device has its own cursor and socket. Read state per device; the sender sees “read” when any recipient device reads (define the rule explicitly). **f) End-to-end encryption.** Server stores ciphertext and per-device keys; group encryption uses sender keys. Mention that search and server-side features become harder.

8. Failure table

Failure	Behavior
Gateway crash	Clients reconnect; resume by cursor; no loss
Chat service crash mid-send	Client retries with same `client_msg_id`; idempotent write
Kafka lag	Delivery delayed; persistence unaffected; users see “sent” not “delivered”
Redis registry lost	Fall back to “push to everyone” for a window, or query gateways; rebuild from heartbeats

9. Probes

- *How do you show messages in order when two arrive simultaneously from different users?* Order by server-assigned `seq` in the conversation; clients never reorder by device time.
- *Why not poll?* Polling every second for 50M users is 50M req/s of mostly empty responses. WebSocket costs memory per connection but eliminates that load.
- *Where is the single point of failure for a conversation?* Its owner partition; mitigate with replicas and fast failover; short unavailability for sends, reads still served from replicas.

12. Full HLD Walkthroughs, Part 2 (Cases 6–10)

Same structure as Part 1: requirements → estimates → API → data model → architecture → flows → deep dives → failures → scale-up → probes.

Case 6: News Feed / Timeline (Twitter, Instagram)

1. Requirements

- Post text/media; follow/unfollow; home timeline (ranked or chronological); profile timeline.
- Timeline loads in < 300 ms p99; new posts appear within ~5 s for followers.
- Users follow up to 5,000 accounts; some accounts have 100M+ followers.
- Eventual consistency acceptable for timeline; posts must never be lost.

2. Estimates

- 300M DAU; each opens the app 5× a day, loads 1 page (20 posts) each time → 1.5B timeline reads/day ≈ **17K reads/s avg**, peak ~50K/s.
- Posts: 100M posts/day ≈ **1,160 writes/s**, peak ~4K/s.
- Fan-out: an average post goes to ~200 followers → 100M × 200 = **20B timeline inserts/day ≈ 230K inserts/s**. This is the real load driver.
- Timeline cache: 300M users × 800 post IDs × 8 B ≈ 1.9 TB. Store IDs, not posts. Reasonable for a few hundred Redis nodes.

3. API

```
POST /v1/posts              {text, media_ids[]}           -> 201 {post_id}
GET  /v1/timeline/home?cursor=  -> {posts[], next_cursor}
GET  /v1/users/{id}/posts?cursor=
POST /v1/follows/{user_id}  / DELETE
```

Cursor = last post's (timestamp, id) — pagination stable under new inserts.

4. Data model

- **posts** sharded by `author_id`: `post_id (Snowflake, time-sortable), author_id, text, created_at, media`.
- **follows** two adjacency tables: `following(user_id, followee_id)` and `followers(followee_id, user_id)`, both sharded by their first key. Writes update both (saga or outbox).
- **timeline cache**: Redis sorted set per user `timeline:{user}` → score = post_id (time-sortable), trimmed to 800.

5. Core decision: fan-out on write vs on read

	Fan-out on write (push)	Fan-out on read (pull)
Write cost	O(followers) per post	O(1)
Read cost	O(1) read cache	O(following) merge at read
Breaks when	Celebrity with 100M followers	User follows 5,000 accounts
Latency	Fast reads	Slow reads

Hybrid (chosen): push for normal authors (< ~10K followers); for celebrities, skip fan-out. At read time, merge the precomputed timeline with posts fetched from the few celebrities the user follows (a small pull set). Threshold is a tunable.

6. Architecture

```
Post Service ─► posts DB (sharded by author) ─► Kafka "post.created" (key=author)
                                              └─► Fan-out workers
                                                    ├─ author follower count < T: push post_id to each follower's timeline ZSET
                                                    └─ celebrity: skip (pulled at read)
Timeline Service (read): ZREVRANGE timeline:{user} → merge celebrity pulls → hydrate post bodies (multi-get, cache)
Post cache: Redis/memcached of post objects (hot posts)
Follow Service: follows DBs + outbox → Kafka → updates timeline (backfill last N posts of new followee)
Ranking Service (optional): scores candidates; merges with ML features
```

7. Flows

- **Post:** write post row → outbox event → fan-out workers push IDs into follower ZSETs in batches (pipeline, 1,000 ops per round trip).
- **Read:** get ~200 candidate IDs from ZSET → merge celebrity posts by time → fetch post bodies via multi-get from post cache (fall back to DB) → apply ranking → return page with cursor.
- **Follow:** write follow rows → backfill: copy the new followee's last 50 post IDs into the follower's timeline ZSET (async).

8. Deep dives

a) Celebrity problem. A post by a 100M-follower account would cause 100M ZSET writes. The hybrid skips it. Cost moves to read time for followers of celebrities; cap the pull set at ~20 celebrities per user. **b) Fan-out at 230K writes/s.** Partition workers by follower shard, batch writes per Redis node, use pipelines. Lag target: p99 < 5 s. Monitor fan-out lag as an SLO. **c) Inactive users.** Don't maintain timelines for users inactive > 30 days; build on demand when they return (pull-based rebuild). This cuts memory and write cost a lot. **d) Cache eviction and rebuild.** If a timeline is evicted, rebuild from follows + recent posts (bounded work). Protect against stampede on a popular cold start with single-flight. **e) Counts (likes, views).** Not synchronous. Increment via batched counters (aggregate per post per second, flush), approximate for display, exact in an offline job. **f) Deletion and privacy.** Deleting a post must remove it from timelines: store deleted flag on post; timelines filter at hydration (lazy), and a background job prunes ZSET entries.

9. Failure table

Failure	Behavior
Fan-out worker down	Kafka replays from offset; idempotent ZADD makes replays safe
Timeline cache down	Rebuild on read (slower, DB load); rate limit rebuilds
Post DB shard down	Posts for that author slow/unavailable; timelines still serve cached IDs

10. Scale-up

More fan-out partitions; regional timelines; split timeline cache by user shard; precomputed ranking features; move celebrity list to config so thresholds adapt.

11. Probes

- *Why store IDs in the timeline not the post?* Post bodies are shared by many timelines; storing IDs avoids 200× duplication and makes edits/deletes single-write.
- *Ordering across fan-out workers?* Timelines sort by time-sortable post_id, so arrival order does not matter.
- *Why not one global feed table?* It would be a hot table; per-user materialization spreads load across shards.

Case 7: Typeahead / Autocomplete

1. Requirements

- Given a prefix, return top 5–10 suggestions ranked by popularity (and optionally personalization).
- Latency p99 < 100 ms, including network; suggestions update within hours (daily freshness acceptable).
- Filter unsafe terms; handle non-English tokens; support up to 15 characters.

2. Estimates

- 1B searches/day ≈ 12K/s average, peak ~40K/s. Each search fires ~5 keystroke requests after client debounce → **~60K prefix requests/s peak**. Client debounce (150–200 ms) cuts this sharply; state that.
- Phrase vocabulary: 500M unique queries; average 25 chars → 12.5 GB raw; trie with compression ~ 10–30 GB. Fits in memory across a few dozen nodes.

3. API

`GET /v1/suggest?q=<prefix>&lang=en&limit=8` → `{suggestions: [{text, score}]}`. Cache-control with short max-age at the CDN for popular prefixes.

4. Data pipeline (offline)

```
Query logs → Kafka → Spark/Flink job (hourly/daily):
  normalize (case, whitespace, unicode), count per phrase, filter (blocklist, PII, low counts)
  → build top-K per prefix (K=10) for every prefix up to length 15
  → write to serving store (versioned snapshot)
```

Top-K per prefix stored at each prefix precomputed: lookup is O(prefix length) and no trie traversal at query time.

5. Serving architecture

```
Client (debounce, local cache of recent prefixes)
  → CDN edge (cache popular prefixes, 5–60 min)
  → Suggest service (stateless) → in-memory trie/hash map shards (partition by first 1–2 characters)
  → Snapshot swap: build new version, load in background, atomic pointer switch
```

Sharding by prefix (first 2 chars) spreads hot prefixes like “a” unevenly; mitigate by splitting heavy shards further or using a hash of the prefix.

6. Deep dives

a) Data structure choice. Trie: O(L) lookup, memory heavy. Radix tree: compressed, far less memory. Precomputed top-K per node: avoids DFS at query time. Chosen: radix tree with top-K stored per node. **b) Freshness vs cost.** Full rebuild daily; trending terms via a streaming counter layer merged at serve time for last-hour spikes (e.g., breaking news). Merge: score = blend of historical and recent counts. **c) Personalization.** Re-rank top-50 candidates with user history (small per-user profile in cache). Keep the base list shared. **d) Safety.** Blocklist applied in the offline pipeline (and a runtime filter for emergent terms). Never rely on runtime only. **e) Storage swap.** Versioned snapshots; keep last two versions for instant rollback if a bad build ships.

7. Failure table

Failure	Behavior
Suggest node down	Replica in same shard; CDN serves popular prefixes
Bad batch build	Rollback to previous snapshot pointer
Pipeline late	Serve yesterday’s snapshot; freshness SLO alarms

8. Probes

- *Why not SQL LIKE ‘prefix%’?* Index scan plus sort per keystroke; cannot rank cheaply at this scale.
- *Typos?* Fuzzy matching via edit distance (BK-tree or n-gram index) as fallback when exact prefix has no results.
- *Why debounce?* Reduces load ~5x and avoids flicker.

Case 8: Ride Sharing (Uber / Lyft)

1. Requirements

- Rider requests a ride from pickup to dropoff; system matches nearest suitable driver; shows ETA and live driver location.
- Driver app sends location every 4 s; trip lifecycle with states; fare estimate and surge pricing.
- Match within 10 s p95; no driver assigned to two riders; location freshness < 10 s.
- Scale: 5M drivers online at peak (city-level hotspots), 20M rides/day.

2. Estimates

- Location updates: 5M drivers / 4 s = **1.25M writes/s** globally. Must be in-memory, not a database.
- Ride requests: 20M/day ≈ 230/s avg, peak ~2,000/s (concert ends, rain).
- Storage: trip records 20M × 2 KB ≈ 40 GB/day; history retention in object store/columnar.
- Per city hotspot: 100K drivers in one metro → 25K location writes/s in one region. Partition by geography.

3. API

```
POST /v1/rides            {pickup, dropoff, product_type}      -> {ride_id, fare_estimate, eta}
GET  /v1/rides/{id}       -> state, driver, live location (WebSocket/SSE subscription)
POST /v1/driver/location  {lat, lng, heading, speed, ts}       (UDP-ish fire-and-forget or batched HTTP)
POST /v1/driver/rides/{id}/accept | /arrive | /start | /complete
```

4. Geospatial index

- Partition the map into cells using **H3** (hexagons at resolution ~8 ≈ 0.7 km² each) or S2 cells.
- Driver location store: `cell_id → set(driver_id)`; `driver_id → (lat, lng, cell, ts)`. Both in memory, sharded by cell (region).

- Moving a driver across cells: remove from old cell set, add to new (atomic on the shard).
- Nearby query: fetch driver sets for the rider's cell and its k-ring neighbors (ring 1 = 7 cells, ring 2 = 19 cells), filter by distance and availability.
- Why not geohash only: boundary effects and uneven cell sizes by latitude; H3 gives uniform neighbor distances.

5. Architecture

```
Driver app ──location──► Location ingest (stateless; batches) ──► in-memory geo index (sharded by cell)
                                                                └──► Kafka (sampled, for analytics/ETA training)
Rider app ──request──► Trip service (state machine, DB) ──► Matching service
Matching: query nearby drivers → ETA service (routing engine on road graph) → rank → offer to best driver
Offer state: driver has 15 s to accept; on timeout, next candidate
Realtime: rider subscribes to trip channel; push driver location every 3–5 s (WebSocket gateway)
Pricing: surge computed per cell from demand (requests) and supply (available drivers) on stream aggregation
```

6. Flows

1. Rider requests → trip created in `REQUESTED`; fare estimated.
2. Matching finds candidates in rider's cell + ring 1; ETAs from routing engine (not straight-line).
3. Offer sent to top driver; `OFFERED` with expiry; driver accepts → conditional update `status='MATCHED' WHERE status='OFFERED' AND driver_id=?` ensures single owner.
4. Driver moves through `ARRIVED → STARTED → COMPLETED`; each transition persisted with timestamps.
5. Payment and receipt after completion (saga).

7. Deep dives

a) Offer correctness (no double assignment). One driver can receive offers for two riders simultaneously. Driver-side lock: a driver in `OFFERED` state is excluded from other matching for the offer window. The final acceptance is a conditional write on the trip row. Losers get a retry. **b) Location write load.** 1.25M/s: keep latest position only in memory; persist a sample every 30 s to the analytics stream. Use batching (driver app sends 4 points per request every 16 s) and UDP-like best effort — losing one point is harmless. **c) Staleness.** Each position has a timestamp; ignore positions older than 15 s for matching, and mark drivers offline if no update in 30 s. **d) Hotspots (concert ends).** Demand spikes in one cell. Mitigations: surge pricing reduces demand; widen search radius for supply; pre-position drivers via incentives; shard by finer cell at hotspots (adaptive resolution). **e) ETA accuracy.** Road-network routing with live traffic weights; cache routes between popular cell pairs; ML correction on historical trips. **f) Failover.** Geo index shards are reconstructable from driver heartbeats within ~5 s, so memory-only is acceptable; trip state is in a durable DB.

8. Failure table

Failure	Behavior
Geo shard down	Drivers in its cells invisible for seconds until heartbeats repopulate
Matching timeout	Next candidate; trip stays `REQUESTED` until limit, then cancel with retry prompt
Driver app offline mid-trip	Trip continues; sync when reconnected; alerts after threshold

9. Probes

- *Why not a PostGIS query on every request?* Millions of updates/s with row locks; in-memory cell index is far cheaper.
- *How do you prevent a rider being matched twice?* Trip-level conditional state transition.
- *How do you compute ETA?* Routing service on road graph; precomputed contraction hierarchies for fast queries.

Case 9: Payment System

1. Requirements

- Accept card payments through an external PSP; authorize, capture, refund, partial refund.
- No double charge under any retry; every money movement recorded in a double-entry ledger.
- Reconcile with PSP daily; PCI scope kept minimal (card data never touches our servers — tokenized by PSP).
- Availability 99.99% for checkout; correctness above availability for money.

2. Estimates

- 1M payments/day ≈ 12/s avg, peak ~150/s (flash sale). Low throughput; correctness dominates.

- Ledger rows: each payment ~6 entries → 6M rows/day ≈ 2.2B/year. Fine for a sharded relational DB.
- PSP latency 1–3 s per call; our timeouts must allow for it, with async completion via webhooks.

3. API

```
POST /v1/payments     Idempotency-Key: <uuid>   {amount_minor, currency, customer_token, order_id}
  -> 202 {payment_id, status: PROCESSING}   (or 201 final if sync)
GET  /v1/payments/{id}
POST /v1/payments/{id}/refunds   Idempotency-Key   {amount_minor}
POST /webhooks/psp              (signed; event_id dedupe)
```

Amounts in integer minor units (paise/cents). Never floats.

4. Data model

- **payments**: `id, order_id, amount, currency, status, version, psp_ref, created_at`. Status transitions guarded by `version`.
- **idempotency_keys**: `(merchant_id, key) PK, request_hash, response, status, created_at, ttl`.
- **ledger_entries**: append-only `(entry_id, txn_id, account, debit, credit, currency, ts)`; invariant: per transaction, debits = credits.
- **accounts**: merchant receivable, platform revenue, PSP clearing, customer funds.
- **psp_events**: raw webhook payloads with event_id unique.

5. State machine

```
CREATED → PROCESSING → AUTHORIZED → CAPTURED → (REFUNDED partially or fully)
                    ↳ FAILED
AUTHORIZED → VOIDED (before capture)
```

Only the orchestrator moves states; each move is a conditional update on `version`, and emits an event via outbox.

6. Flows

1. Check idempotency key: insert `(key, PROCESSING)` in unique table. Existing key with same hash → return stored response; different hash → 422.
2. Create payment `CREATED`; outbox event.
3. Call PSP with PSP idempotency key = our payment ID (PSPs support this). Timeout → status stays `PROCESSING`; do not retry blindly without the same key.
4. PSP response (sync or webhook) → transition to `AUTHORIZED`/`FAILED`; write ledger entries in the same transaction as the state change.
5. Capture → ledger: move from PSP clearing to merchant receivable.
6. Daily reconciliation: compare our captured set with PSP settlement file; mismatches → queue for investigation.

7. Deep dives

a) Exactly-once effect. Retries at three layers (client, our API, our call to PSP) all carry the same key; uniqueness constraints enforce single effect. Webhook duplicates are dropped by event_id. **b) Unknown outcome.** PSP timeout = unknown. Never mark failed. Resolve by querying PSP status with our payment ID (reconciliation/poller) before any retry or customer message. **c) Ledger correctness.** Each transaction is one DB transaction with balanced entries; a periodic job asserts sum of debits = sum of credits globally and per currency. **d) Concurrency on refunds.** Two concurrent refunds that together exceed the captured amount: lock the payment row (`SELECT … FOR UPDATE`) or conditional update on `refunded_total + amount <= captured`. **e) Multi-currency.** Store original currency and exchange rate snapshot; never convert at display time for accounting. **f) PCI scope.** Hosted fields or PSP SDK tokenize cards in the browser; our services only see tokens. Shrinks compliance surface dramatically. **g) Availability.** Synchronous auth in checkout; if PSP is down, fail fast with clear message; failover to a secondary PSP only with the same idempotency key guarantees and reconciliation coverage.

8. Failure table

Failure	Behavior
Client retries after timeout	Idempotency key returns the same payment result
Webhook arrives before sync response	State machine accepts out-of-order if version allows; otherwise buffer
DB failover during ledger write	Transaction atomic; either all entries or none; PSP reconciliation catches mismatches

9. Probes

- *Why not Kafka for payments?* You can use it for events, but the source of truth is a transactional DB with constraints; Kafka alone cannot enforce “one captured payment per order.”
- *Why not eventual consistency for balances?* Balances gate money movement; strong consistency on those rows is required.
- *Settlement mismatch of ₹1 found?* Investigate via PSP refs; never adjust ledger without an auditable correction entry.

Case 10: Web Crawler

1. Requirements

- Crawl the web (say 5B pages) with a refresh policy: important/news pages daily, others monthly.
- Respect robots.txt and per-host politeness (e.g., max 1 request/s per host by default).
- Deduplicate content; avoid crawler traps; store raw HTML and extracted links.
- Throughput: 500M pages/day ≈ **5,800 pages/s** average (state assumption; the number is the scale of the design).

2. Estimates

- 5B pages × 100 KB average compressed raw ≈ **500 TB** per full snapshot; keep last 2–3 snapshots and deltas.
- Fetchers: each takes ~1 s per page including network/parsing → 5,800 concurrent fetches in flight → ~600 fetcher workers with 10 connections each.
- URL frontier: ~50B URLs known; need disk-backed storage, not memory alone.

3. Architecture

```
Seeds → URL Frontier
          ├ Priority queues (importance score, freshness due time)
          └ Per-host queues (politeness: next allowed fetch time per host)
       → Fetcher pool (DNS cache, robots.txt cache, HTTP keep-alive, timeouts, max size)
       → Parser (extract text, links, canonical URL, meta)
       → Dedupe: content fingerprint (SimHash) lookup
       → Link extractor → normalize URL → seen-filter (Bloom filter + persistent store) → back to frontier
       → Storage: raw object store (WARC files), index pipeline (Kafka → indexer)
```

4. Frontier design

- **Politeness:** each host has a queue and a `next_allowed_at`. Scheduler pops hosts whose time has arrived; a host with 10M URLs cannot starve others.
- **Priority:** importance (PageRank-like score) + freshness deadline. Combine into a scheduling key.
- **Storage:** frontier backed by sharded disk stores (e.g., RocksDB per shard) keyed by host; in-memory only for active hosts.

5. Deep dives

a) Duplicate URLs. Normalize: lowercase host, remove default ports, sort query params, remove tracking params (utm_*), strip fragments, resolve relative paths. Seen-check: Bloom filter for fast negative answers (no false negatives) with persistent backing for exactness. Sizing: 50B URLs at 1% false positive ≈ 10 bits/URL ≈ 62 GB — shard it. **b) Duplicate content.** Exact hash catches byte-identical; SimHash (64-bit) catches near duplicates: Hamming distance ≤ 3 means near-duplicate. Index SimHash blocks for lookup. **c) Traps.** Infinite calendars, session IDs in URLs, generated pages. Limits: max depth per host, max URLs per host per day, max URL length, pattern detection (same template repeated thousands of times). **d) robots.txt and legal.** Cache robots per host (TTL ~24 h); respect Crawl-delay; honor meta robots and noindex; keep opt-out handling. **e) DNS.** Local DNS cache with TTL; the resolver is often the bottleneck at scale — run your own caching resolver. **f) Fault tolerance.** Checkpoint frontier state; fetch jobs are idempotent (re-fetch is harmless); a crashed fetcher’s leases expire and jobs return to the queue. **g) Freshness.** Estimate change rate per page from history; schedule re-crawl by expected change interval (adaptive), not fixed period.

6. Failure table

Failure	Behavior
Host returns 5xx storms	Exponential backoff per host; circuit per host
Fetcher crash	Lease expires; another fetcher takes the URL
Frontier shard lost	Rebuild from replicas or re-seed from link graph

7. Probes

- *Why Bloom filter instead of a hash set?* Memory: a hash set of 50B URLs is terabytes; a Bloom filter is an order of magnitude smaller, with an acceptable false-positive rate (a few URLs skipped).
- *How do you avoid hammering one site?* Per-host queues with time-gated scheduling.
- *Is the crawler legal everywhere?* Respect robots, terms, and local law; the design includes opt-outs and rate limits.

13. Full HLD Walkthroughs, Part 3 (Cases 11–15)

Case 11: Distributed File Storage (Dropbox / Google Drive)

1. Requirements

- Upload, download, and sync files across devices; folders; versions (keep 30 days); sharing with permissions; offline edits sync when online.
- Files up to 50 GB; sync latency < 10 s for small changes; no lost updates on concurrent edits (conflict copies, never silent overwrite).
- Scale: 100M users, 500M devices, average 2 GB stored per user → **200 PB** total logical data before dedupe. (Say this explicitly, then scope: design the mechanisms that scale to that.)

2. Estimates

- Changes: 500M devices × 10 file changes/day ≈ 5B events/day ≈ **58K change events/s** average.
- Bandwidth: with chunk-level sync, a typical 1 MB edit to a 1 GB file transfers ~4 MB of chunks, not 1 GB.
- Chunks: 4 MB average chunk → 200 PB / 4 MB ≈ **50 billion chunks**. Chunk index must be sharded by hash.
- Metadata: 100M users × 10K files × 500 B ≈ 500 TB. Sharded relational store.

3. Core idea: chunking + content addressing

- Split file into chunks using **content-defined chunking** (rolling hash, e.g., Rabin fingerprint, boundary when hash matches a pattern). An insertion at the start of a file only changes nearby chunks, not all of them (fixed-size chunks would shift everything).
- Each chunk stored under `sha256(content)`. Identical chunks across users are stored once (dedupe, per-user encryption complicates cross-user dedupe — mention the trade-off).
- A file = ordered list of chunk hashes ("manifest"). Version = new manifest.

4. API

```
POST /v1/files/upload-session  {path, size, chunk_hashes[]}  -> {missing_chunks[], upload_urls[]}
PUT  <presigned chunk url>       (chunk body)
POST /v1/files/commit          {path, manifest_id, base_version}  -> 200 or 409 (conflict)
GET  /v1/changes?cursor=       -> change feed for sync
GET  /v1/files/{id}/versions
```

The client sends hashes first; the server replies which chunks are missing. Only missing chunks upload.

5. Architecture

```
Client: chunker + local index (path → manifest, chunk hashes), change watcher
   │ metadata calls                       │ chunk bytes
   ▼                                      ▼
Metadata service (sharded by user_id)   Block service → object store (chunks keyed by hash)
   ├─ file tree, versions, ACLs           (pre-signed URLs; CDN for downloads)
   ├─ change log (per user, ordered)
Notification service: long-poll / WebSocket "namespace X changed, cursor=N"
Sync: client fetches changes after cursor, downloads missing chunks, reassembles
```

6. Data model

- **files**: `(user_id, file_id)` PK, `path, current_version, deleted, updated_at`.
- **versions**: `(file_id, version)` PK, `manifest_id, size, created_at, device_id`.
- **manifests**: `manifest_id → [chunk_hash...]` (stored as blob or rows).
- **chunks**: `chunk_hash PK, size, storage_location, refcount`.
- **change_log**: `(user_id, seq)` PK — monotonically increasing per user; sync reads by seq.

7. Flows

- **Upload:** chunk locally → send hashes → get missing list → upload missing chunks → commit manifest with `base_version`. If `base_version` ≠ current, return 409 → client creates a conflict copy `file (conflicted copy from device X)`.
- **Download/sync:** client reads change feed after cursor; for each changed file, fetches manifest, requests chunks it lacks, reassembles to a temp file, atomic rename.

8. Deep dives

a) Conflict resolution. Optimistic concurrency via `base_version`. Never last-write-wins silently for files; create conflict copies. Text merge possible for specific formats (not general files). **b) Garbage collection.** Chunk refcount decremented when versions expire; GC is a background mark-and-sweep: mark chunks reachable from live manifests, delete unreferenced chunks older than a grace period (protects in-flight uploads). **c) Consistency between metadata and blobs.** Upload chunks first, then commit metadata. A commit referencing missing chunks is rejected (server checks existence). Orphan chunks (uploaded but never committed) are reclaimed by GC after grace period. **d) Large files and resumability.** Chunk-level resumes: upload session records which chunks are done; retries re-send only missing chunks. Chunk uploads are idempotent (same hash = same content). **e) Sync latency at scale.** Change feed per user is tiny; notification service fans out "changed" pings only to devices of that user (per-user channels), not broadcasts. **f) Sharing.** ACL table `(file_id, principal, role)`; permission checks at metadata service; links with tokens and expiry; download via CDN with signed URLs. **g) Encryption.** Per-user or per-file keys (envelope encryption); cross-user dedupe trades privacy (convergent encryption leaks equality). State the choice.

9. Failure table

Failure	Behavior
Object store region down	Replicated buckets; read failover; uploads queued on client
Client crash mid-sync	Temp file discarded; cursor not advanced until apply completes
Metadata shard down	That user's sync pauses; no corruption since commits are atomic

10. Probes

- *Why content-defined chunking?* Fixed-size chunks re-upload everything after an insertion; CDC limits changes to nearby chunks.
- *How do you handle a 50 GB upload on flaky wifi?* Resumable chunked upload sessions with per-chunk acks.
- *Dedupe across users is a privacy risk; how do you mitigate?* Per-user keys make cross-user dedupe impossible; or dedupe only within a user, or use convergent encryption knowingly.

Case 12: Metrics / Monitoring System (Prometheus / Datadog)

1. Requirements

- Ingest metrics from 100K hosts × 500 series × every 10 s ≈ **5M points/s**.
- Dashboard queries < 1 s for last 24 h; alerting evaluation every 30 s; retention: raw 7 days, 1-minute rollups 90 days, 1-hour rollups 2 years.
- Multi-tenant; cardinality limits per tenant.

2. Estimates

- 5M points/s × 16 B (compressed to ~1.5 B/point with Gorilla-style compression) ≈ 7.5 MB/s compressed ingest → ~650 GB/day raw compressed? Compute carefully: 5M × 86,400 = 432B points/day; × 1.5 B ≈ **650 GB/day** compressed. With 7-day raw retention ≈ 4.5 TB, replication ×3 ≈ 14 TB.
- Series count: 100K hosts × 500 = **50M active series**. Each series index entry ~200 B → 10 GB of index: keep index in memory across shards.

3. Data model

- A series = metric name + label set (`host`, `service`, `region`). Identity = hash of sorted labels.
- Storage: time-bucketed blocks (2-hour blocks in Prometheus-style TSDB), each with series index + compressed chunks.
- Inverted index: label → series IDs (postings lists), intersected for queries like `service="api" AND region="eu"`.

4. Architecture

```
Agents/SDKs → ingestion gateway (auth, batch, rate and cardinality limits)
  → Kafka (partition by series hash, for ordering per series and parallelism)
  → Writers (write-ahead log → in-memory head block → flush to block store every 2 h)
  → Storage: local SSD (recent) + object store (blocks, long-term)
  → Query frontend (split queries by time, cache results, fan out to shards)
  → Rules engine: recording rules (pre-aggregate) + alert rules every 30 s
  → Alert manager: dedupe, group, silence, route (paging, chat)
Downsampler: raw → 1 min → 1 h rollups as background jobs
```

5. Compression

- Timestamps: delta-of-delta encoding (regular 10 s intervals compress to ~1 bit per point).
- Values: XOR encoding of floats (consecutive values often similar → few bits).
- Result ~1–2 bytes per point (Gorilla paper's approach).

6. Deep dives

a) Cardinality explosion. A label with unbounded values (user_id, request path with IDs) creates millions of series; memory and index explode. Defenses: label allowlist, cardinality limits per tenant (reject new series beyond quota), normalize paths to route templates, monitor series churn. **b) Late and out-of-order data.** Agents buffer during outages and send old points. Accept within a window (e.g., 1 h); head block supports out-of-order inserts within its range; older data needs a backfill path. **c) Query performance.** Split long ranges into sub-queries by time; cache results of immutable past buckets; pre-aggregate with recording rules for dashboards that run constantly. **d) Downsampling correctness.** Store aggregates per bucket (min, max, sum, count) not just average, so averages of averages don't lie; percentiles need histograms or sketches (t-digest), not averaged percentiles. **e) Alert evaluation at scale.** 50M series with thousands of rules: evaluate rules by query groups; sharded evaluators; a missing data alert is separate from threshold alerts (no data ≠ zero). **f) Ingestion durability.** Kafka between agents and writers buffers spikes; writers ack after WAL write; replicas across zones. **g) Cost.** Tiered storage: recent on SSD, older on object store; retention per tenant; sampling for high-volume debug metrics.

7. Failure table

Failure	Behavior
Writer down	Kafka retains; partition reassigned; replay from offset
Query shard down	Partial results flagged; retry other replica
Alerting pipeline lag	Alert delay monitored by meta-alert (dead man's switch)

8. Your project mapping

Your Druid work is the same family: Kafka ingestion, segments, rollup, compaction, and query latency. Explain rollup as the time-series downsampling story and compaction as the small-files story from your Iceberg work.

9. Probes

- *Why not store metrics in PostgreSQL?* Write rate and compression; time-series engines have append-optimized blocks and time partitioning.
- *Average latency dashboard is misleading — why?* Averages hide tails; show percentiles from histograms.
- *High-cardinality request IDs in labels?* Put them in logs or traces, not metrics.

Case 13: Video Streaming (YouTube / Netflix)

1. Requirements

- Upload videos (up to 12 hours, 20 GB); transcode to multiple resolutions and codecs; playback with adaptive bitrate; start playback < 2 s.
- Search and recommendations (overview only); view counts; comments (out of scope detail).
- Scale: 500 hours uploaded per minute ≈ **30,000 hours/day**; 1B views/day.

2. Estimates

- Raw upload: 30,000 h/day × ~1 GB/hour source ≈ **30 TB/day raw**.
- Transcoded output: ~6 renditions (240p–4K) with total ≈ 3× raw bitrate → 90 TB/day output; storage per year ≈ 30 PB (before tiering). Justify tiering: cold renditions for long-tail videos.
- Playback: 1B views × avg 15 min × ~3 Mbps ≈ 1B × 0.34 GB ≈ **340 PB/day egress** — this is why CDN is the core of the design. (Show the order-of-magnitude math; mention most traffic is served by edge caches.)
- Concurrent viewers at peak: 1B views spread over day with peaks ~5× average → ~50M concurrent streams. Edge capacity is the constraint.

3. Architecture

```
Upload: client → resumable upload (pre-signed multipart) → raw storage
  → event → Transcode orchestrator (DAG per video)
       ├─ split into chunks (GOP-aligned) → parallel encode workers (per resolution/codec)
       ├─ audio encode, thumbnails, subtitles
       └─ package: HLS/DASH segments (2–6 s) + manifests (.m3u8 / .mpd)
  → origin store (object storage) → CDN (edge → regional mid-tier → origin shield)
Playback: client fetches manifest → picks rendition by bandwidth/buffer (ABR) → fetches segments from CDN
Metadata DB + search index + recommendation service + view counter pipeline (Kafka → batch aggregation)
```

4. Transcoding details

- Split by GOP-aligned chunks (e.g., 30 s) and encode in parallel → speed by parallelism; stitch segments.
- Encode ladder per content: complex content (sports) needs higher bitrates; simple (animation) less — per-title encoding saves 20–40% bandwidth. Mention as an optimization.
- Priority: encode the first low-res rendition fast so the video is playable quickly; higher resolutions follow.
- Cost control: encode popular-predicted ladder fully; long-tail videos get fewer renditions and encode on demand for rarely watched ones.

5. Adaptive bitrate (ABR)

- Manifest lists renditions; client measures throughput per segment and buffer level.
- Algorithms: throughput-based (EWMA of download speed) and buffer-based (BOLA, buffer thresholds). Hybrid in practice.
- Segment duration trade-off: shorter segments adapt faster but cost more requests and overhead; 2–6 s typical.

6. Deep dives

a) Egress is the cost. CDN hit ratio is the key metric. Popular content lives at edges; long tail is pulled from regional tier. Origin shield prevents thundering herd on a newly viral video (all edges missing at once). **b) Start-up latency.** Manifest and first segments cached at edge; first segment at lowest rendition; pre-fetch first segments for trending videos. **c) View counts.** 1B views/day = 11.5K writes/s. Don't write per view synchronously. Edge/clients batch view beacons; Kafka → windowed aggregation (1 min) → periodic write; displayed counts are approximate and rounded ("1.2M"). **d) Upload reliability.** Multipart pre-signed uploads with resumable parts; checksum per part; transcode starts only after full upload verified. **e) DRM and access control.** License server issues keys per session; signed URLs/tokens to CDN with short expiry; prevents hotlinking. **f) Recommendations and search are separate systems.** Keep them out of the playback path; playback uses only the manifest and CDN.

7. Failure table

Failure	Behavior
Transcode worker crash	Chunk retried; idempotent output keyed by (video, rendition, chunk)
CDN POP down	Anycast reroutes to next POP; brief rebuffering
Origin overload	Origin shield; request coalescing at mid-tier

8. Probes

- *Why not serve the original file?* Devices and bandwidth vary; ABR needs multiple renditions; codecs differ.
- *Why segments instead of one file?* Enables adaptation, caching of small pieces, and resuming.
- *Biggest cost driver?* Egress and transcoding compute; CDN hit ratio and encode ladder decisions move both.

Case 14: Distributed Job Scheduler / Workflow Engine

1. Requirements

- Schedule one-time jobs (run at time T) and recurring jobs (cron); workflows with multiple steps, retries, timeouts, and compensation.
- At-least-once execution with idempotent handlers; at most one concurrent run per job unless configured; visibility (status, history, logs).
- Scale: 100M scheduled jobs; 50K jobs starting per second at peak on top of the minute boundary (e.g., many cron jobs at `00:00`).

2. Estimates

- 100M job records × 1 KB ≈ 100 GB — fits in a sharded DB.

- Dispatch rate: 50K/s at boundaries; average much lower. Design for bursts with a queue.
- Workers: each job 1 s average → 50K concurrent executions → 500 workers with 100 concurrent tasks each.

3. Data model

- **jobs**: `job_id PK, type, payload, schedule (cron/at), next_run_at, status, attempts, lease_owner, lease_expires_at, version`.
- Index: `(status, next_run_at)` for due-job scan.
- **runs/history**: `(job_id, run_id)` with state transitions (immutable events, event-sourced).
- Partition by `job_id` hash (or tenant) for write scale.

4. Architecture (two designs)

Design A — Poll-based (simple):

```
Scheduler nodes poll: SELECT ... WHERE status='SCHEDULED' AND next_run_at <= now() LIMIT 1000 FOR UPDATE SKIP LOCKED
  → claim with lease (lease_owner, lease_expires_at = now + 30 s)
  → enqueue to task queue (Kafka/SQS/Redis streams) → workers execute
  → heartbeat extends lease; on lease expiry, job becomes claimable again
```

Design B — Time wheel / delayed queue (fine precision): - Hierarchical timing wheel in memory per scheduler node, backed by the DB for durability; jobs due in next N seconds loaded from DB into the wheel. Used when precision < 1 s and volume is huge.

Durable execution / workflows (Temporal-style):

```
Client starts workflow → workflow history persisted (event-sourced)
Workflow code (deterministic) → schedules activities → activities run on worker task queues
Activity results recorded in history; on worker crash, another worker replays history to rebuild state
Timers (sleep 7 days) are durable events in history
```

5. Deep dives

a) Exactly-once-ish execution. Leases prevent two schedulers from claiming the same job, but lease expiry during a long pause can cause double execution. Defense: fencing — each claim gets an incrementing `version`; the job's side effects use the version as an idempotency or fencing token; handlers must be idempotent. **b) Retries and backoff.** Per-job policy: max attempts, exponential backoff with jitter, non-retryable error types. Dead-lettered after max attempts with the error stored. **c) Thundering herd at minute boundaries.** Jitter cron jobs (spread start over a window) — a default of spreading by hash of job ID across 0–30 s. Protect DB from scanning at boundaries by prefetching into the queue ahead of time. **d) Clock skew.** Use DB time (`now()` on the DB) for due checks, not each scheduler's wall clock; or monotonic scheduling with leases. **e) Concurrency control per job.** "Only one run at a time": a job-level lock (lease); skip or queue the new run if the previous is still running (policy choice, state it). **f) Workflow determinism (Temporal).** Workflow code must not call non-deterministic APIs directly (current time, random, network); use the SDK's deterministic equivalents. Versioning of workflow code when running instances exist: patch markers so old histories replay correctly. **g) Visibility.** Query runs by status and time; stream state transitions to a search index; keep logs linked by run ID. **h) Multi-tenant fairness.** Per-tenant queues with weighted fair scheduling so one tenant cannot starve others.

6. Failure table

Failure	Behavior
Scheduler dies	Leases expire; another scheduler claims due jobs; slight delay
Worker dies mid-activity	Lease/heartbeat timeout; activity retried; handler idempotent
DB slow	Scheduling delayed; queue buffers; alerts on due-job lag

7. Probes

- *Cron on one server is simplest; why not?* Single point of failure, no retries, no visibility, no scale; cron is fine for a single box.
- *How do you guarantee a job runs even if the scheduler crashes just after claiming?* Lease expires → reclaim → at-least-once; handler idempotency makes it effectively once.
- *How does Temporal recover workflow state?* Event history replay with deterministic code; not by serializing memory.

Case 15: Event Pipeline / Log Analytics

1. Requirements

- Collect events from web/mobile/backend (clicks, purchases, logs); near real-time dashboards (< 1 min freshness); historical analytics over years; ad-hoc SQL.
- Exactly-once-effect in aggregates; replay for backfills and bug fixes; schema evolution.
- Scale: 1B events/day average (≈ 11.6K/s), peak 50K/s.

2. Estimates

- Event size ~500 B → 500 GB/day raw; compressed Parquet ~10x smaller ≈ 50 GB/day ≈ 18 TB/year.
- Kafka: 50K/s × 500 B = 25 MB/s peak ingress; 3 replicas, 7-day retention ≈ 25 MB/s × 86,400 × 7 × 3 ≈ 45 TB stored — plan disk accordingly (or compress and retain 3 days in Kafka, rely on lake for longer).
- Partitions: 25 MB/s ÷ ~10 MB/s per partition target ≈ 3, round up to 30–60 for consumer parallelism headroom.

3. Architecture

```
SDK/agents → Collector edge (batching, compression, auth, schema validation)
  → Kafka topics per domain (partition key = user_id or entity_id), schema registry (Avro/Protobuf)
  → Stream processing (Flink / Kafka Streams):
       - enrich (lookups), sessionize (windows keyed by user, 30 min gap), dedupe by event_id (state with TTL)
       - aggregate (1-min tumbling counts) → OLAP store (Druid/ClickHouse) for dashboards
  → Lake sink: Kafka → Iceberg (hourly commits, exactly-once via transactional/idempotent writer)
  → Batch jobs (Spark/Trino): daily aggregates, backfills, compaction
  → Query: dashboards on OLAP; ad hoc on Trino over Iceberg
```

4. Key decisions

Partition key: user_id gives per-user ordering for sessionization and even spread; a hot bot account can skew — detect and split. **Event time vs processing time:** dashboards by event time with watermark (allowed lateness 5 min for mobile); late events beyond lateness go to a correction stream and the lake (batch fix-up job). **Dedupe:** event_id from client (UUID) + server-side dedupe state with TTL = 2× max retry window (e.g., 24 h) in the stream job; or idempotent upserts in OLAP keyed by event_id. **Schema evolution:** registry with BACKWARD compatibility; add fields with defaults; never change type; breaking change = new topic version.

5. Iceberg sink details

- Writers buffer records per partition; commit a new snapshot each hour (or on size threshold) → queries see whole commits only.
- Exactly-once: store Kafka offsets in the snapshot summary (or a transactional sink) so a restart does not double-append.
- Small files: hourly commits produce many small files → compaction job merges into 256–512 MB files; sort within files by common filter columns (user_id, event_type) for pruning.
- Partitioning: `days(event_time), hour` maybe plus `event_type` bucket; avoid high-cardinality partition keys.

6. Deep dives

a) Backpressure. If the OLAP store slows, Kafka consumer lag grows (buffered, not dropped). Alert on lag growth rate; scale consumers up to partition count. **b) Replay.** Kafka retention covers recent replays; for older, replay from the Iceberg lake through the same job code (Kappa-style reprocessing with versioned jobs writing to new tables, then swap). **c) Hot partitions.** Skew from a single high-volume source: split by (entity, random salt) for raw writes; aggregate in second stage. **d) Cost.** Tiered retention: Kafka 3 days, lake forever; compress with zstd; drop debug-level events at collector for production. **e) Privacy.** PII removed or tokenized at the collector before Kafka; retention and deletion policies per topic; GDPR deletes via lake rewrite job.

7. Failure table

Failure	Behavior
Stream job crash	Restart from checkpoint; state restored; reprocess since checkpoint
Bad deploy writes wrong aggregates	Replay from Kafka retention; reprocess into new table; swap
Lake commit failure	Retry; snapshot not visible until commit succeeds
Schema-incompatible producer	Registry rejects; producer alerts; no corrupt data

8. Your project mapping

Your Kafka → Druid ingestion and Kafka → Iceberg migration map directly here. Be ready to explain: partitioning, supervisor load, compaction, why the lake is the source of truth for history, and the freshness vs cost trade-off that made you split the paths.

9. Probes

- *Exactly-once end to end?* Achieved by idempotent sinks and transactional commits within the pipeline; external side effects still need idempotency keys.
- *Why not write directly from producers to the warehouse?* Coupling, no buffering, no replay, and load spikes hit the warehouse directly.
- *Late data after dashboards are published?* Allowed lateness for near-real-time, and correction jobs for finalized daily numbers; label as “preliminary” until finalized.

14. Resume Alignment (based on your current PDF)

Source of truth: `Abhishek_Kumar_Backend_Engineer.pdf` (the version you shared). The older `.tex` file in this folder has different numbers (1B+ events/day, 10M+ files/day, a KPI formula engine). Use **only** the PDF numbers in interviews, and update the .tex if you keep using it.

1. Facts to Know Cold

Resume line	Number to know	How to explain it (be ready)
Kafka → Druid, real-time dashboards	1M+ events/day, sub-second queries	Partitioning key, consumer group size, Druid ingestion lag, query latency measurement
Temporal parallel workflows	1M+ XML files/day	Workflow per file or batch, activity retries/timeouts, task queue isolation, idempotent activities
Kafka → Iceberg migration	Druid supervisor load down 40%	Measured how? (CPU, task count, lag). Trade-off: freshness vs cost
Python compaction, dedup, rollup	up to 10x faster queries	Baseline query time vs after; small-file problem; rollup granularity
SQL query-generation service	95% less report generation time	Before/after timing; what was repeated work that got removed
Automation with Python and CI/CD	70–80% less manual effort	Which tasks, how measured (hours per week before/after)
Distributed cache (Node.js)	200 vnodes, master–slave, custom LB	Ring lookup, churn behavior, replication failure window
CamelThar	13+ REST endpoints, sub-10ms cache hits, 50ms chat delivery	Measurement method (p50/p99? where measured?)

2. Corrections to Make Before Any Interview

1. **“Zero cache invalidation”** (distributed cache). This is strong and easy to challenge. Say precisely what happened to keys on node add/remove (re-fetched from source, migrated, or lost until re-populated). Interviewers will ask.
2. **“Sub-second query latency”** — state the p95/p99 you measured and the query shape.
3. **“Ingesting 1M+ events/day”** is modest for Kafka. Be ready to say why the design still needed partitioning and idempotency (correctness, not raw volume). Do not oversell scale; match the number to the story.
4. **“AI-Engineered” and “Prompt-Driven Design” in the project and skills**: be ready to explain exactly what AI tools did in your work and what you designed yourself.
5. **Serverless + WebSockets:** CamelThar used Redis-based real-time without WebSockets. Expect “why not WebSockets?” and answer with the Vercel constraint and the trade-off (polling latency and load vs operational cost).

3. Story Map (one diagram each, 5 minutes)

- **Telecom pipeline:** Temporal orchestration → parsers → Kafka → Druid → query API; side path Kafka → Iceberg (see 03 Part C).
- **Distributed cache:** ring with 200 vnodes → shard nodes with replicas → LB routing (see 03 Part C).
- **CamelThar:** Next.js API → Redis layers (cache, rate limit, presence sorted sets, typing TTLs) → Postgres fallback (see 03 Part C).

4. Gaps an Interviewer May Probe (prepare an honest answer)

- Multi-region or large-scale ownership: you have not owned a multi-region system. Say so and describe how you would approach it (see 07 §7.7).
- Consensus and leader election: you used Temporal and Kafka, which rely on these; be able to explain the concepts (05 §5.6).
- On-call and incident leadership: prepare one real incident story (see 03 Part B).

5. Resume Keywords vs Deep Files

Keyword	Where to study

Kafka, partitions, offsets	06 §6.2
Temporal, durable execution	06 §6.4, 13 Case 14
Druid, rollup, segments	13 Case 12, 12 (metrics), 06 §6.5
Iceberg, compaction, snapshots	06 §6.5, 13 Case 15
Consistent hashing, replication	05 §5.2–5.3, 11 Case 3
Redis sorted sets, TTL, presence	06 §6.6.2, 12 Case 6 (ZSET timeline), 11 Case 5
GraphQL, Apollo, DataLoader	07 §7.1.3
Keycloak, JWT, OAuth, RBAC	07 §7.5
Zod, rate limiting	07 §7.3, 11 Case 2
Serverless, Neon	08 §8.1 (serverless), 04 §4.5 (isolation in serverless Postgres)

15. Advanced LLD: More Problems and Deeper Techniques

Read after file 02. Same method: requirements → entities → patterns → code → extensions → concurrency.

Problem A: Chess

Requirements: two players, 8x8 board, all standard piece moves, check, checkmate, stalemate, castling, en passant, promotion, undo.

Entities: `Board`, `Piece` (abstract; `Rook, Bishop, Queen, King, Knight, Pawn` subclasses), `Move`, `Player`, `Game`, `GameState`.

Patterns: Polymorphism for piece moves (each piece implements `candidate_moves`); Command for moves (enables undo); State for game status.

```
        from abc import ABC, abstractmethod
        from dataclasses import dataclass

        @dataclass(frozen=True)
        class Move:
            src: tuple[int, int]
            dst: tuple[int, int]
            promotion: str | None = None

        class Piece(ABC):
            def __init__(self, color: str):
                self.color = color
                self.moved = False

            @abstractmethod
            def candidate_moves(self, board, pos):   # pseudo-legal, ignores check
                ...

        class Rook(Piece):
            def candidate_moves(self, board, pos):
                return board.slide(pos, [(1,0),(-1,0),(0,1),(0,-1)], self.color)

        class Board:
            def __init__(self):
                self.grid: list[list[Piece | None]] = [[None]*8 for _ in range(8)]
                self.history: list[tuple[Move, object]] = []   # for undo

            def slide(self, pos, directions, color):
                """Walk each direction until edge or blocked; include capture square."""
                out = []
                r0, c0 = pos
                for dr, dc in directions:
                    r, c = r0 + dr, c0 + dc
                    while 0 <= r < 8 and 0 <= c < 8:
                        target = self.grid[r][c]
                        if target is None:
                            out.append(((r0,c0),(r,c)))
                        else:
                            if target.color != color:
                                out.append(((r0,c0),(r,c)))
                            break
                        r, c = r + dr, c + dc
                return out

            def in_check(self, color) -> bool:
                """King is attacked by any opposing pseudo-legal move."""
                king = next(((r,c) for r in range(8) for c in range(8)
                             if isinstance(self.grid[r][c], King) and self.grid[r][c].color == color), None)
                for r in range(8):
                    for c in range(8):
                        p = self.grid[r][c]
                        if p and p.color != color:
                            if any(dst == king for _, dst in p.candidate_moves(self, (r,c))):
                                return True
                return False

            def make(self, move: Move):
                (r0,c0),(r1,c1) = move.src, move.dst
                captured = self.grid[r1][c1]
                self.history.append((move, captured))
                self.grid[r1][c1], self.grid[r0][c0] = self.grid[r0][c0], None

            def undo(self):
                move, captured = self.history.pop()
                (r0,c0),(r1,c1) = move.src, move.dst
                self.grid[r0][c0], self.grid[r1][c1] = self.grid[r1][c1], captured

        class Game:
            def legal_moves(self, board: Board, color: str):
                """Filter pseudo-legal moves: a move is legal only if it doesn't leave own king in check."""
                legal = []
                for r in range(8):
                    for c in range(8):
                        p = board.grid[r][c]
                        if p and p.color == color:
                            for src, dst in p.candidate_moves(board, (r,c)):
                                board.make(Move(src, dst))
                                ok = not board.in_check(color)
                                board.undo()
                                if ok:
                                    legal.append(Move(src, dst))
                return legal
            # Checkmate = in_check and no legal moves; stalemate = not in_check and no legal moves.
```

Key points to say: pseudo-legal vs legal generation (check filter by make/undo); castling and en passant need extra state (`moved` flags, last double-step pawn); repetition and 50-move rules as extensions; performance is not the focus in LLD.

Problem B: Library Management

Requirements: books with multiple copies; members borrow up to N books for D days; reservations queue when no copy; fines for late returns; search by title/author/ISBN.

Entities: `Book` (metadata), `BookCopy` (barcode, status), `Member`, `Loan`, `Reservation`, `Fine`, `Catalog` (search index).

Patterns: Strategy for fine calculation (flat, per-day, tiered); Observer for notifying the next reservation when a copy returns; Factory for member types with different limits.

Core flow: 1. `borrow(member, copy)`: check member limit and outstanding fines; set copy `ON_LOAN`; create `Loan(due=now+D)`. 2. `return_copy(copy)`: close loan; compute fine via strategy; if reservation queue non-empty, mark copy `RESERVED_FOR(next)` and notify. 3. `reserve(member, book)`: enqueue; if a copy is available, reserve immediately.

Concurrency: two members borrowing the last copy: `borrow` does a check-and-set on copy status under a lock (or conditional update). Reservation queue is a FIFO per book, guarded the same way.

Extension points: renewals (limit count), different loan periods per member type, digital books (separate `Asset` hierarchy with license limits).

Problem C: Food Delivery (Swiggy/Zomato-style)

Requirements: restaurants have menus; customers place orders; restaurant accepts; delivery partner assigned; order status tracked; payment; ratings.

Entities: `Restaurant`, `MenuItem`, `Cart`, `Order` (state machine), `DeliveryPartner`, `Assignment`, `Payment`.

Patterns: State (`PLACED → ACCEPTED → PREPARING → READY → PICKED_UP → DELIVERED`, plus `CANCELLED`); Strategy for partner assignment (nearest, least-loaded, batch); Observer for status updates to customer/restaurant/partner; Factory or Strategy for discount rules.

Key logic: - Price calculation: sum items → apply discount strategies in defined order → taxes → fees. Keep the calculation pure (no DB) so it can be tested. - Assignment: candidate partners within radius; a partner receives one offer at a time with timeout; on reject, move to next. Same conditional-assignment trick as Case 8 in the HLD walkthroughs. - Cancellation rules depend on state (State pattern: cancellable before `PREPARING` only).

Concurrency: order state transitions are guarded by a version or lock; partner offers are exclusive.

Problem D: Meeting Room Scheduler

Requirements: rooms with capacity and features (projector); book a time slot; no overlaps; search rooms for a time window and capacity; recurring meetings (optional).

Entities: `Room`, `Booking(room, start, end, owner)`, `Scheduler`.

Core structure: per room, keep bookings sorted by start time (interval list). Overlap check: `new.start < existing.end and existing.start < new.end`. Binary search for insertion point → O(log n) check plus neighbor checks.

```python
import bisect
from dataclasses import dataclass

@dataclass(frozen=True, order=True)
class Booking:
    start: int
    end: int
    owner: str = ""

class Room:
    def __init__(self, room_id: str, capacity: int):
        self.id, self.capacity = room_id, capacity
        self._bookings: list[Booking] = []          # sorted by start

    def is_free(self, start: int, end: int) -> bool:
        i = bisect.bisect_left(self._bookings, Booking(start, start))
        if i < len(self._bookings) and self._bookings[i].start < end:
            return False
        if i > 0 and self._bookings[i-1].end > start:
            return False
        return True

    def book(self, start: int, end: int, owner: str) -> bool:
        if start >= end or not self.is_free(start, end):
            return False
        bisect.insort(self._bookings, Booking(start, end, owner))
        return True
```

Extensions: recurring bookings (generate instances, check all), time zones (store UTC), locking per room for concurrency, search via index on capacity and features.

Problem E: Snake and Ladder

Requirements: N players, board size configurable, dice (configurable count), snakes and ladders, winner determination.

Entities: `Board` (jump map: start → end), `Dice` (Strategy: fair, loaded, multiple dice), `Player`, `Game`.

Key logic: `new_pos = pos + roll`; if `new_pos` in jump map, apply jump; if `> N`, stay. Turn order with a queue. Winner: first to reach exactly N (or ≥ N per rule — state the rule).

Extensions: Dice strategy; undo; game-over state; persistence of game for replay.

Problem F: Multi-Level Cache (LLD + Concurrency)

Requirements: L1 in-process LRU, L2 shared (interface), loader on miss; write-through or write-behind policy; thread-safe; TTL per entry.

Patterns: Strategy (write policy), Decorator (adds TTL or metrics around a cache), Template Method (get flow: check L1 → L2 → loader → populate).

Concurrency point: single-flight loading — concurrent misses for the same key wait for one loader call instead of N loader calls. Implementation: per-key `Future` stored in a map under lock; first caller creates it and runs the loader; others wait on it.

```python
import threading
from concurrent.futures import Future

class SingleFlight:
    def __init__(self):
        self._lock = threading.Lock()
        self._inflight: dict[str, Future] = {}

    def do(self, key, loader):
        with self._lock:
            fut = self._inflight.get(key)
            leader = fut is None
            if leader:
                fut = Future()
                self._inflight[key] = fut
        if leader:
            try:
                fut.set_result(loader())
            except Exception as e:
                fut.set_exception(e)
            finally:
                with self._lock:
                    self._inflight.pop(key, None)
        return fut.result()
```

Advanced LLD Techniques (concepts to name)

- **Immutability:** prefer immutable value objects (`frozen dataclass`, `record`) for Money, Money-like amounts, Move, Booking. Reduces concurrency bugs.
- **Invariants:** enforce in constructors and methods, not in callers. Example: `Money` cannot be negative unless allowed; `Booking.start < end`.
- **Dependency injection:** pass clocks, repositories, and strategies into constructors. Makes tests deterministic (inject a fake clock).
- **Testing strategy:** unit tests per strategy; state machine transition table tests; concurrency test with many threads asserting invariants (no double booking).
- **Error handling:** domain exceptions (`SeatUnavailable`, `InvalidTransition`) instead of boolean returns when the caller must react; document which you chose.
- **Extensibility checklist:** new type of X → add a class, not an `if`; new rule → new strategy; new state → new state class and transition table entry.
- **Java mapping:** interfaces for strategies, `enum` with abstract methods for state transitions, `ConcurrentHashMap.computeIfAbsent` for single-flight-like maps, `ReentrantLock` with `tryLock` for timeouts, `record` for immutable values.

Practice Set (after file 02 problems)

1. Chess (pieces and legal move filter).
2. Library management (state + concurrency).
3. Food delivery (state machine + assignment).
4. Meeting room scheduler (intervals + bisect).
5. Snake and ladder (simple, fast warm-up).
6. Multi-level cache with single-flight. Write each from a blank file within a timer (45 min), then compare.